

M A S T E R T H E S I S

Title: Adversarial Attacks on AI-Powered Cyber Defense Systems and their Impact on Digital Forensics

submitted by:

Hemal Sai Manyam

(student number: 200390)

submitted on: 08.06.2026

Examiner: Prof.Dominik Bösl

Supervisor: Prof.Dominik Bösl (Hochschule der Bayerischen Wirtschaft)

Ralf Vollert (Allianz Technology SE)

Eidesstattliche Erklärung / Affidavit

Hiermit versichere ich, dass die vorliegende Arbeit ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt wurde. Alle Gedanken, die direkt oder indirekt aus externen Quellen stammen, sind entsprechend gekennzeichnet.

Ich bestätige, dass ich bei der Erstellung dieser Arbeit Künstliche Intelligenz-Tools zur Verbesserung der Sprache und zur Entwicklung von Ideen verwendet und die entsprechenden Tools im KI-Verzeichnis aufgeführt habe.

Ich bin mir bewusst, dass diese Arbeit auf die Verwendung nicht autorisierter Mittel überprüft und daraufhin kontrolliert wird, ob sie teilweise oder vollständig plagiiert ist. Um meine Arbeit mit bestehenden Quellen zu vergleichen, gestatte ich, dass sie in eine Datenbank aufgenommen wird und dort verbleibt, um künftig eingereichte Arbeiten auf Plagiate zu überprüfen. Weitere Rechte zur Vervielfältigung oder Verwertung werden nicht gewährt.

Diese Arbeit wurde, in gleicher oder ähnlicher Form, bisher noch nicht einer Prüfungsbehörde vorgelegt oder veröffentlicht.

[Anmerkung: Sie können die Passage „Ich bestätige, dass ich bei der Erstellung dieser Arbeit Künstliche Intelligenz-Tools zur Verbesserung der Sprache und zur Entwicklung von Ideen verwendet und die entsprechenden Tools im KI-Verzeichnis aufgeführt habe.“ auslassen, wenn Sie in der Tat nichts dergleichen verwendet haben.]

Declaration on Thesis

I hereby certify that this work was completed without the help of third parties and only with the sources and resources indicated. All ideas that originate directly or indirectly from external sources are properly denoted as such.

I acknowledge the use of artificial intelligence tools to improve the language and develop ideas in the creation of this work and have listed the respective tools in the AI directory.

I am aware that this work will be screened for the use of unauthorized means and subsequently be checked for partial or complete plagiarism. To compare my work with existing sources, I consent to it being included in a database and remaining there for the purpose of checking future submissions for plagiarism. Any other rights of reproduction or exploitation are not granted.

This work has, in the same or in a similar form, not previously been submitted to an examination authority or published.

Hemal Sai Manyam

München, 08.06.2026

Table of Contents

1. INTRODUCTION 9

1.1 The Paradigm Shift Toward AI-Driven Cyber Defense 9
1.2 Industrial AI-Based Intrusion Detection Architecture 10
1.3 Adversarial Machine Learning in Cyber Defense 12
1.4 Threat Model Formalization 12
1.5 The Forensic Integrity Problem 13
1.6 Research Question 14
1.7 Scope of Operation 14
1.8 Industrial Implementation Strategy 14
1.9 Contributions 15

2. THEORETICAL BACKGROUND 16

2.1 Artificial Intelligence in Modern Cyber Defense Architectures 16
2.2 Adversarial Machine Learning in Security-Critical Systems 20
2.3 Digital Forensics in AI-Assisted Security Environments 24

3. RESEARCH INFRASTRUCTURE OVERVIEW 29

3.1 Introduction to Infrastructure 29
3.2 Experimental Infrastructure Platform 34
3.3 Research Framework 54
3.4 Formalization of the Evidence Generation Pipeline 55
3.5 Research Variables and Experimental Design 56
3.6 Experimental Scenario Design 57
3.7 Evaluation Methodology 58
3.8 Chapter Summary 60

4. SYSTEM DESIGN AND IMPLEMENTATION 62

4.1 Design Objectives and Research Requirements 62
4.2 System Architecture and Component Integration 66
4.3 AI-Based Detection Framework 77
4.4 Experimental Design and Scenario Framework 82

5. EXPERIMENTAL RESULTS AND FORENSIC EVIDENCE EVALUATION 93

5.2 Experimental Dataset and Evidence Repository 98
5.3 Baseline Network Attack Evaluation (Scenario S1) 104
5.4 Evaluation of Adversarial Network Evasion (Scenario S2) 107
5.5 Evaluation of Authentication-Based Adversarial Scenarios (S3 and S4) 111
5.6 Evaluation of Application-Layer Adversarial Scenarios (S5, S6 and S8) 119
5.7 Cross-Layer Correlation and Forensic Contradiction Analysis 129
5.8 Quantitative Forensic Integrity Evaluation 141
5.9 Evaluation of Forensic Degradation Caused by Adversarial Attacks 150
5.10 Summary of Experimental Findings and Research Question Validation 160

6. FORENSIC REALITY DISTORTION AND EVIDENTIARY RELIABILITY IN AI ASSISTED CYBER DEFENSE SYSTEMS 168

6.1 Introduction 168
6.2 From Attack Detection to Evidence Interpretation 170
6.3 Forensic Reality Distortion 172
6.4 Cross-Layer Evidence Contradictions 176
6.5 Implications for Digital Forensic Investigations 179
6.6 Scientific Contributions of the Research 181
6.7 Limitations and Threats to Validity 184
6.8 Chapter Summary 187

7. CONCLUSIONS AND FUTURE RESEARCH DIRECTIONS 189

7.1 Research Overview 189
7.2 Answers to the Research Questions 190
7.3 Principal Findings 191
7.4 Scientific and Practical Contributions 191
7.5 Future Research Directions 192
7.6 Final Conclusion 193

8. LIST OF LITERATURE 194

8.1 Declaration on the Use of Artificial Intelligence (AI) Tools 194
8.2 Books (Monographs) 196
8.3 Journal Articles 197
8.4 Conference Papers 198
8.5 Reports 199
8.6 Standards 199
8.7 Online Sources 200

9. APPENDIX A 200

9.1 AI-Assisted Detection System Implementation 200
9.2 Application Dataset Generation 201
9.3 Authentication Dataset Generation 203
9.4 Application Log Processing Pipeline 207
9.5 Authentication Log Processing Pipeline 209
9.6 Process file 214
9.7 Application Classification Model 214
9.8 Application Model Components 216
9.9 Authentication Classification Model 216
9.10 Live Inference Engine 219
9.11 Network AI Evidence Collection 219
9.12 Experimental Environment Reset Procedure 219
9.13 Web app to collect logs 220

List of figures

Figure 1 Experimental Cyber Range Architecture 30
Figure 2 List of Virtual Machines used 34
Figure 3 Kali-Red VM overview 35
Figure 4 Ubuntu VM overview 36
Figure 5 pfSense Firewall VM overview 37
Figure 6 Wazuh-SIEM VM overview 38
Figure 7 AI-Analysis VM overview 39
Figure 8 Virtual networks overview 42
Figure 9 Virtual Network Segmentation Model 43
Figure 10 pfSense VM interface assignment 44
Figure 11 Kali VM details and rechability 46
Figure 12 pfSense GUI Dashboard 49
Figure 13 WAN firewall rules 50
Figure 14 LAN firewall rules 50
Figure 15 Secops firewall rules 51
Figure 16 Firewall-Controlled Evidence Generation Path 51
Figure 17 Suricata availability 53
Figure 18 Multi-Layer AI Cyber Defense Architecture 65
Figure 19 Detailed kali VM status 68
Figure 20 AI-Analysis VM directory structure 70
Figure 21 Ubuntu VM directory structure 72
Figure 22 AI Evidence Processing Pipeline 76
Figure 23 Ground Truth Repository 84
Figure 24 Ground Truth Example 85
Figure 25 Ground Truth to Evidence Comparison Model 85
Figure 26 Benign Scenario Results 87
Figure 27 Baseline Scan Detection Evidence 87
Figure 28 Comparison of S1 and S2 scan 88
Figure 29 Failed Login Burst Artifacts 89
Figure 30 Credential Abuse Scenario Evidence 89
Figure 31 Benign Prompt Classification 90
Figure 32 Cross-layer evidence generated by correlated authentication and application events. 91
Figure 33 Scenario-Based Evaluation Workflow 98
Figure 34 Evidence Prism Model Multi-Layer cyber defense analysis 128
Figure 35 Cross-Layer Evidence Contradiction Architecture 132
Figure 36 Forensic Reality Distortion Model 175
Figure 37 Future Research Roadmap for Trustworthy AI Forensics 193

List of Tables

Table 1 Simulated Enterprise Components and Their Purpose 30
Table 2 Security Monitoring Layers within the Experimental Infrastructure 31
Table 3 Virtual Machine Roles and Architectural Functions 42
Table 4 pfSense Interface Assignment 45

TABLE 5 HOST ADDRESSING AND REACHABILITY 47
TABLE 6 INDEPENDENT VARIABLES 56
TABLE 7 DEPENDENT VARIABLES 57
TABLE 8 EXPERIMENTAL SCENARIOS 57
TABLE 9 INFRASTRUCTURE COMPONENTS AND ARCHITECTURAL ROLES 60
TABLE 10 RESEARCH REQUIREMENTS AND CORRESPONDING ARCHITECTURAL COMPONENTS 64
TABLE 11 NETWORK DETECTION FEATURES USED BY THE RANDOM FOREST CLASSIFIER 78
TABLE 12 AUTHENTICATION BEHAVIOR FEATURES USED BY THE DETECTION MODEL 80
TABLE 13 GROUND TRUTH RECORD STRUCTURE 84
TABLE 14 EVIDENCE SOURCES OVERVIEW 96
TABLE 15 EVIDENCE SOURCES USED DURING EXPERIMENTAL EVALUATION 97
TABLE 16 NETWORK EVIDENCE REPOSITORY COMPONENTS AND FUNCTIONS 99
TABLE 17 PACKET CAPTURE EVIDENCE REPOSITORIES 100
TABLE 18 AVAILABLE SCENARIO REPOSITORIES 103
TABLE 19 BASELINE NETWORK DETECTION RESULTS 105
TABLE 20 FORENSIC ASSESSMENT OF BASELINE SCENARIO 106
TABLE 21 AUTHENTICATION-FOCUSED EXPERIMENTAL SCENARIOS 111
TABLE 22 GROUND TRUTH EVENT SEQUENCE FOR SCENARIO S3 112
TABLE 23 GROUND TRUTH EVENT SEQUENCE FOR SCENARIO S4 115
TABLE 24 COMPARATIVE ANALYSIS OF AUTHENTICATION SCENARIOS 117
TABLE 25 APPLICATION-LAYER EXPERIMENTAL SCENARIOS 119
TABLE 26 GROUND TRUTH EVENT SEQUENCE FOR SCENARIO S5(1) 120
TABLE 27 GROUND TRUTH EVENT SEQUENCE FOR SCENARIO S5(2) 122
TABLE 28 CROSS-LAYER VISIBILITY CHARACTERISTICS OF PROMPT INJECTION ATTACKS 123
TABLE 29 PROMPT INJECTION CROSS-LAYER CONTRADICTION MATRIX 124
TABLE 30 EVIDENCE SOURCES USED IN SCENARIO S8 126
TABLE 31 COMPARATIVE ANALYSIS OF APPLICATION-LAYER SCENARIOS 127
TABLE 32 TYPE I FORENSIC CONTRADICTION PATTERN 133
TABLE 33 TYPE II FORENSIC CONTRADICTION PATTERN 134
TABLE 34 TYPE III FORENSIC CONTRADICTION PATTERN 135
TABLE 35 TYPE IV FORENSIC CONTRADICTION PATTERN 136
TABLE 36 SCENARIO S1 - BASELINE SCAN 136
TABLE 37 SCENARIO S2 - SLOW SCAN 137
TABLE 38 SCENARIO S3 - FAILED LOGIN BURST 138
TABLE 39 SCENARIO S6 - PROMPT INJECTION 138
TABLE 40 SCENARIO S8 - LOGIN PLUS PROMPT 139
TABLE 41 CROSS-LAYER EVIDENCE CONSISTENCY MATRIX 140
TABLE 42 GROUND TRUTH EVALUATION STRUCTURE 142
TABLE 43 EVIDENCE COMPLETENESS SCORE INTERPRETATION 143
TABLE 44 TEMPORAL COHERENCE SCORE INTERPRETATION 144
TABLE 45 CORRELATION CONSISTENCY SCORE CLASSIFICATION 146
TABLE 46 CORRELATION CONSISTENCY SCORE INTERPRETATION 146
TABLE 47 ANALYTICAL RELIABILITY SCORE INTERPRETATION 147
TABLE 48 FORENSIC INTEGRITY EVALUATION ACROSS EXPERIMENTAL SCENARIOS 148
TABLE 49 EXAMPLE OF CROSS-LAYER EVIDENCE CONTRADICTION 151
TABLE 50 FORENSIC ASSESSMENT OF BASELINE NETWORK ACTIVITY 153

TABLE 51 AUTHENTICATION EVIDENCE PRESERVATION RESULTS 155
TABLE 52 CROSS-LAYER CONTRADICTIONS OBSERVED IN PROMPT INJECTION ATTACKS 156
TABLE 53 PRIMARY EVIDENCE SOURCES ACROSS EXPERIMENTAL SCENARIOS 158
TABLE 54 SUMMARY OF OBSERVED FORENSIC DEGRADATION PATTERNS 158
TABLE 55 NETWORK-LAYER SCENARIOS 161
TABLE 56 AUTHENTICATION-LAYER SCENARIOS 161
TABLE 57 APPLICATION-LAYER SCENARIOS 161
TABLE 58 CROSS-LAYER EVIDENCE CONTRADICTIONS SUPPORTING RESEARCH QUESTION VALIDATION 163
TABLE 59 MONITORING LAYER VISIBILITY CHARACTERISTICS 164

1. Introduction

1.1 The Paradigm Shift Toward AI-Driven Cyber Defense

The modern cyber threat landscape is characterized by automation, adaptive malware, polymorphic payloads, fileless attacks, and increasingly AI-assisted offensive tooling. Traditional signature-based intrusion detection systems (IDS), rule-based correlation engines, and static security analytics platforms are fundamentally constrained by the "closed-world assumption" - namely, that future attacks resemble past ones.

However, enterprise infrastructures now generate:

1. High-dimensional network telemetry (NetFlow, PCAP, DNS, TLS metadata)
2. Host-level behavioral traces (Sysmon, auditd)
3. Identity-centric authentication patterns
4. Cloud-native distributed event streams

The volume, velocity, and heterogeneity of this data necessitate intelligent, automated, and adaptive detection mechanisms. As a result, Artificial Intelligence (AI) and Machine Learning (ML) have become central to contemporary cyber defense architectures.

Modern AI-powered detection systems perform:

1. Supervised classification of malicious traffic
2. Unsupervised anomaly detection
3. Behavioral profiling
4. Probabilistic risk scoring
5. Event correlation across distributed sources

Yet this transition introduces a structural dependency:
security decision-making becomes probabilistic and model-driven rather than deterministic and rule-based.

This probabilistic dependency creates a new and insufficiently explored attack surface - the machine learning model itself.

1.2 Industrial AI-Based Intrusion Detection Architecture

This thesis implements and evaluates an industrial-grade AI-powered intrusion detection system (AI-IDS) deployed within a controlled enterprise-scale environment.

The architectural model under investigation is shown below.

1.2.1 Layered System Model

The system is structured across five logical layers:

(L1) Data Acquisition Layer

Let D_t denote raw telemetry at time t .

$$\mathrm{D_t} = \mathrm{N_t}, \mathrm{H_t}, \mathrm{A_t}, \mathrm{C_t} \quad (1)$$

Where:

1. N_t = network traffic features
2. H_t = host logs
3. A_t = authentication traces

A transformation function Φ converts raw telemetry $\mathbf{D_t}$ into structured feature vectors $\mathbf{X_t}$.

The feature representation can be expressed as:

$$X_t \in \mathbb{R}^n \quad (2)$$

where *n* denotes the dimensionality of the feature space.

Features include:

1. Flow duration
2. Packet entropy
3. Connection frequency
4. Byte distributions
5. Time-windowed aggregates

(L3) AI Detection Layer

The detection model is defined as:

$$\mathbf{f\theta(X_t) \rightarrow y_t} \quad (3)$$

where:

1. **θ** denotes the learned model parameters
2. $\mathbf{y_t \in \{0,1\}}$ represents a binary classification (benign or malicious)

(L4) Alert & Correlation Layer

Detected events generate:

$$\mathrm{At} = \mathrm{g(f\theta(Xt), threshold, correlation\ log\ i\ c)} \quad (4)$$

Where *ggg* produces:

1. Alerts
2. Risk scores
3. Incident chains
4. Timeline nodes

(L5) Digital Forensic Interface Layer

Forensic artifacts:

$$\mathrm{E = At, Lt, Tcorr} \quad (5)$$

Where:

1. At = alerts
2. Lt = log entries

3. $Tcorr$ = reconstructed event timeline

This thesis investigates how adversarial manipulation propagates from $Xt \rightarrow E$

1.3 Adversarial Machine Learning in Cyber Defense

Adversarial machine learning (AML) introduces input perturbations δ such that:

$$\mathrm{Xt}' = \mathrm{Xt} + \delta \tag{6}$$

subject to constraint:

$$\| \delta \| < \epsilon \tag{7}$$

but causing:

$$\mathrm{f}\theta(\mathrm{Xt}') = \mathrm{f}\theta \tag{8}$$

In cybersecurity, this translates to:

1. Malicious traffic classified as benign
2. Alert suppression
3. Risk score reduction
4. Misleading anomaly baselines

Unlike traditional exploits, adversarial attacks do not compromise system availability; they compromise system epistemology.

1.4 Threat Model Formalization

We define an adversary A with capability:

$$\mathrm{A}: \mathrm{Xt} \rightarrow \mathrm{Xt}' \tag{9}$$

subject to:

1. Black-box or limited white-box access
2. Control over traffic generation
3. No direct model parameter access

Attack objectives:

1. Evasion

$$\mathrm{f}\theta(\mathrm{Xt}') = 0 \mathrm{where} \mathrm{f}\theta(\mathrm{Xt}) = 1 \tag{10}$$

2. Poisoning

$$\theta' = \mathrm{Train}(\mathrm{D} \cup \mathrm{Dpoison}) \quad (11)$$

3. Concept Drift Manipulation

Gradual shift:

$$\mathrm{P}(\mathrm{Xt} + \mathrm{k}) = \mathrm{P}(\mathrm{Xt}) \quad (12)$$

leading to boundary shifts.

1.5 The Forensic Integrity Problem

Current AML literature focuses on:

1. Accuracy degradation
2. Robustness metrics
3. Attack success rate

However, enterprise security investigations depend on:

1. Alert completeness
2. Log consistency
3. Timeline continuity
4. Cross-source correlation integrity

Let forensic evidence quality be defined as:

$$\mathrm{F} = (\mathrm{C}, \mathrm{I}, \mathrm{T}, \mathrm{R}) \quad (13)$$

Where:

1. C = completeness
2. I = integrity
3. T = temporal coherence
4. R = reliability score

Adversarial manipulation affects:

$$\mathrm{F}(\mathrm{Xt}) \rightarrow \mathrm{F}(\mathrm{Xt}') \quad (14)$$

Even if detection accuracy drops marginally, forensic degradation may be substantial.

1.6 Research Question

Primary Research Question:

How do adversarial attacks targeting AI-powered intrusion detection systems influence the integrity, completeness, and reliability of digital forensic artifacts?

Sub-questions:

1. Does alert suppression distort attack timelines?
2. How does risk score manipulation affect investigative decisions?
3. Can forensic reconstruction detect adversarial tampering?
4. What defensive mechanisms restore evidentiary trust?

1.7 Scope of Operation

This thesis operates at:

1. AI Detection Layer (model + feature space)
2. Alert & Correlation Layer
3. Forensic Output Layer

It does not focus on:

1. Kernel-level ML exploits
2. Hardware side-channel AML
3. Cryptographic backdoors

1.8 Industrial Implementation Strategy

This thesis is implementation driven.

Phase 1 -Enterprise Lab Construction

1. Segmented VLAN network
2. Active Directory
3. Linux servers
4. Database
5. SIEM (Wazuh/ELK)
6. Traffic capture

Phase 2 -AI-IDS Development

1. Flow feature extraction
2. Supervised classifier (XGBoost / DNN)

3. Autoencoder anomaly model
4. Real-time inference pipeline
5. API integration with SIEM

Phase 3 -Baseline Forensic Benchmark

1. Simulated attacks
2. Timeline reconstruction
3. Alert chain evaluation
4. Evidence completeness scoring

Phase 4 -Adversarial Attack Injection

1. Traffic morphing scripts
2. FGSM-style perturbation simulation
3. Data poisoning retraining
4. Drift exploitation experiments

Phase 5 -Forensic Degradation Quantification

Define:

$$\Delta F = \text{Fbaseline} - \text{Fadversarial} \quad (15)$$

Measure:

1. Missing alert ratio
2. Timeline gap length
3. Correlation graph fragmentation
4. Attribution deviation

Phase 6 -Robustness & Hardening

1. Adversarial training
2. Ensemble diversification
3. Explainability verification
4. Log cryptographic integrity

1.9 Contributions

1. Formal forensic integrity degradation model
2. Industrial AI-IDS implementation framework
3. Empirical adversarial impact evaluation
4. Forensic reliability metric definition
5. Defensive architecture recommendations

2. Theoretical Background

2.1 Artificial Intelligence in Modern Cyber Defense Architectures

The integration of Artificial Intelligence into cybersecurity systems represents not merely a technological enhancement but a structural transformation in how defensive mechanisms operate. Traditional detection systems rely on static signatures, predefined rule sets, and deterministic logic. In contrast, AI-driven systems operate on statistical inference, probabilistic modeling, and adaptive learning from dynamic data streams. This distinction fundamentally alters both the operational capabilities and the trust assumptions of security infrastructures.

2.1.1 From Deterministic Detection to Statistical Inference

Conventional intrusion detection systems (IDS) are typically categorized as:

1. Signature-based systems
2. Rule-based anomaly detection systems
3. Threshold-triggered event monitors

These mechanisms assume that malicious activity can be described through known patterns or explicit behavioral deviations. However, modern attack strategies increasingly exhibit polymorphism, encrypted communication, living-off-the-land techniques, and AI-assisted automation. Under such conditions, static rules become brittle and reactive rather than predictive.

AI-based systems, by contrast, model patterns in high-dimensional feature spaces. Let $X \in Rn$ denote a feature vector derived from telemetry data. An AI detection system constructs a decision function:

$$f\theta: Rn \rightarrow 0,1 \tag{16}$$

Where θ represents learned parameters, and the output indicates benign or malicious classification.

Unlike deterministic rule engines, $f\theta$ is derived from optimization procedures over training data:

$$\theta *= arg\theta minL(f\theta(X), y) \tag{17}$$

L is a loss function and y are labeled instances.

This optimization-based foundation introduces both strength and fragility:

1. Strength, because it generalizes beyond explicit rules.
2. Fragility, because it depends on training data integrity and feature stability.

2.1.2 Architectural Positioning of AI in Security Pipelines

AI components in enterprise security architectures typically operate within layered pipelines consisting of:

1. Data acquisition and aggregation
2. Preprocessing and normalization
3. Feature construction
4. Model inference
5. Alert prioritization and correlation

The detection model does not operate in isolation. It depends on upstream transformations and influences downstream decision processes.

Formally, if Dt represents raw telemetry at time t, and Φ represents preprocessing and feature transformation:

$$Xt = \Phi(Dt) \tag{18}$$

The detection output is:

$$yt = f\theta(Xt) \tag{19}$$

However, the practical operational output is not yt alone, but rather:

$$At = g(yt, \tau, C) \tag{20}$$

where:

1. τ = thresholding mechanism
2. C = correlation logic across multiple events

Thus, AI influences not only classification but also alert semantics and incident structuring.

2.1.3 Types of AI Models Used in Cyber Defense

Modern AI-based cyber defense systems typically employ a combination of learning paradigms:

Supervised Learning

Used for known attack categories such as:

1. Brute-force authentication attempts
2. Port scanning patterns
3. Malware beaconing traffic

Models include:

1. Gradient boosting trees
2. Random forests
3. Deep neural networks

Supervised learning assumes the availability of labeled datasets and optimizes discriminative boundaries.

Unsupervised Learning

Used to detect behavioral anomalies without explicit attack labels.

Common approaches include:

1. Autoencoders
2. Isolation Forest
3. One-class SVM

Here, the objective is to model the distribution $P(X)$ of benign behavior and identify deviations:

$$Anomaly if P(Xt) < \epsilon \quad (21)$$

Unsupervised systems are particularly sensitive to distributional shifts and data contamination.

Semi-Supervised and Hybrid Models

Enterprise systems often combine supervised and anomaly detection outputs through ensemble strategies:

This layered modeling increases detection robustness but also increases attack surface complexity.

2.1.4 AI Systems as Evidence Generators

A critical yet underexplored dimension of AI integration in cyber defense is its role in forensic artifact generation.

AI outputs influence:

1. Alert severity
2. Event aggregation
3. Incident prioritization
4. Log retention triggers
5. Investigation workflows

Thus, AI does not merely detect threats; it shapes the evidentiary record of cyber incidents.

Let:

$$E = At, Lt, Tt \tag{22}$$

where:

1. At = alerts
2. Lt = logs
3. Tt = reconstructed timelines

If the model output yt changes, then E changes accordingly. This dependency introduces a structural coupling between detection of reliability and forensic integrity.

2.1.5 Trust Assumptions in AI-Driven Security Systems

Traditional rule-based systems allow for deterministic auditing of decision logic. AI systems, particularly deep models, introduce opacity due to:

1. Non-linear feature interactions
2. High-dimensional decision boundaries
3. Probabilistic confidence outputs

This opacity complicates:

1. Model interpretability
2. Decision reproducibility
3. Forensic validation

Moreover, AI models operate under implicit assumptions:

1. Training data reflects future distributions.
2. Feature space remains stable over time.
3. Adversaries do not intentionally manipulate input distributions.

When these assumptions fail, model reliability degrades - sometimes without visible operational failure.

2.1.6 Structural Vulnerabilities Introduced by AI Integration

The integration of AI into cyber defense introduces novel systemic vulnerabilities:

1. Dependence on data quality
2. Sensitivity to feature manipulation
3. Vulnerability to distribution shifts

4. Dependence on threshold calibration
5. Hidden decision boundary exposure

These vulnerabilities differ fundamentally from traditional software flaws. They do not require code execution exploits; rather, they exploit statistical properties of learned representations.

This shift in the vulnerability domain motivates the need for adversarial analysis, which is examined in the subsequent section.

2.2 Adversarial Machine Learning in Security-Critical Systems

The deployment of machine learning models in cybersecurity environments introduces a fundamentally new threat dimension: adversaries can manipulate not only software vulnerabilities but the statistical behavior of detection systems themselves. Unlike traditional exploitation techniques, adversarial attacks target the learned decision function rather than the execution environment.

This section formalizes adversarial machine learning (AML) within the context of AI-driven intrusion detection systems and establishes the analytical foundation for the experimental work presented later in this thesis.

2.2.1 Formal Model of Adversarial Manipulation

Let $X \in Rn$ represent a feature vector derived from telemetry data and $f\theta$ denote a trained detection model.

The classification output is:

$$f\theta(X) = y \in 0,1 \tag{23}$$

An adversarial attack introduces a perturbation δ such that:

$$X' = X + \delta \tag{24}$$

subject to a constraint:

$$\| \delta \| \leq \epsilon \tag{25}$$

where ϵ is sufficiently small to preserve functional plausibility of the traffic.

The adversarial objective is:

$$f\theta(X') \neq f\theta(X) \tag{26}$$

In cybersecurity contexts, this typically translates to:

$$f\theta(X') = 0 while f\theta(X) = 1 \quad (27)$$

That is, malicious activity is misclassified as benign.

Importantly, the perturbation δ must preserve operational constraints such as protocol compliance, traffic validity, and system usability. This distinguishes adversarial ML in cybersecurity from adversarial attacks in image classification.

2.2.2 Threat Model Taxonomy

Adversarial capabilities vary depending on the adversary's knowledge and control.

Knowledge-Based Classification

White-box attacker

1. Full knowledge of model architecture
2. Access to parameters
3. Access to gradients

Black-box attacker

1. No knowledge of internal parameters
2. Observes model outputs only
3. Learns decision boundaries via probing

In enterprise security systems, the realistic assumption is often a black-box attacker with partial observability.

Capability-Based Classification

Adversarial attacks may target different phases of the learning lifecycle:

1. Inference-time attacks (Evasion)
2. Training-time attacks (Poisoning)
3. Model extraction
4. Concept drift exploitation

This thesis focuses primarily on evasion and poisoning, as these directly influence detection and forensic outputs.

2.2.3 Evasion Attacks in Networked Systems

In evasion attacks, the adversary manipulates observable behavior to move feature representations across decision boundaries.

Let DM represent the malicious region in feature space and DB the benign region.

The decision boundary is defined implicitly by:

$$f\theta(X) = 0.5 \tag{28}$$

An evasion attack aims to move:

$$X \in DM \rightarrow X' \in DB \tag{29}$$

In network security contexts, perturbations may include:

1. Modifying packet timing
2. Injecting benign padding
3. Splitting payloads
4. Mimicking benign traffic distributions

Unlike image perturbations, these modifications must preserve protocol correctness and attack functionality.

This introduces an additional constraint:

$$Functionality(X') = Functionality(X) \tag{30}$$

Thus, adversarial manipulation operates within a constrained operational manifold.

2.2.4 Training-Time Data Poisoning

Poisoning attacks manipulate the training distribution $P(X, y)$

Let the clean dataset be:

$$D = (Xi, yi)i = 1N \tag{31}$$

An attacker injects:

$$D = (X_i, y_i)_{i=1}^{N} \tag{32}$$

Small poisoning ratios can shift decision boundaries significantly, particularly in high-dimensional feature spaces.

In intrusion detection systems that rely on periodic retraining, poisoning may gradually normalize malicious behaviors.

2.2.5 Concept Drift Exploitation

Unlike explicit perturbation, drift exploitation leverages gradual shifts in feature distributions.

Let:

$$P_t(X) \tag{33}$$

denote the feature distribution at time t .

If:

$$P_{t+k}(X) \neq P_t(X) \tag{34}$$

and the model updates based on new observations, malicious behaviors may be incorporated into the baseline.

This is particularly relevant in anomaly detection systems where “normal” behavior is continuously learned.

Drift-based attacks are subtle and often indistinguishable from legitimate environmental changes.

2.2.6 Decision Boundary Fragility in High-Dimensional Spaces

In high-dimensional feature spaces, decision boundaries can become locally unstable.

For linear classifiers:

$$f_\theta(X) = \text{sign}(w^T X + b) \tag{35}$$

Small changes along adversarial aligned directions can cause sign inversion.

For non-linear models such as deep neural networks, local linear approximations show similar vulnerability characteristics.

This fragility is exacerbated by:

1. Sparse features
2. Correlated inputs
3. Imbalanced datasets
4. High confidence calibration

Thus, robustness must be evaluated not only in terms of global accuracy but also local boundary sensitivity.

2.2.7 Security-Specific Constraints on Adversarial ML

Unlike computer vision or NLP domains, adversarial manipulation in cybersecurity must satisfy additional constraints:

1. Protocol validity
2. Timing feasibility
3. System stability
4. Operational stealth

These constraints restrict the perturbation of space but do not eliminate adversarial feasibility.

Therefore, cybersecurity AML must be evaluated under domain-specific feasibility models rather than abstract vector perturbations alone.

2.2.8 Limitations of Accuracy-Centric Evaluation

Most adversarial ML research evaluates:

1. Accuracy drops
2. Attack success rate
3. Robustness metrics

However, in operational cybersecurity systems, the more relevant metrics include:

1. Alert suppression rate
2. Severity distortion
3. Correlation fragmentation
4. Log generation variance

These metrics connect adversarial manipulation directly to forensic outcomes.

This gap motivates the forensic integrity framework introduced later in this thesis.

2.3 Digital Forensics in AI-Assisted Security Environments

The growing integration of AI-based detection systems into enterprise security operations has altered not only detection capabilities but also the structure and provenance of digital evidence. In many modern infrastructures, alerts, correlated events, and model-generated classifications form the primary basis for incident investigation. Consequently, AI systems increasingly function as implicit evidence of generators rather than merely detection tools.

This section formalizes the relationship between AI-driven detection mechanisms and digital forensic processes and identifies how adversarial manipulation introduces structural risks to evidentiary integrity.

2.3.1 Digital Evidence in Network-Centric Environments

Digital forensics traditionally focuses on the acquisition, preservation, analysis, and presentation of digital artifacts. In networked environments, relevant artifacts include:

1. Packet captures
2. Authentication logs
3. Process execution traces
4. File access records
5. Network flow metadata

However, in operational Security Operations Centers (SOC), investigators rarely analyze raw telemetry directly. Instead, they rely on pre-processed and curated artifacts generated by detection systems.

Let:

$$E_t = L_t, A_t, C_t \tag{36}$$

Where:

1. L_t = raw log entries
2. A_t = alerts generated by detection systems
3. C_t = correlated event chains

AI systems directly influence A_t and indirectly influence C_t . Therefore, forensic reconstruction increasingly depends on AI-generated outputs.

2.3.2 The Forensic Reconstruction Process

Digital forensic reconstruction aims to establish:

1. Sequence of events
2. Causality relationships
3. Scope of compromise
4. Attribution hypotheses

Formally, let a cyber incident consist of an ordered event sequence:

$$T = e1, e2, \dots, en \tag{37}$$

A forensic timeline is constructed by mapping observable artifacts to event hypotheses:

$$\hat{T} = R(E) \tag{38}$$

Where:

1. R = reconstruction function
2. E = collected evidence

If evidence is incomplete or distorted, then:

$$\hat{T} \neq T \tag{39}$$

In AI-assisted environments, E is partially shaped by model outputs. Therefore, adversarial influence on detection may propagate into the reconstruction function itself.

2.3.3 AI as an Evidence Filter

AI detection systems operate as selective filters on telemetry streams. They determine:

1. Which events are flagged
2. Which logs are enriched
3. Which activities trigger long-term storage
4. Which sequences are prioritized

Thus, AI modifies the evidentiary landscape by altering visibility.

Let raw telemetry be D_t .
The system generates filtered evidence:

$$E_t = h\left(f_\theta\left(\Phi(D_t)\right)\right) \tag{40}$$

Where:

1. Φ = feature transformation
2. f_θ = detection model
3. h = alert and correlation logic

This composition implies that evidence is not a direct reflection of activity but a model-mediated projection.

When adversarial perturbation modifies f_θ, it indirectly modifies E_t .

2.3.4 Forensic Integrity Dimensions

Traditional digital forensics emphasizes evidence integrity in terms of:

1. Authenticity
2. Chain of custody
3. Cryptographic validation

However, AI-assisted systems introduce additional dimensions of integrity:

1. Completeness

Does the evidence capture all relevant malicious events?

Let:

$$C = \frac{|E_{relevant}|}{|E_{actual}|} \tag{41}$$

If adversarial evasion suppresses alerts, completeness decreases.

2. Temporal Coherence

Does the evidence preserve correct chronological ordering?

If certain alerts are missing or mis-timestamped, the reconstructed sequence becomes inconsistent.

3. Correlation Consistency

Are event relationships preserved?

Correlation engines depend on alert presence. Missing intermediate nodes can fragment incident graphs.

4. Analytical Reliability

Does the reconstructed narrative align with actual system behavior?

This depends not only on data availability but also on model calibration and alert severity interpretation.

2.3.5 Structural Vulnerability of Forensic Pipelines

In AI-assisted SOC pipelines, the following dependency chain exists:

$$D_t \rightarrow X_t \rightarrow f_\theta(X_t) \rightarrow A_t \rightarrow C_t \rightarrow \hat{T} \tag{42}$$

Each stage compounds uncertainty.

Adversarial manipulation at stage f_θ affects all downstream stages.

Unlike traditional forensic tampering, adversarial ML attacks do not alter stored logs directly; they alter which logs are generated or emphasized.

This distinction is critical:

1. Traditional tampering modifies evidence of post-event.
2. Adversarial ML modifies evidence of generation at event time.

Thus, adversarial attacks represent a proactive distortion of future forensic artifacts.

2.3.6 Alert Suppression and Forensic Blind Spots

Consider an attack sequence:

$$e_1 \rightarrow e_2 \rightarrow e_3 \rightarrow e_4 \tag{43}$$

If adversarial manipulation suppresses detection of e_2 , then:

1. Correlation logic may fail to link e_1 and e_3
2. Timeline reconstruction may misinterpret lateral movement
3. Attribution hypotheses may shift

Therefore, adversarial impact is multiplicative rather than additive.

Even minor detection degradation can produce disproportionate forensic distortion.

2.3.7 Evidentiary Confidence Under Probabilistic Detection

AI systems often output confidence scores:

$$p_t = P(y = 1 \mid X_t) \tag{44}$$

These scores influence:

1. Alert severity
2. Case prioritization
3. Investigation allocation

Adversarial manipulation may reduce p_t without flipping classification.

This creates “low-confidence alerts” that may be deprioritized, leading to delayed or incomplete investigation.

Thus, adversarial influence can manifest as subtle severity of distortion rather than binary misclassification.

2.3.8 Research Gap: From Detection Robustness to Forensic Reliability

Existing adversarial ML literature focuses primarily on classification metrics such as:

1. Accuracy
2. Precision
3. Recall
4. Robustness bounds

However, forensic processes depend on:

1. Event coverage
2. Causal consistency
3. Narrative coherence
4. Investigative interpretability

The relationship between adversarial ML and forensic reliability remains insufficiently quantified.

This thesis addresses that gap by introducing a structured forensic degradation framework and empirically evaluating adversarial impact within an enterprise-scale AI-IDS environment.

3. Research Infrastructure Overview

3.1 Introduction to Infrastructure

Modern cyber defense systems operate in complex enterprise environments where large volumes of heterogeneous data are continuously generated. Network traffic, authentication logs, system events, and application telemetry collectively form the primary data sources used by security monitoring platforms to detect malicious activity. Evaluating the effectiveness of intrusion detection systems (IDS) and analyzing adversarial attacks against such systems therefore requires an experimental environment capable of reproducing these operational conditions.

Many intrusion detection studies rely on publicly available datasets such as KDD-Cup, UNSW-NB15, and CIC-IDS2017. While these datasets are useful for training and benchmarking machine learning models, they represent static collections of network traces and do not capture the dynamic interactions that occur in real enterprise environments. In operational networks, events occur continuously and detection decisions depend on the correlation of multiple telemetry sources. Static datasets therefore cannot accurately represent how attacks propagate through real infrastructure or how security monitoring systems generate alerts and logs during active incidents.

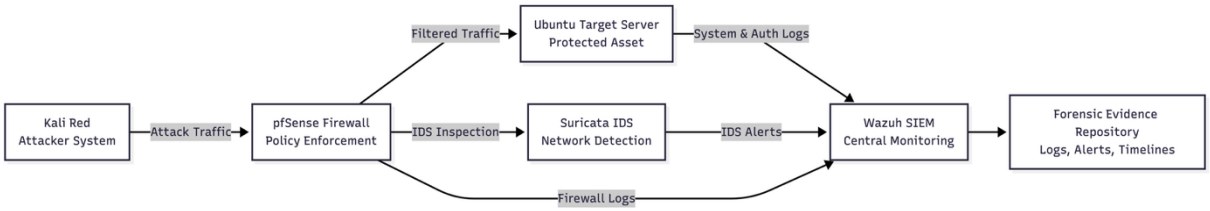
For this reason, this research implements a **controlled cybersecurity laboratory environment** that simulates a simplified enterprise network. The objective of this environment is to generate realistic telemetry that can be analyzed by intrusion detection systems and security monitoring platforms while executing controlled attack scenarios. By reproducing an enterprise-like network architecture, the research can observe how cyber-attacks interact with defensive mechanisms and how the resulting events are recorded by monitoring systems.

The experimental infrastructure was designed with three primary objectives:

1. **Generate realistic attack traffic** within a controlled network environment.
2. **Observe the behavior of security monitoring systems**, including firewall enforcement, intrusion detection mechanisms, and centralized monitoring platforms.
3. **Collect forensic artifacts generated by these systems** in order to analyze how detection mechanisms influence digital evidence and incident reconstruction.

To achieve these objectives, the laboratory environment simulates several key components commonly found in enterprise cybersecurity architectures.

Figure 1 Experimental Cyber Range Architecture



Simulated Enterprise Components

The infrastructure includes multiple functional components that collectively reproduce a simplified cyber defense architecture.

Table 1 Simulated Enterprise Components and Their Purpose

Component	Purpose
Attacker system	Generates simulated cyber attacks
Enterprise hosts	Target systems used during attack simulations
Firewall gateway	Controls network traffic and enforce security policies
Network intrusion detection system	Monitors network traffic for malicious patterns
Security monitoring platform	Aggregates logs and security alerts

Each component represents a critical element of modern enterprise security infrastructure. Together, they form an integrated environment capable of producing realistic telemetry for experimental analysis.

Enterprise Network Simulation

The laboratory environment simulates an enterprise network in which internal systems communicate through a centralized gateway while security monitoring tools observe network activity. The network contains several virtual machines representing typical enterprise systems such as servers, workstations, and monitoring infrastructure.

These systems interact with each other during normal operation while also serving as targets for simulated cyber-attacks generated by the attacker system. This approach allows the research to observe how attacks propagate through the network and how monitoring systems respond to malicious activity.

The enterprise environment is implemented using a **virtualized infrastructure platform**, which hosts multiple virtual machines representing different roles within the network. Virtualization enables the entire experimental infrastructure to run on a single physical host while maintaining logical separation between systems.

The simulated enterprise network includes the following elements:

1. Application servers hosting services
2. Workstation systems representing user endpoints
3. Monitoring infrastructure responsible for security event analysis
4. Network security controls enforcing traffic policies

This environment enables realistic attack execution while allowing monitoring systems to capture and analyse the resulting activity.

Security Monitoring Stack

Modern enterprise security architectures rely on **multiple monitoring layers** to detect malicious activity. Instead of depending on a single detection mechanism, organizations deploy several complementary technologies that analyse different aspects of system behavior.

The experimental infrastructure replicates this layered defense model by incorporating the following monitoring components.

Table 2 Security Monitoring Layers within the Experimental Infrastructure

Monitoring Layer	Function
Firewall	Controls network traffic and enforce access policies

Network IDS	Inspects network packets for malicious patterns
SIEM platform	Aggregates and correlates security events

The firewall acts as the central network gateway through which all traffic flows. This design allows the monitoring infrastructure to observe both legitimate communication and malicious activity generated during attack simulations.

The network intrusion detection system analyses traffic passing through the network and produces alerts when suspicious patterns are detected. These alerts provide an initial indication of potential malicious activity.

The SIEM platform collects logs and alerts from multiple systems within the infrastructure, enabling event correlation and supporting digital forensic analysis.

Generation of Security Telemetry

One of the key advantages of the experimental infrastructure is the ability to generate **live security telemetry**. When attacks are executed within the network, they produce several types of observable events including:

1. Network traffic flows
2. Intrusion detection alerts
3. Firewall log entries
4. Host system events
5. Authentication attempts

These events collectively form the **security artifacts** used by monitoring systems to detect and investigate cyber incidents.

Unlike static datasets that contain pre-recorded traffic, the laboratory environment produces telemetry dynamically as systems interact with each other. This enables the research to observe how detection systems respond to attacks in real time and how the resulting events are recorded by monitoring platforms.

Experimental Cyber Range Environment

The experimental infrastructure can be viewed as a **cyber range**, a controlled environment used to simulate cyber-attacks and defensive responses. Cyber ranges are widely used in cybersecurity research because they allow offensive and defensive systems to interact safely without affecting external networks.

In this research, the cyber range consists of multiple virtual machines connected through a virtualized network architecture. These machines simulate both attacker systems and enterprise hosts while monitoring tools analyse the generated activity.

The virtualized environment provides several important advantages:

Isolation -experiments remain confined to the laboratory environment.
Reproducibility -system snapshots allow experiments to be repeated under identical conditions.
Flexibility -additional systems can be added to extend the infrastructure as needed.

These properties ensure that experimental results remain reliable and reproducible.

Relationship to the Research Objective

The experimental infrastructure plays a central role in enabling the empirical analysis conducted in this thesis. By simulating an enterprise security environment, the laboratory allows the research to observe how cyber-attacks interact with intrusion detection systems and security monitoring platforms.

The infrastructure enables observation of the complete monitoring pipeline. Through this pipeline, malicious activity executed within the environment produces the evidence that investigators would typically analyse during incident response.

By examining these artifacts, the research can evaluate how adversarial manipulation of network traffic affects both detection systems and the integrity of the resulting forensic evidence.

At the current stage of the research, the infrastructure focuses on rule-based detection mechanisms and centralized monitoring systems. These components provide the baseline telemetry required for subsequent experimental phases. In later stages of the research, this monitoring pipeline will be extended with **AI-based intrusion detection components**, enabling the study of adversarial attacks targeting machine learning–driven cyber defense systems.

Summary

This section introduced the experimental infrastructure used in this research. The laboratory environment simulates a simplified enterprise network containing attacker systems, internal hosts, and layered security monitoring components. By generating live telemetry within this controlled environment, the research can observe how cyber-attacks interact with intrusion detection systems and how these interactions influence the forensic artifacts produced by monitoring platforms.

The following sections describe the technical implementation of this infrastructure in detail, including the virtualization platform, network architecture, and security monitoring components deployed within the experimental environment.

3.2 Experimental Infrastructure Platform

The experimental environment implemented for this research was designed as a controlled cyber range capable of reproducing the essential properties of an enterprise security architecture while remaining sufficiently observable for rigorous experimental analysis. The primary purpose of this platform was to provide a technically realistic environment in which attack traffic, network policy enforcement, intrusion detection, centralized monitoring, and resulting forensic artifacts could be studied as parts of a single operational system rather than as isolated components.

In contrast to evaluation approaches based solely on pre-recorded datasets, the implemented infrastructure allows security-relevant events to emerge dynamically from interactions between systems. This distinction is important for the present research. The objective is not merely to determine whether an attack would match a signature or trigger a classifier, but to examine how attack activity traverses infrastructure, how different defensive layers respond, and how these responses become logs, alerts, and evidence used during incident reconstruction. Such questions require a live environment with routable hosts, administrative controls, monitoring components, and reproducible traffic paths.

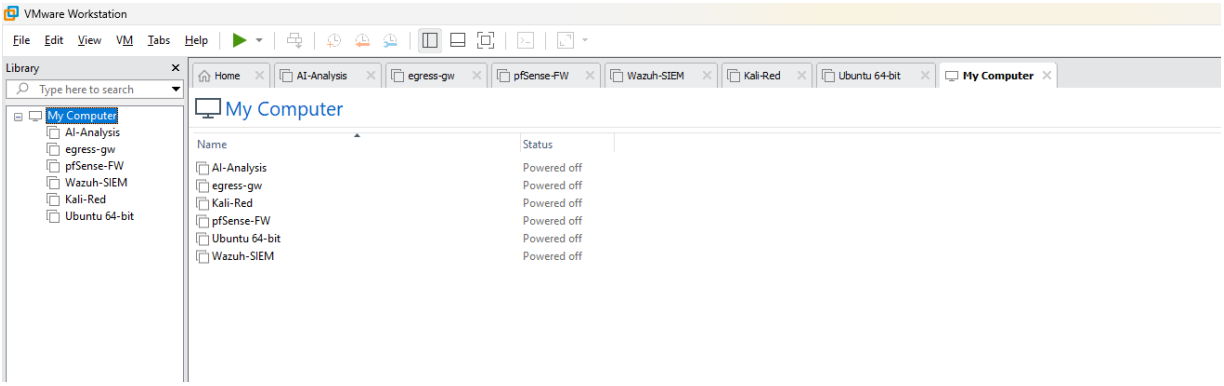
The implemented platform therefore combines virtualized hosts, segmented network zones, a stateful firewall, a network intrusion detection layer, and a centralized security monitoring stack. These elements together create the baseline cyber defense environment required for later adversarial experimentation against AI-based detection components.

3.2.1 Implemented Virtualization Platform

The current baseline infrastructure was implemented in **VMware Workstation**, as evidenced by the virtual machine library and virtual network editor configuration captured during deployment. The use of desktop virtualization was appropriate for this phase of the research because it enabled rapid prototyping, strict local isolation, reproducible network configuration, and controlled execution of attack scenarios without requiring external infrastructure.

Figure 3.2 should therefore present the virtualization platform first, before discussing any individual machine roles.

Figure 2 List of Virtual Machines used



From the virtualization library, four principal machines are visible:

1. Kali-Red
2. Ubuntu 64-bit
3. pfSense-FW
4. Wazuh-SIEM
5. AI-Analysis

Figure 3 Kali-Red VM overview

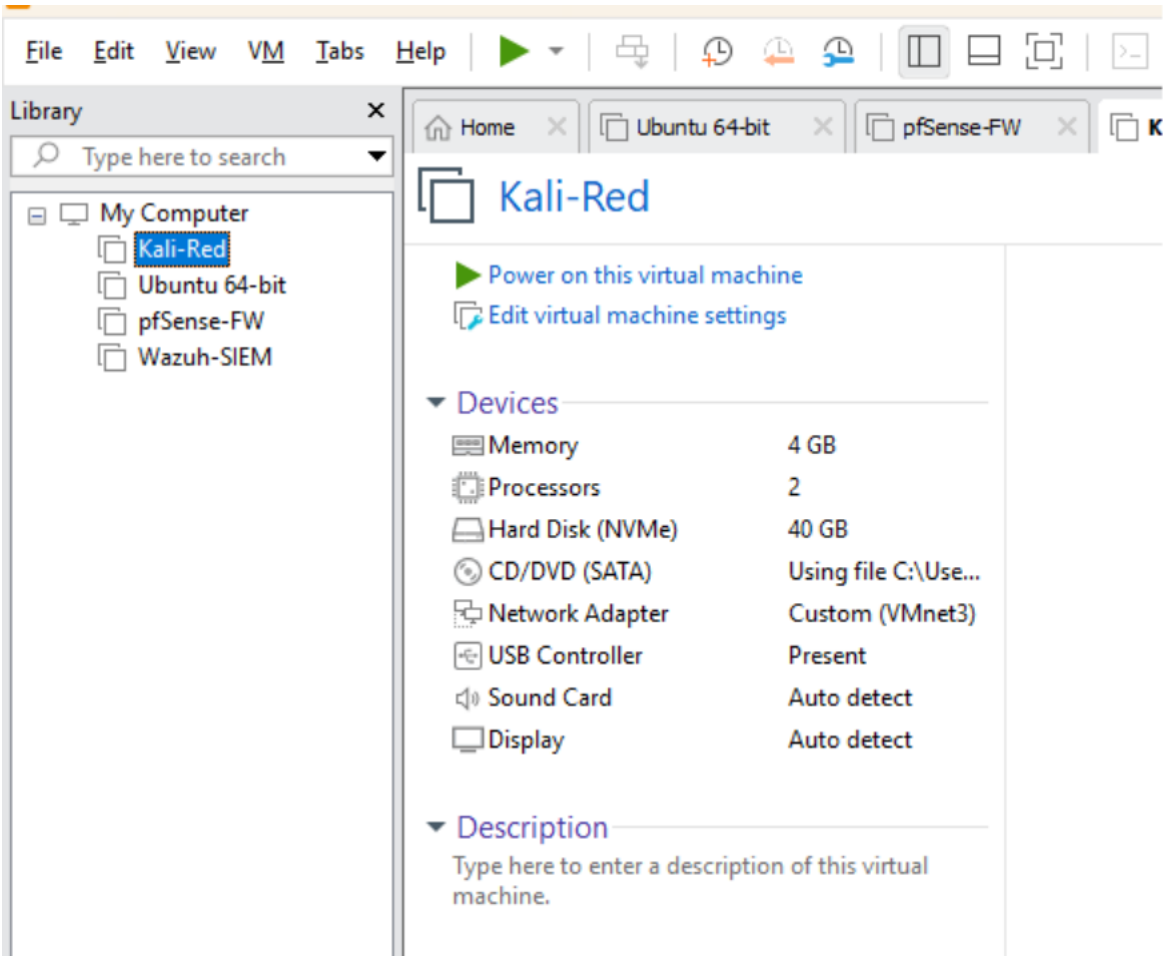


Figure 4 Ubuntu VM overview

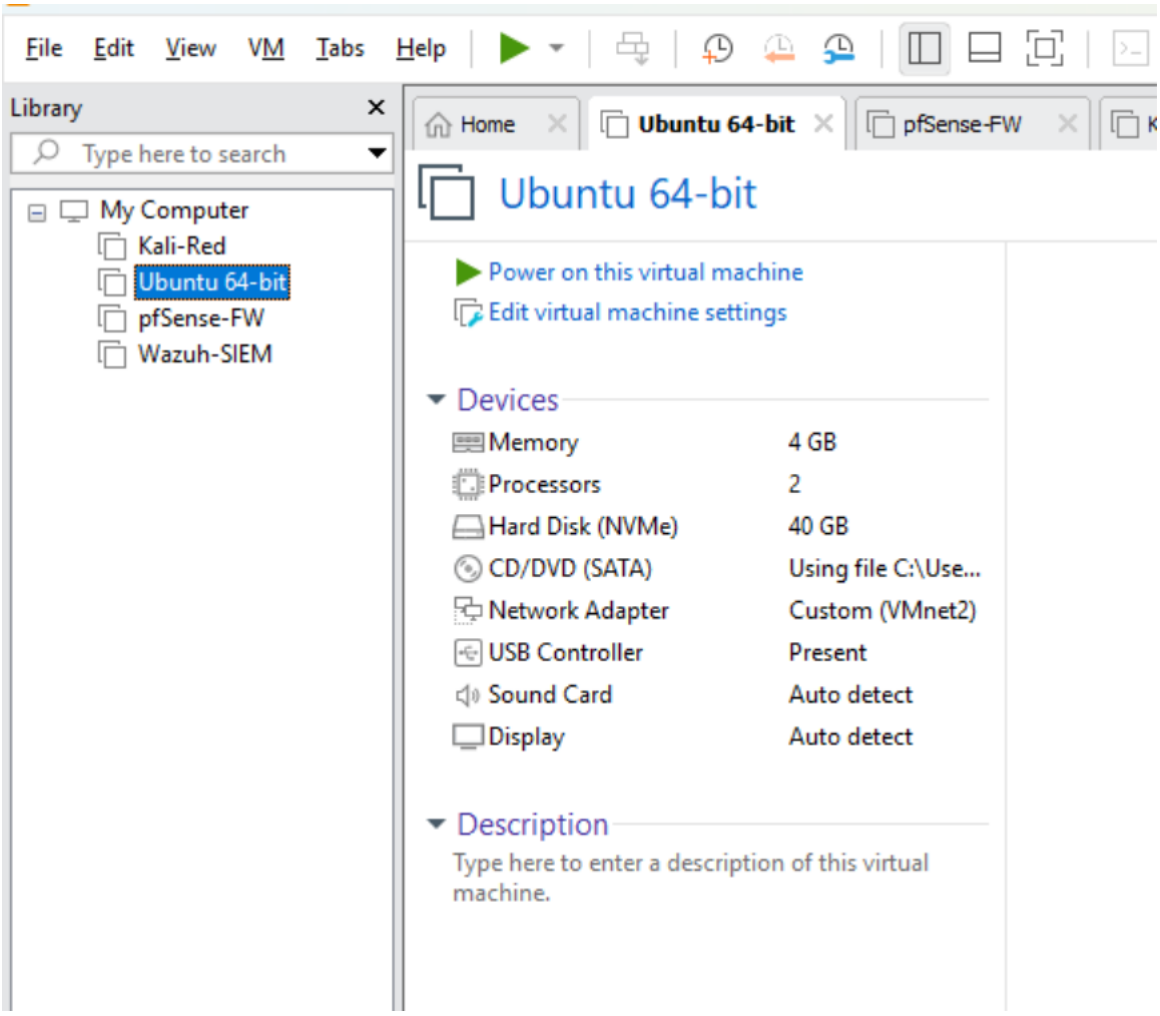


Figure 5 pfSense Firewall VM overview

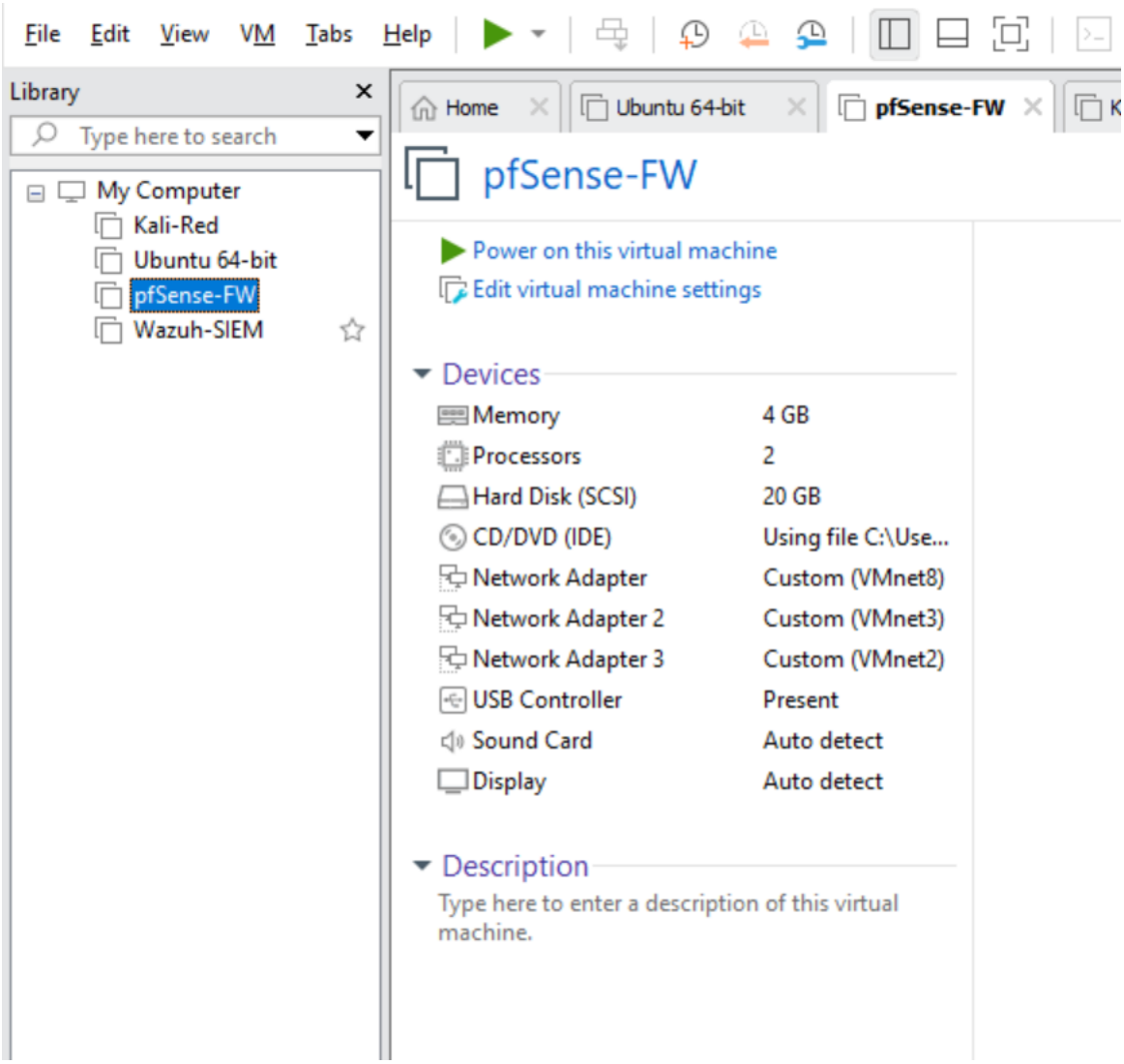


Figure 6 Wazuh-SIEM VM overview

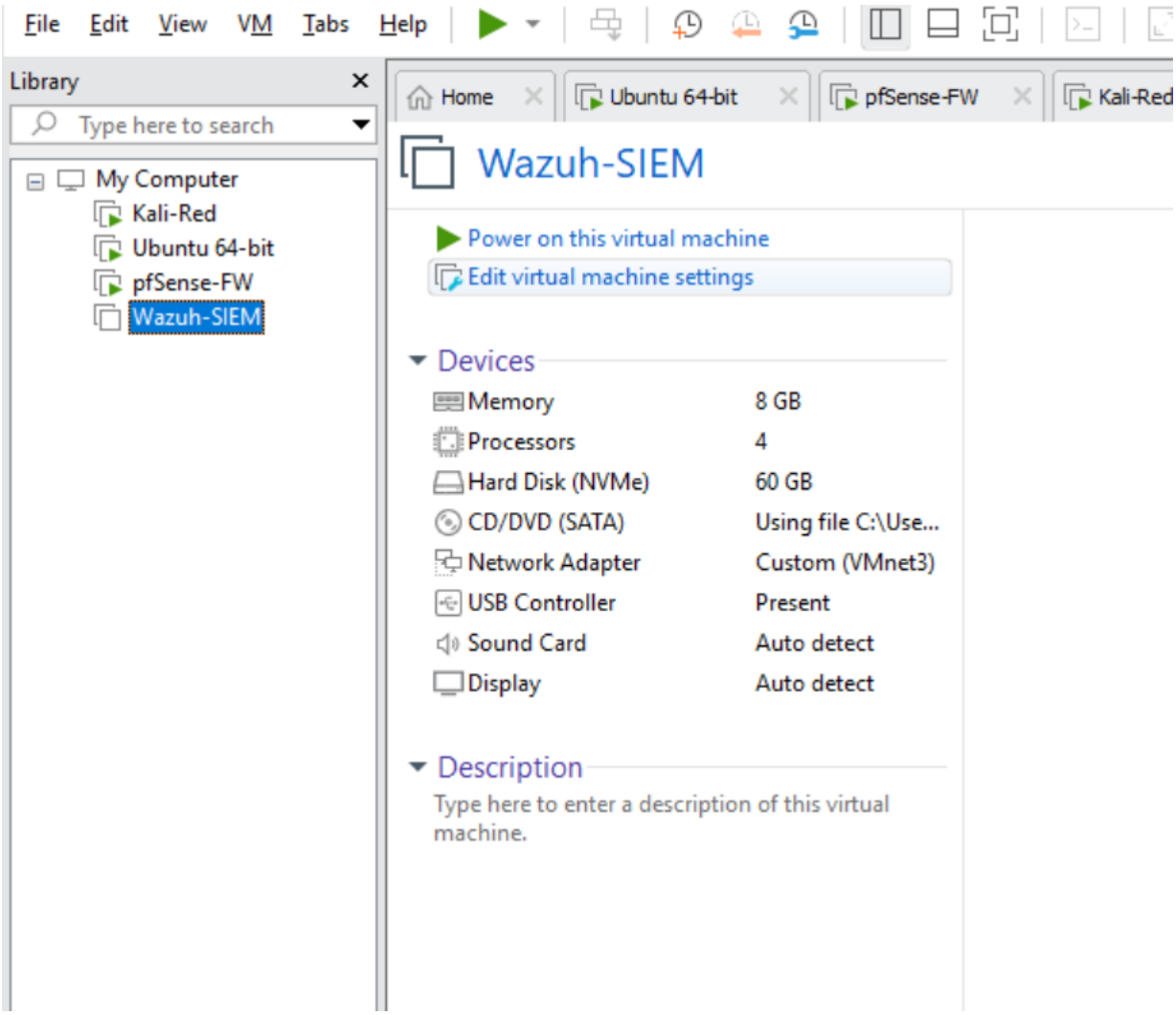
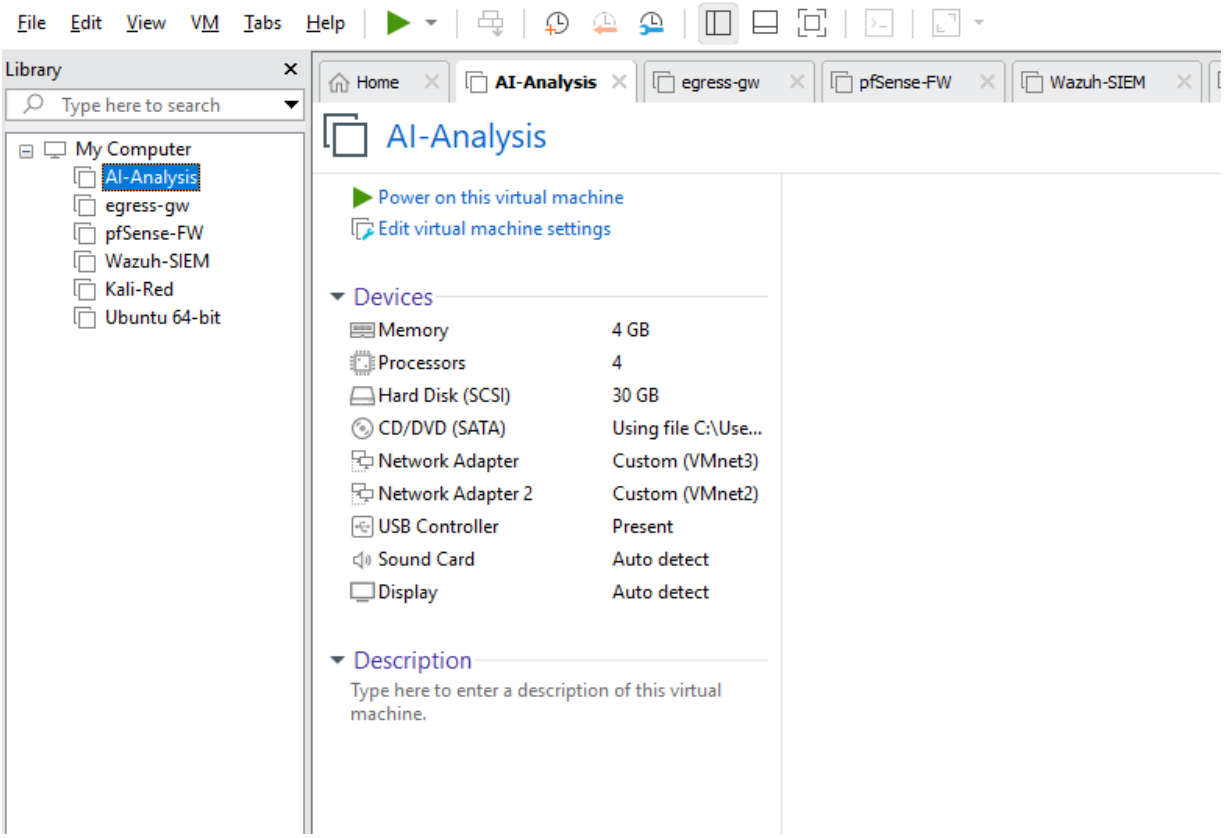


Figure 7 AI-Analysis VM overview



These systems were instantiated as separate virtual machines in order to preserve role separation. Such separation is methodologically important. If the attacker, firewall, monitored hosts, and SIEM platform were combined into fewer systems, the resulting telemetry would be less realistic and the causal chain between attack execution, network traversal, defensive processing, and evidence generation would become blurred. By assigning these roles to distinct machines, the infrastructure preserves the same architectural logic found in operational enterprise networks, namely that traffic originates on one host, is routed or filtered by another, affects a third, and is observed by an independent monitoring layer.

The resource allocation visible in the screenshots also reflects this role-based design. The attacker and standard hosts were allocated moderate resources sufficient for tool execution and service hosting, whereas the Wazuh monitoring server was provisioned with higher memory and CPU because centralized logging, indexing, and dashboard services are substantially more resource-intensive than simple client operations.

3.2.2 Virtual Machine Roles and System Separation

The next part of the documentation should introduce the machines in terms of architectural function rather than in the order they were created.

Kali Linux attacker system

The **Kali-Red** virtual machine serves as the primary offensive node in the environment. Its role is to generate controlled malicious traffic, including reconnaissance, session establishment attempts, and later attack activity directed toward protected systems. The screenshot shows that the machine was configured with 4 GB of memory, 2 processors, a 40 GB disk, and a network adapter attached to **VMnet3**. The purpose of this machine is not simply to act as a penetration testing host, but to provide an externalized traffic source whose behavior can be observed by intermediate security controls.

The practical value of this design is methodological: attacks must originate from a host other than the firewall and other than the monitored server. Only then can the experiment capture a proper telemetry chain from source system to gateway to target to SIEM.

Ubuntu server

The **Ubuntu 64-bit** virtual machine acts as an internal system within the laboratory. The screenshot shows that it was configured with 4 GB of memory, 2 processors, a 40 GB disk, and a network adapter attached to **VMnet2**. In architectural terms, this system represents an internal enterprise asset that can be administered, monitored, and potentially targeted during experimental scenarios.

Even if the precise application-layer workload on the Ubuntu host is not yet the central subject of this section, its placement is important because it creates a protected internal endpoint behind the firewall. This allows the research to study not only whether traffic is generated, but whether access to an internal asset is allowed, denied, selectively exposed, or logged under specific policy conditions.

pfSense firewall and gateway

The **pfSense-FW** virtual machine is the structural center of the environment. Unlike the other machines, it is multi-homed and participates in multiple networks simultaneously. The screenshot shows three separate network adapters attached to **VMnet8**, **VMnet3**, and **VMnet2**, which means pfSense performs routing and filtering between three network spaces rather than acting as a simple endpoint.

This design is crucial. Without a dedicated gateway, traffic between virtual machines would remain flat, and there would be no meaningful policy enforcement point. By introducing pfSense as a dedicated firewall/router, the laboratory reproduces one of the most important characteristics of enterprise network defense: the existence of a chokepoint where flows are segmented, filtered, logged, and inspected.

Wazuh SIEM server

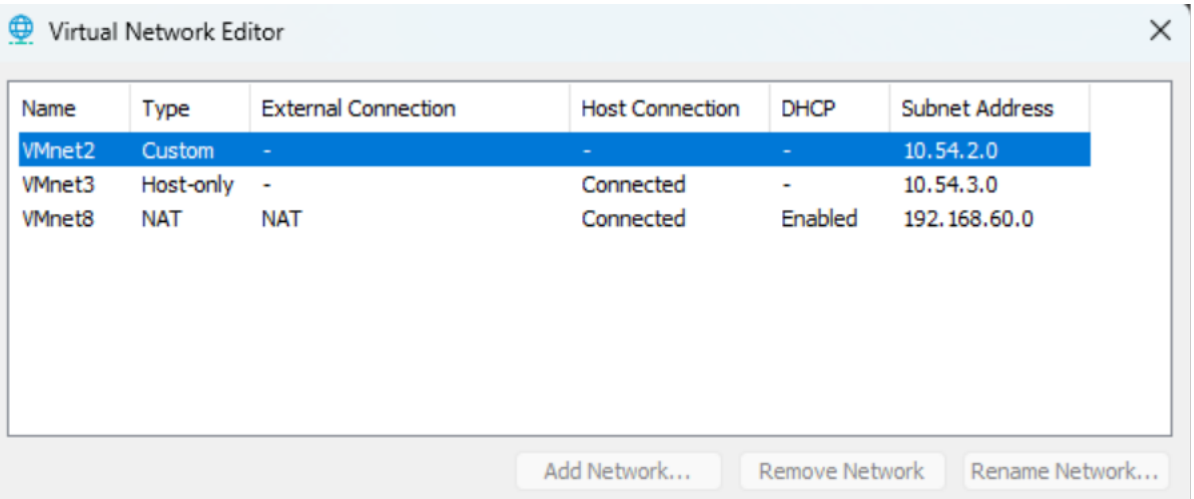
The **Wazuh-SIEM** virtual machine acts as the centralized monitoring platform. Its higher allocation of 8 GB of memory, 4 processors, and a 60 GB disk reflects the heavier workload associated with log ingestion, indexing, dashboard rendering, and security event analysis. The machine is attached to **VMnet3**, placing it within the same logical segment as the firewall LAN interface and the attack host visible in the current baseline screenshots.

This monitoring node is important because the research question is not limited to whether an attack occurs but extends to the forensic consequences of that attack. Wazuh provides the central point at which security-relevant telemetry is aggregated and made analyzable.

3.2.3 Virtual Network Construction and Address Spaces

Once the machine roles are introduced, the next step is to explain the virtual network design. This is where the Virtual Network Editor screenshot should be inserted.

Figure 8 Virtual networks overview



The screenshots show three relevant virtual networks:

Table 3 Virtual Machine Roles and Architectural Functions

Virtual network	Type	Subnet	Functional interpretation
VMnet2	Custom	10.54.2.0	Protected internal segment
VMnet3	Host-only	10.54.3.0	Local lab operations / monitored segment

VMnet8	NAT	192.168.60.0	Upstream external connectivity

This network structure is technically meaningful for several reasons.

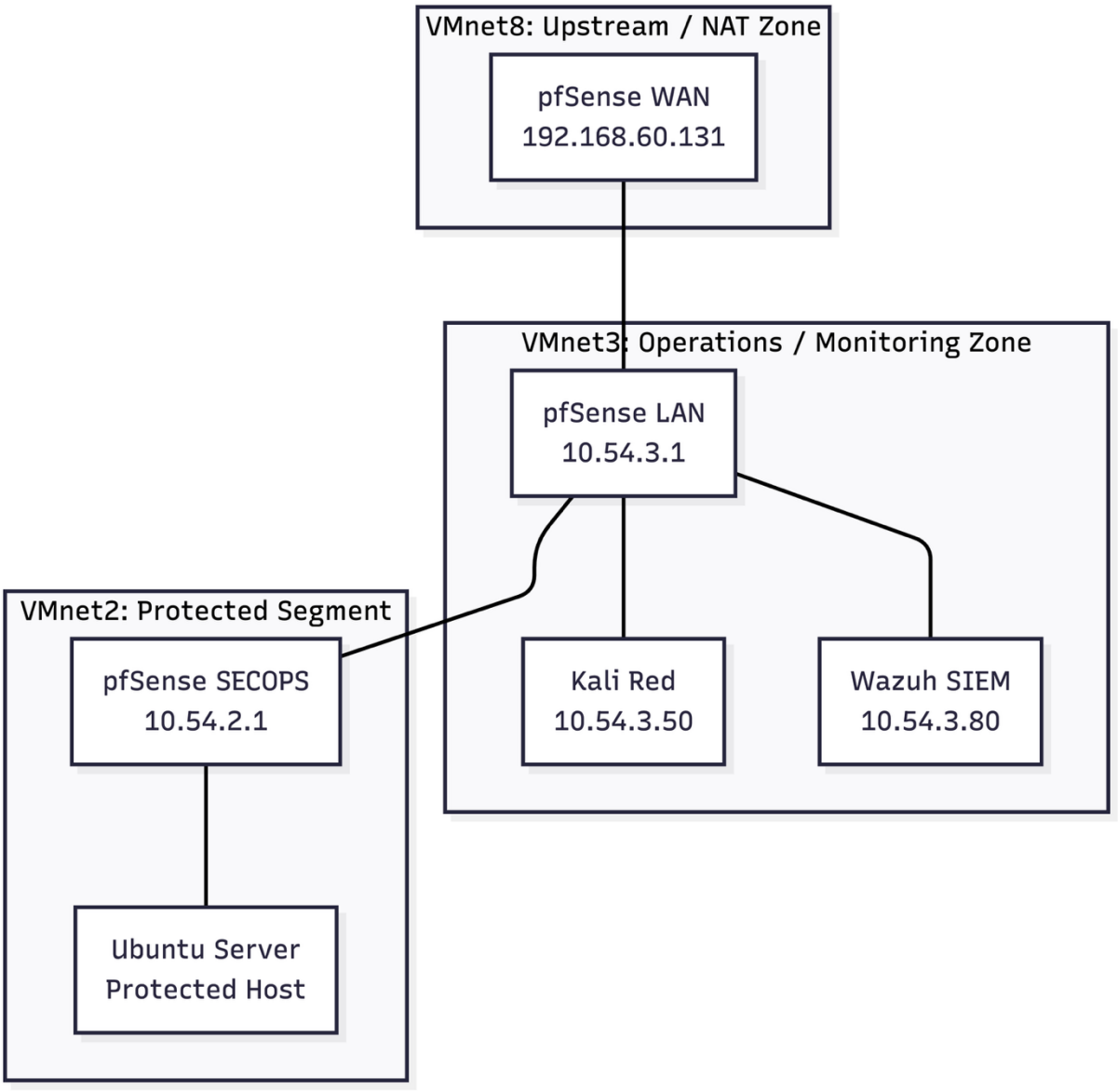
First, the **NAT-connected VMnet8** provides the firewall with an upstream-facing interface. This makes it possible to model an external-facing side of the environment without directly exposing the lab to the physical enterprise network. From a research perspective, this enables a bounded notion of “outside” traffic while preserving host isolation.

Second, **VMnet3** functions as a lab-accessible operational network. The screenshots show both the Kali system and the Wazuh server on this subnet, and pfSense uses this segment as its **LAN** interface. This means that VMnet3 is the primary segment from which the gateway is administered and from which at least part of the monitored traffic originates in the current phase of the setup.

Third, **VMnet2** is configured as a separate custom subnet and is attached to pfSense as **SECOPS**. Because Ubuntu is also attached to VMnet2, this creates a second protected segment distinct from the pfSense LAN. This separation matters because it allows controlled inter-zone policy. Rather than allowing unrestricted host-to-host communication across all machines, the infrastructure can selectively permit or deny flows between VMnet3 and VMnet2 according to firewall policy.

This is precisely the kind of segmentation that makes the environment suitable for cyber defense experimentation. Segmentation is not merely a network management convenience; it is a research enabler. It allows the thesis to examine how different classes of traffic behave when crossing a routed security boundary, how logs are created at those boundaries, and how defensive visibility changes depending on placement of the traffic source and destination.

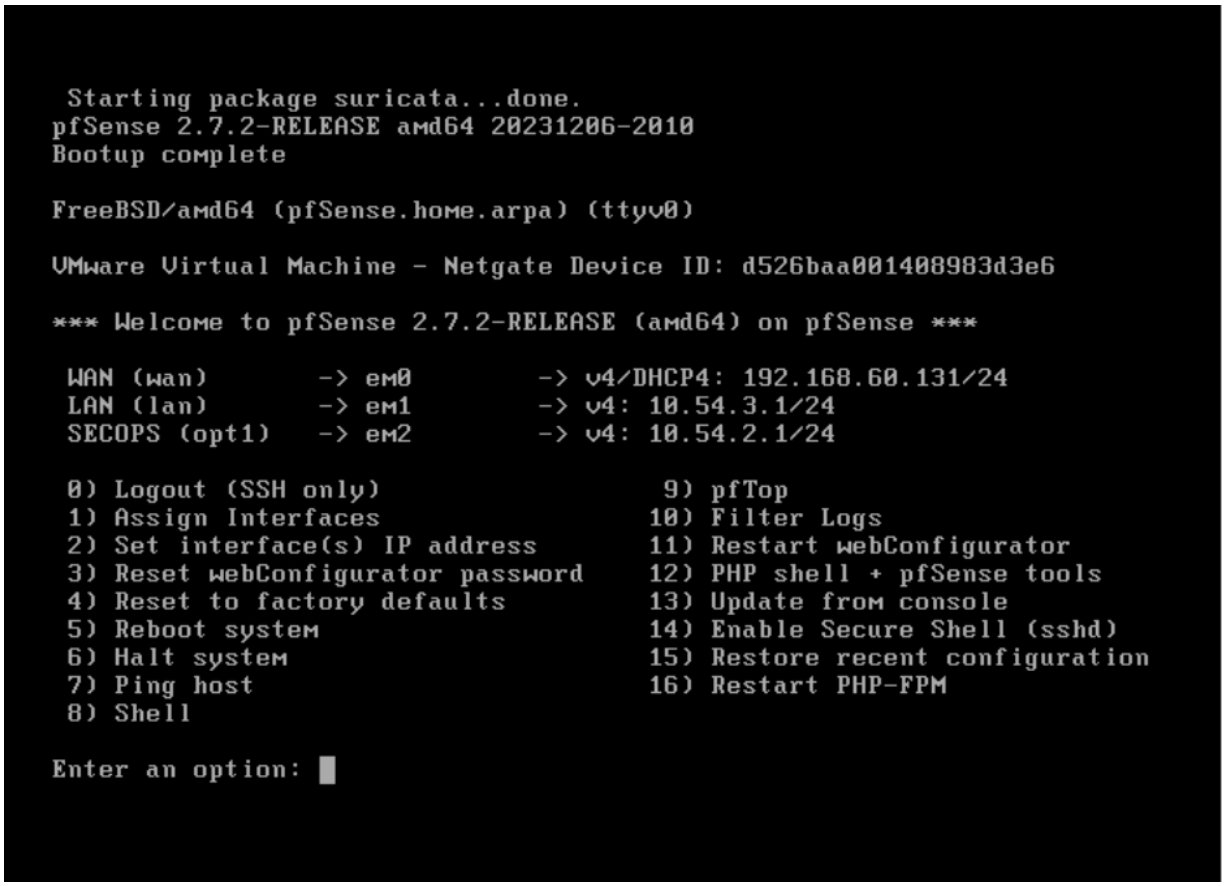
Figure 9 Virtual Network Segmentation Model



3.2.4 Firewall Interface Assignment and Layer-3 Design

After network construction, the thesis should move to the firewall's interface assignments, because they formalize the logical topology into a routable architecture.

Figure 10 pfSense VM interface assignment



The pfSense console screenshot shows the following active interface configuration:

Table 4 pfSense Interface Assignment

Interface	pfSense name	IPv4 address	Function
em0	WAN	192.168.60.131/24	Upstream external side
em1	LAN	10.54.3.1/24	Primary internal routed segment
em2	SECOPS (opt1)	10.54.2.1/24	Secondary protected segment

This interface structure demonstrates that pfSense is not being used merely as a packet filter attached to one segment. It is being used as an actual **layer-3 gateway** between three separate network contexts. That distinction is important academically because it means the

infrastructure can support routing decisions, stateful inspection, access policy, logging, and selective inter-zone reachability.

The naming visible in the screenshots also deserves explanation. Although the label **SECOPS** is attached to the 10.54.2.0/24 network in pfSense, the current baseline topology places the Wazuh SIEM on 10.54.3.80 within the LAN segment. This indicates that the lab was configured iteratively and that the architectural naming reflects intended security roles rather than a final production-grade naming convention. In the thesis, this should be described clearly and neutrally: the experimental environment evolved during implementation, but the important point is that the firewall enforces policy across more than one internal zone, thereby enabling controlled administrative and monitored paths.

From a security architecture perspective, this three-interface design works because it separates three functions that are often conflated in smaller lab setups:

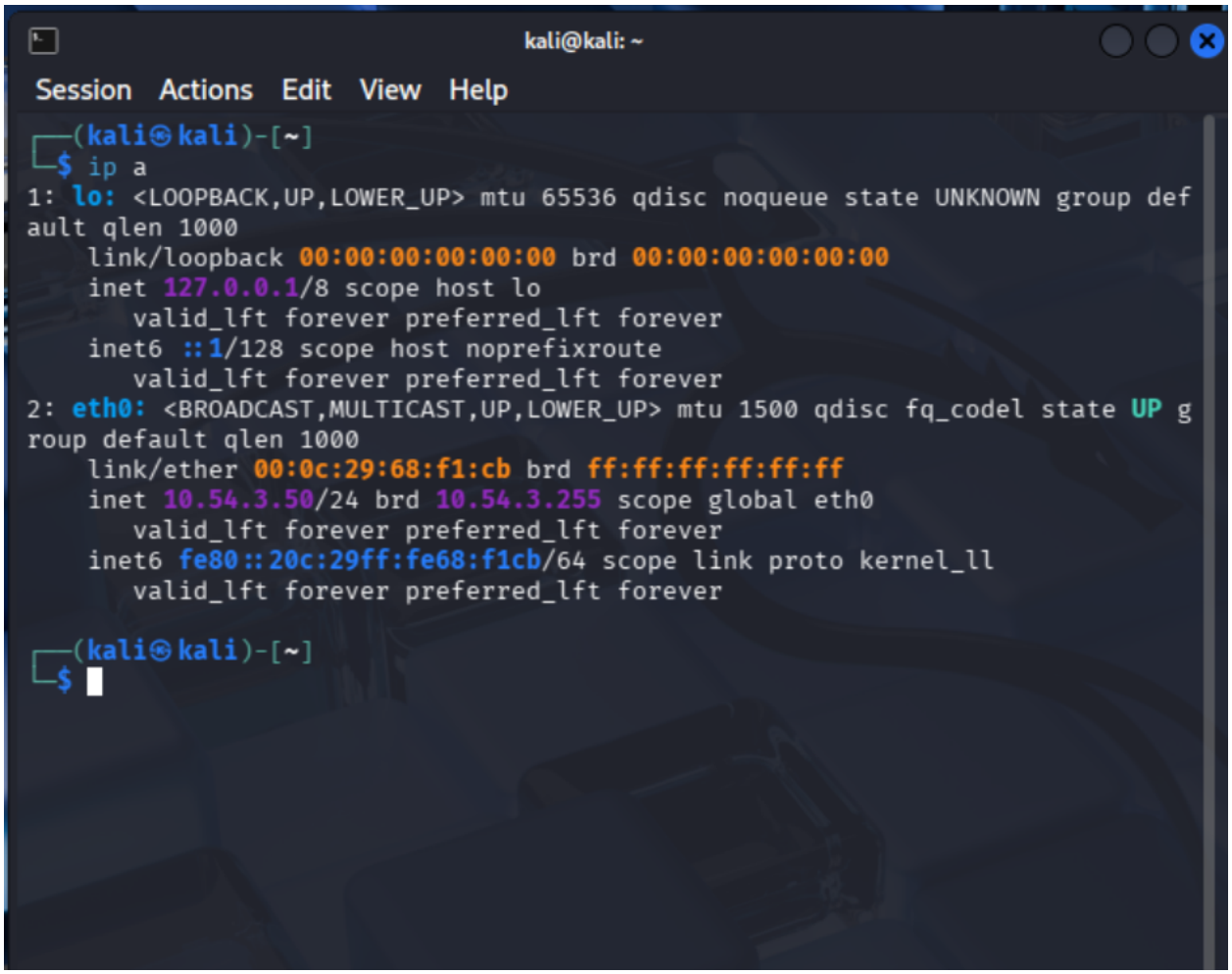
1. External/upstream access
2. Primary routed internal communications
3. Secondary protected or selectively reachable zone

By maintaining these as separate interfaces, the laboratory achieves policy granularity. That granularity later becomes essential when analyzing which evidence is produced for allowed, blocked, or anomalous flows.

3.2.5 Observed Host Addressing and Reachability

The thesis should then present the addressing evidence visible from the host consoles and dashboards.

Figure 11 Kali VM details and reachability



The screenshots confirm at least the following active addresses:

Table 5 Host Addressing and Reachability

System	Observed IP address
pfSense WAN	192.168.60.131
pfSense LAN	10.54.3.1
pfSense SECOPS	10.54.2.1
Wazuh-SIEM	10.54.3.80

Kali-Red	10.54.3.50

These observations matter because they establish that the lab is operating as a routed IP environment rather than as a purely conceptual diagram. They also show that the Wazuh interface is reachable via its local web dashboard and that pfSense is being administered through its HTTPS interface on 10.54.3.1.

In practical terms, the addressing confirms three things.

First, the firewall is correctly providing gateway IPs in both internal ranges.

Second, the Wazuh server is reachable from the management/operational side of the lab.

Third, the attacker host is not co-located on the upstream NAT interface but instead placed in an internal routed segment behind pfSense, which supports controlled east-west as well as north-south testing depending on policy design.

The Kali console screenshot additionally confirms active IP configuration on the interface and indicates that the attacker host is live on the routed subnet. The Wazuh console similarly confirms its interface address on 10.54.3.80, while the browser screenshots confirm successful access to both the pfSense web configurator and the Wazuh dashboard. These observations collectively serve as infrastructure validation evidence and should be explicitly used as such in the thesis.

3.2.6 Firewall Policy Design and Why It Matters

The next logical step is to discuss the firewall rules, because the value of the architecture lies not simply in segmentation but in **policy-controlled segmentation**.

The WAN rules screenshot shows no pass rules defined on the external-facing interface other than the default protections such as blocking RFC1918 networks and bogon networks. This means unsolicited inbound traffic from the upstream side is not freely admitted. From a defensive architecture perspective, this is the correct baseline posture. The WAN side is treated as untrusted, and the default position is denial unless an explicit exposure is required.

This matters for the thesis because it establishes that the environment is not artificially permissive. If the external side were configured to allow arbitrary inbound sessions, later conclusions about detection and forensic evidence would be distorted by weak baseline controls rather than meaningful attack interaction.

The LAN rules screenshot shows a more nuanced internal policy. In addition to the pfSense anti-lockout rule, the rules visible indicate:

1. General IPv4 allowance from LAN subnets
2. A specific rule allowing TCP from LAN subnets to **10.54.2.20** on **port 22 (SSH)**
3. A block rule involving traffic from LAN subnets toward **10.54.2.0/24**
4. A default IPv6 allowance rule

Even without over-claiming beyond the screenshot, the architectural meaning is clear: policy on the LAN side was not left entirely flat. Instead, the researcher introduced selective reachability, including a specific SSH path, while restricting broader subnet access. This is a very important design decision and should be explained in the thesis.

A completely flat network would have been simpler to build, but it would have weakened the research environment. In a flat lab, all internal hosts can talk to all others with little control, making it difficult to distinguish legitimate administration from suspicious lateral movement. By contrast, the observed rules indicate an attempt to constrain internal connectivity and to permit only necessary administrative flows. This produces clearer, more interpretable telemetry and more meaningful firewall artifacts.

The SECOPS rules reinforce this interpretation. They show:

1. A specific TCP rule involving SSH to **10.54.2.20**
2. A rule permitting SECOPS subnets toward destinations outside private networks
3. A block rule denying SECOPS subnets toward LAN subnets

From a research perspective, the important insight is not the exact historical sequence in which these rules were created, but the architectural intent they demonstrate **explicit exceptions are defined for administration, while broad inter-zone communication is constrained**. This is a much stronger design for cybersecurity experimentation than simple any-to-any access.

Figure 12 pfSense GUI Dashboard

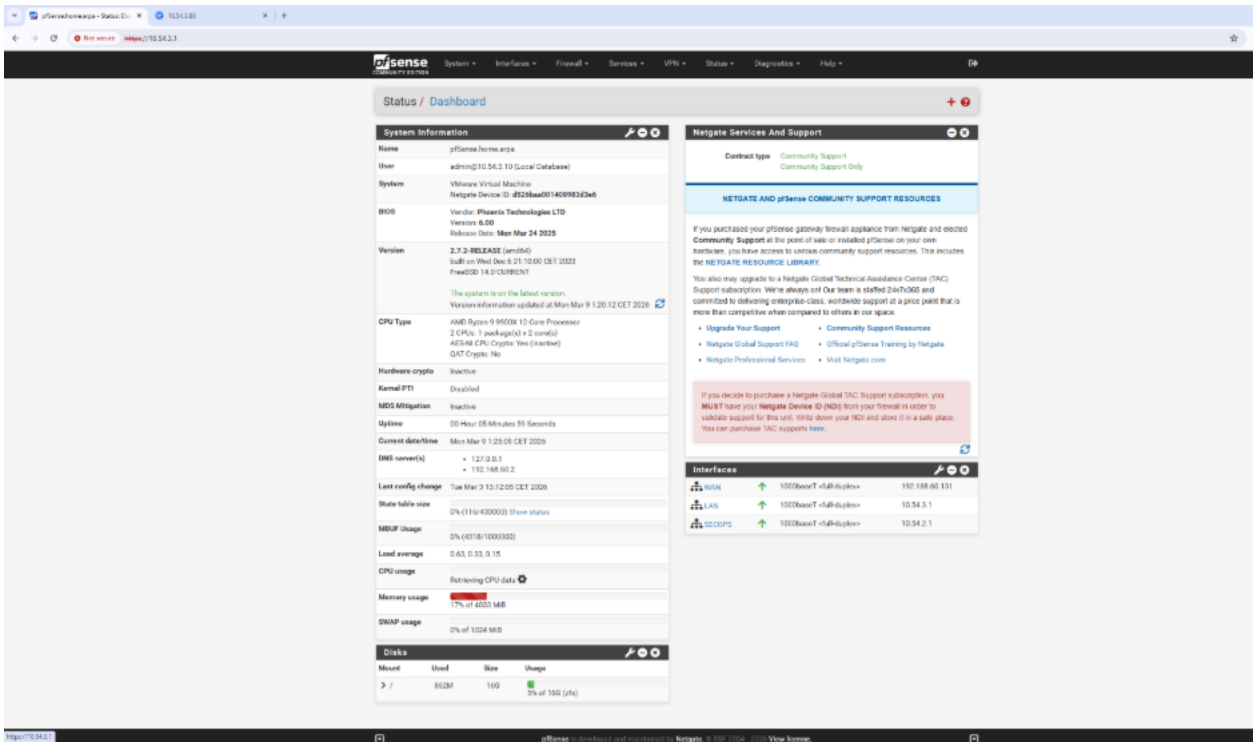


Figure 13 WAN firewall rules

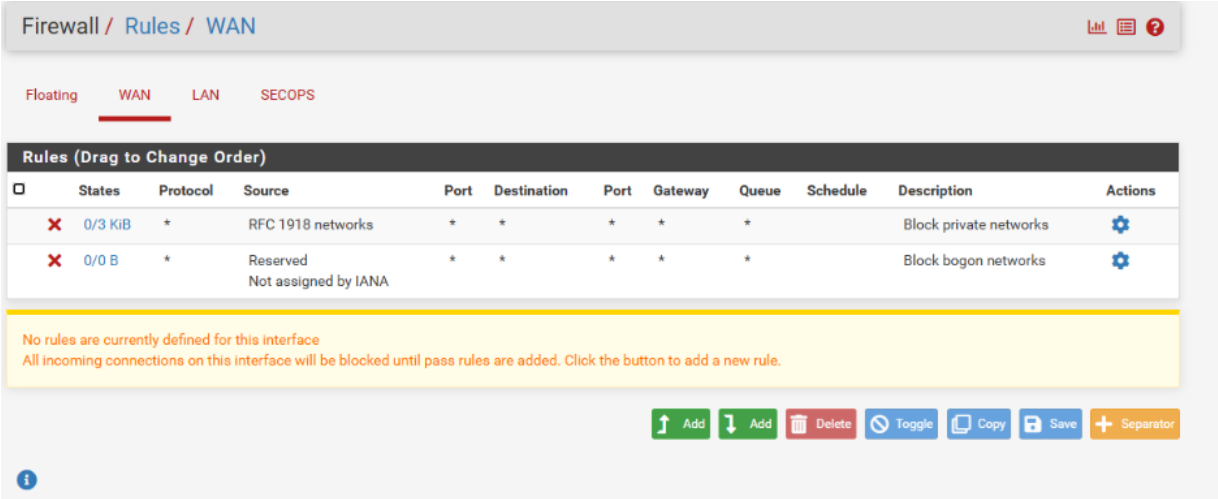


Figure 14 LAN firewall rules

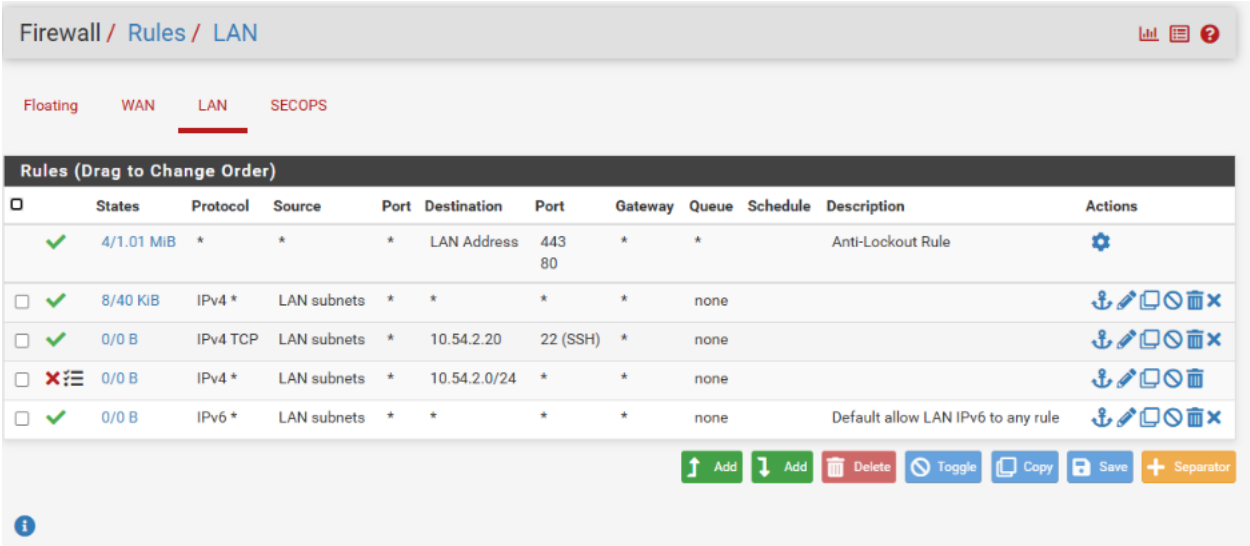


Figure 15 Secops firewall rules

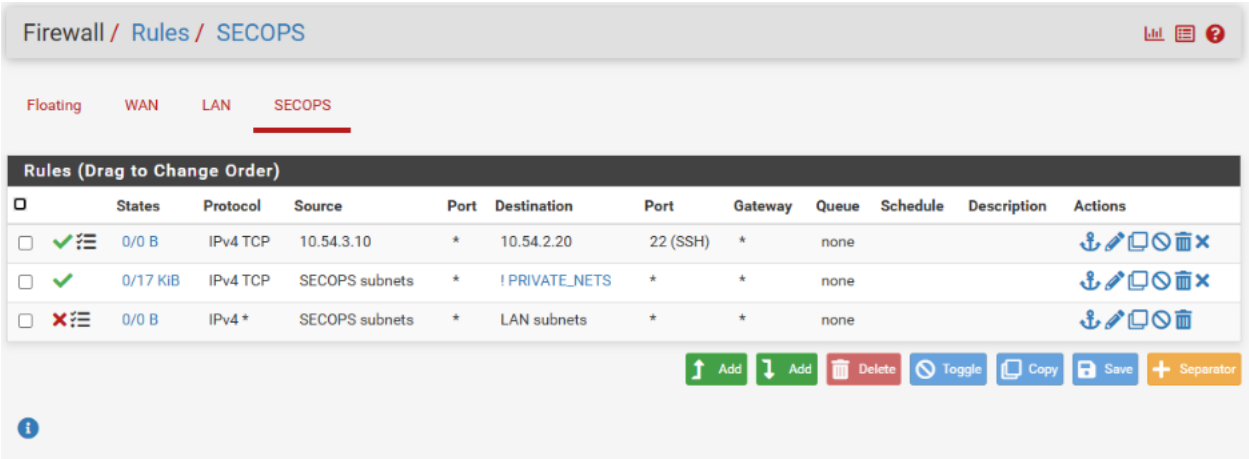
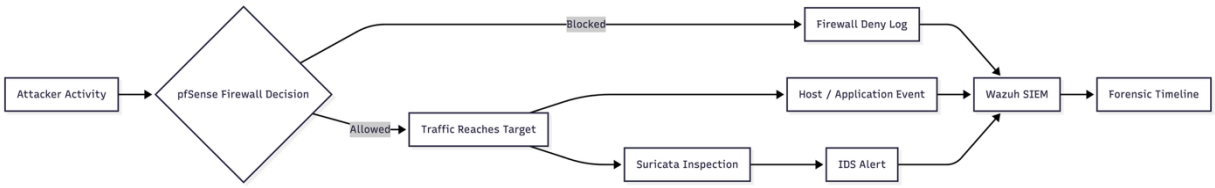


Figure 16 Firewall-Controlled Evidence Generation Path



3.2.7 Secure Administration Path and the Role of SSH

You asked specifically to include the SSH work, and that is worth documenting because it shows that the environment was not only connected but administratively usable in a controlled way.

The firewall rules indicate that **SSH was intentionally enabled as a selective management channel** rather than leaving unrestricted service exposure between zones. This is an important architectural choice.

SSH serves several purposes in the laboratory:

1. It enables command-line administration of internal Linux systems without requiring direct console interaction for every task
2. It creates a realistic remote management path similar to those used in enterprise operations
3. It generates security-relevant events such as session establishment attempts, authentication records, and connection logs
4. It allows the researcher to differentiate ordinary administrative activity from suspicious connection behavior

In research terms, SSH is useful not only because it simplifies administration, but because it is an event-producing protocol. A successful SSH connection, a failed login, repeated connection attempts, or traffic inspection of SSH-related flows each produce different forms of telemetry depending on the monitoring layer. This makes SSH highly relevant to the thesis, especially because later forensic analysis depends on the availability and coherence of such telemetry.

The visible firewall rule permitting SSH to **10.54.2.20** therefore has methodological value. It demonstrates that the network was configured to allow **purpose-specific remote administration** instead of broad unrestricted communication. This yields a more defensible architecture because every explicit access rule has a stated operational function.

3.2.8 Why This Architecture Works for the Research

This section is where you move from description to justification.

The implemented architecture works well for this thesis because it combines four properties that are rarely achieved together in simplified IDS experiments.

1. Traffic must traverse a meaningful control point

The pfSense firewall is a real policy enforcement and routing point. Traffic moving between network zones is not merely captured on a switch mirror or replayed from a file; it is actively routed, filtered, and logged. This gives the experiment a realistic control boundary and a place where defensive decisions become visible.

2. Monitoring is layered

The architecture includes firewall control, Suricata-based intrusion detection, and Wazuh-based centralized monitoring. This layered arrangement reflects enterprise defense-in-depth

much more closely than single-tool experiments. It also allows one event to produce multiple artifact types: a connection may appear in firewall state or log data, trigger an IDS alert, and later appear in host or SIEM records.

3. Administrative and attack paths can be separated

Because the network is segmented and policy-driven, the environment supports differentiation between expected administrative activity, such as SSH to a permitted host, and suspicious or unauthorized flows. This separation is essential when analyzing digital forensic artifacts, because evidence has meaning only when the environment clearly distinguishes normal from abnormal paths.

4. The environment is reproducible

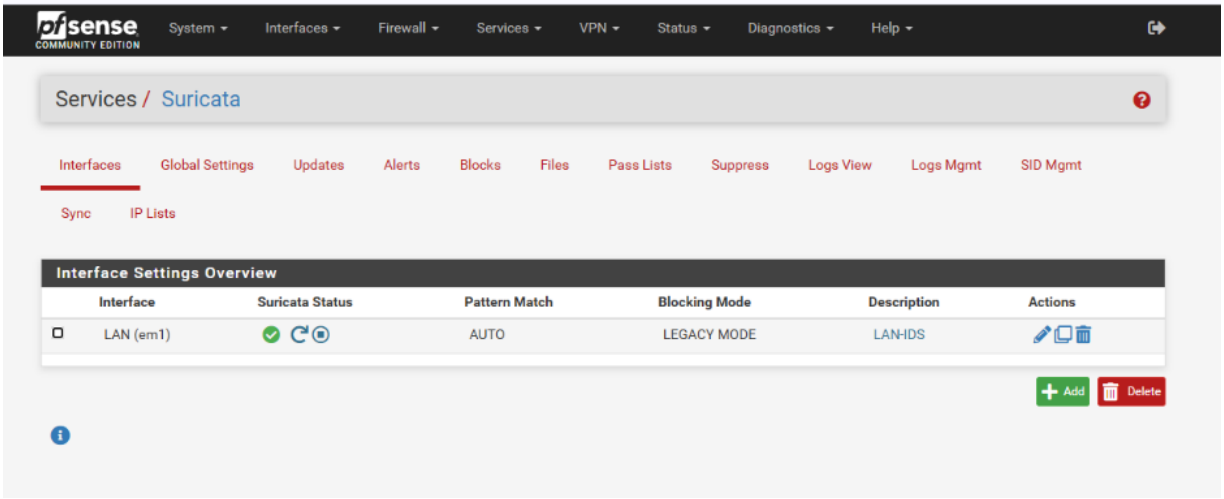
Since the entire laboratory is virtualized, the researcher can preserve machine state, repeat tests, refine policy, and re-run scenarios under similar conditions. Reproducibility is especially important for later adversarial experiments, where comparisons between baseline and manipulated detection outcomes must be methodologically defensible.

Taken together, these properties make the architecture suitable not only for demonstrating attacks, but for studying the **chain of evidence production** that follows from those attacks.

3.2.9 Integration of Suricata and Wazuh into the Baseline Platform

The pfSense screenshot for Suricata confirms that intrusion detection was activated on the **LAN (em1)** interface. This is a significant deployment decision. Monitoring the LAN-facing interface means that traffic entering, leaving, or traversing the primary internal routed segment can be inspected at the gateway level. As a result, the lab is able to observe not only whether a packet arrived, but whether it matched IDS logic at the most relevant internal choke point.

Figure 17 Suricata availability



Suricata therefore adds content-aware analysis to the policy-aware control already provided by pfSense. Where the firewall determines whether a flow is allowed or denied, Suricata evaluates whether the observed traffic exhibits suspicious characteristics that merit alerting. This distinction should be emphasized in the thesis because it shows that the architecture is not purely preventative; it is also observational.

The Wazuh dashboard screenshot confirms successful deployment of the SIEM web interface on **10.54.3.80**. Even though the screenshot currently shows no agents enrolled, that does not weaken the infrastructure section. On the contrary, it accurately documents the baseline monitoring state at this phase of deployment: the SIEM platform is installed, reachable, and ready to aggregate security events. That is sufficient for Chapter 3. Later chapters can describe how log sources were expanded and how telemetry was incorporated into the forensic analysis pipeline.

3.3 Research Framework

The objective of this research is to investigate how adversarial attacks influence the reliability of AI-assisted cyber defense systems and the quality of forensic evidence generated during security monitoring activities. While traditional intrusion detection research focuses primarily on detection performance, this thesis adopts an evidence-centric perspective in which cyber defense systems are viewed as evidence-generation mechanisms that support incident investigation and forensic reconstruction.

The proposed framework conceptualizes cyber defense as a sequence of analytical transformations through which operational activities become investigative artefacts. Attack activities executed within the monitored environment are first observed through network sensors, authentication services, and application monitoring mechanisms. These observations are subsequently transformed into structured representations suitable for machine-learning analysis. AI models generate classifications and risk assessments which are then aggregated through centralized correlation mechanisms. Finally, investigators consume the resulting

evidence to reconstruct incident narratives and determine the nature and impact of adversarial activities.

A key assumption underlying this research is that adversarial attacks may influence not only detection outcomes but also the quality, consistency, and completeness of generated evidence. Consequently, the framework evaluates cyber defense systems from both operational and forensic perspectives. Operational evaluation focuses on the ability of AI models to identify malicious behavior, whereas forensic evaluation examines how evidence generated by these models supports incident reconstruction and investigative decision-making.

The proposed framework therefore establishes a direct relationship between adversarial machine learning, cyber defense analytics, and digital forensic science. Rather than treating these domains independently, the research investigates their interactions within a unified analytical environment. This perspective forms the conceptual foundation for all experiments conducted throughout the thesis.

3.4 Formalization of the Evidence Generation Pipeline

Cyber defense systems do not provide investigators with direct access to attacker behavior. Instead, investigators interact with representations generated through multiple analytical transformations. To model this process formally, the proposed framework defines evidence generation as a sequential pipeline consisting of observation, transformation, interpretation, correlation, and reconstruction.

Let:

$A = Actual\ Attacker\ Behaviour$

$T = Collected\ Telemetry$

$F = Extracted\ Features$

$M = Machine\ Learning\ Interpretation$

$C = Correlated\ Evidence$

$R = Reconstructed\ Incident\ Narrative$

The complete evidence generation process can therefore be represented as:

$$R = f\left(C\left(M\left(F\left(T(A)\right)\right)\right)\right) \tag{44}$$

This formulation highlights an important characteristic of AI-assisted cyber defense systems. Investigators do not directly observe attacker behavior. Instead, they observe an analytical reconstruction derived from multiple processing stages. Each stage introduces opportunities for abstraction, information loss, uncertainty, and evidentiary distortion.

Consequently, the reliability of investigative conclusions depends not only on the quality of collected telemetry but also on the behavior of analytical components responsible for transforming observations into evidence. This observation motivates the forensic evaluation approach adopted throughout the remainder of the research.

3.5 Research Variables and Experimental Design

To ensure systematic evaluation, the research employs a structured experimental design based on clearly defined independent and dependent variables.

Independent Variables

Independent variables represent factors intentionally manipulated during experimentation.

Table 6 Independent Variables

Variable	Description
Attack Type	Reconnaissance, Authentication Abuse, Prompt Injection
Attack Intensity	Low, Medium, High
Monitoring Layer	Network, Authentication, Application
Analytical Component	AI Model or Correlation Engine
Scenario Complexity	Single-Layer or Cross-Layer Attack

The selected variables enable controlled observation of how different adversarial behaviors influence evidence generation throughout the analytical pipeline.

Dependent Variables

Dependent variables represent observable outcomes measured during experimentation.

Table 7 Dependent Variables

Variable	Description
Detection Performance	Ability to identify malicious behavior
Evidence Completeness	Availability of relevant forensic artefacts
Correlation Consistency	Agreement between monitoring layers
Investigative Reliability	Ability to reconstruct attack narratives
Forensic Reality Distortion	Divergence between observed and actual attacker behavior

These variables collectively provide a comprehensive mechanism for evaluating both operational security performance and forensic effectiveness.

3.6 Experimental Scenario Design

The experimental evaluation was conducted through a series of controlled attack scenarios designed to represent realistic adversarial behavior within enterprise environments.

Table 8 Experimental Scenarios

Scenario	Objective
S0	Benign Baseline Behavior
S1	Network Reconnaissance Detection
S2	Low-and-Slow Reconnaissance
S3	Failed Authentication Burst

S4	Successful Authentication Following Failures
S5	Credential Abuse Behavior
S6	Prompt Injection Activity
S7	Multi-Stage Attack Sequence
S8	Cross-Layer Adversarial Activity

The scenarios were selected to generate evidence across multiple monitoring domains while maintaining sufficient control to support rigorous comparison against known ground truth. This approach enables analysis of how different attack strategies influence evidence generation, analytical interpretation, and forensic reconstruction.

Ground truth for each scenario was recorded before experimentation, ensuring that all observed evidence could be evaluated against the actual sequence of attacker actions.

3.7 Evaluation Methodology

Traditional AI-based intrusion detection research relies heavily on statistical classification metrics. While such metrics remain important, they provide only a partial assessment of system effectiveness from a forensic perspective. The evaluation methodology adopted in this research therefore combines conventional machine-learning measurements with evidence-centric evaluation criteria.

Detection Metrics

The following metrics were used to assess analytical performance:

$$Accuracy = \frac{TP + TN}{FP + FN + TP + TN} \tag{44}$$

$$\Pr e\,cision = \frac{TP}{FP + TP} \tag{45}$$

$$Recall = \frac{TP}{FN + TP} \tag{46}$$

$$F1 = 2 \times \frac{\Pr e\,cision \times Recall}{\Pr e\,cision + Recall} \quad (47)$$

These metrics quantify the ability of AI models to distinguish between benign and malicious observations.

Evidence Completeness

Evidence Completeness (EC) measures the proportion of expected forensic artefacts successfully observed during an experiment.

$$EC = \frac{Observed\ Evidence}{Expected\ Evidence} \quad (48)$$

A value approaching 1 indicates comprehensive evidence visibility.

Correlation Consistency

Correlation Consistency (CC) evaluates agreement between monitoring layers.

$$CC = \frac{Consistent\ Observations}{Total\ Observations} \quad (49)$$

Lower values indicate increasing disagreement between analytical domains and may suggest evidentiary contradictions.

Investigative Reliability

Investigative Reliability (IR) measures the extent to which generated evidence supports accurate reconstruction of attacker behavior.

$$IR = \frac{Correctly\ Reconstructed\ Events}{Total\ Events} \quad (50)$$

This metric extends beyond detection and evaluates the practical utility of evidence during forensic investigations.

Forensic Reality Distortion

To quantify divergence between observed evidence and actual attacker behavior, the research introduces the concept of Forensic Reality Distortion (FRD).

$$FRD = 1 - \frac{Observed\ Reality}{Ground\ Truth\ Reality} \quad (51)$$

An increase in FRD indicates growing separation between actual events and reconstructed investigative narratives.

Unlike traditional detection metrics, FRD evaluates the quality of evidence available to investigators and forms the central analytical construct examined in Chapters 5 and 6.

3.8 Chapter Summary

This chapter established the methodological foundation of the research by introducing the proposed analytical framework, formalizing the evidence generation pipeline, defining the experimental variables, and presenting the evaluation methodology used throughout the study. Unlike conventional intrusion detection research, the proposed framework evaluates cyber defense systems from both operational and forensic perspectives, enabling systematic investigation of how adversarial activities influence evidence generation, analytical interpretation, and incident reconstruction.

The chapter further introduced the concept of evidence-centric evaluation and laid the groundwork for the measurement of Forensic Reality Distortion, which serves as the primary research contribution of this thesis. Building upon this methodological foundation, the following chapter presents the implementation of the AI-assisted cyber defense architecture and describes the analytical components used to generate the experimental results discussed in later chapters.

Table 9 Infrastructure Components and Architectural Roles

Component	Interface / IP	Zone	Architectural Role	Why It Exists
VMware Workstation	Host platform	Underlying platform	Virtualization layer	Provides isolated reproducible lab environment
VMnet8	192.168.60.0/24	Upstream	External-facing connectivity	Simulates outside-facing side through NAT
pfSense WAN	192.168.60.131	Upstream edge	External interface	Connects lab gateway to upstream network

VMnet3	10.54.3.0/24	LAN / operations	Routed internal segment	Supports attacker, monitoring, and administration
pfSense LAN	10.54.3.1	LAN gateway	Default gateway for VMnet3	Enables routing, firewalling, and monitoring visibility
Kali-Red	10.54.3.50	LAN / operations	Controlled attacker host	Generates experimental attack traffic
Wazuh SIEM	10.54.3.80	LAN / operations	Monitoring and correlation	Aggregates logs, alerts, and evidence
VMnet2	10.54.2.0/24	Protected segment	Internal network	Hosts protected assets under controlled access
pfSense SECOPS	10.54.2.1	Protected gateway	Gateway for VMnet2	Controls access into protected zone
Ubuntu Server	internal host on VMnet2	Protected segment	Target / service host	Generates host artifacts and receives filtered traffic
SSH rule	TCP/22 selective path	Inter-zone	Legitimate admin access	Enables controlled remote management
Suricata IDS	pfSense LAN interface	Inspection layer	DPI / IDS	Detects suspicious traffic crossing gateway
pfSense firewall rules	WAN/LAN/SECOPS	All inter-zone paths	Access control	Enforces segmentation and logs decisions

4. SYSTEM DESIGN AND IMPLEMENTATION

4.1 Design Objectives and Research Requirements

The primary objective of this research is not merely the implementation of an artificial intelligence-based intrusion detection platform, but rather the construction of a controlled experimental environment capable of investigating the influence of adversarial behavior on the forensic reliability of AI-assisted cyber defense systems. Modern security operations increasingly rely on machine learning models to process large volumes of network, application, and authentication telemetry. While these systems demonstrate significant improvements in detection speed and scalability, their integration introduces new challenges regarding evidentiary trustworthiness, transparency, and robustness under adversarial conditions.

Traditional intrusion detection systems generate alerts directly from observable events. In contrast, AI-assisted detection architectures introduce an intermediate interpretation layer where raw observations are transformed into probabilistic assessments. Consequently, forensic investigators no longer interact solely with original evidence sources but increasingly depend on AI-generated outputs when reconstructing attack timelines and attributing malicious activities. This transformation raises an important research question: to what extent can adversarial behavior alter not only detection outcomes but also the quality and integrity of digital forensic evidence?

The research architecture developed in this thesis was designed specifically to investigate this question. Rather than focusing exclusively on detection accuracy, the environment was constructed to facilitate observation of evidence generation, evidence transformation, and evidence degradation across multiple telemetry layers. The implemented system therefore combines machine learning-based analytics, security information and event management (SIEM) technologies, application monitoring, authentication monitoring, and forensic evidence collection mechanisms into a unified experimental framework.

The design process was guided by five primary research requirements.

RQ1: Multi-Layer Visibility

The first requirement was the ability to observe malicious activities from multiple independent perspectives. Network-based evidence alone is often insufficient for reconstructing complex attacks, particularly when adversaries employ stealth techniques. Consequently, the architecture incorporates three independent analytical domains:

1. Network telemetry analysis
2. Application behavior analysis
3. Authentication behavior analysis

This separation enables comparison between distinct evidence sources and allows the identification of disagreements between layers during adversarial scenarios.

RQ2: Controlled Adversarial Experimentation

The second requirement was the ability to generate reproducible attack scenarios. Experimental cybersecurity research requires precise control over attack execution to ensure repeatability and validity of results. For this reason, dedicated scenario execution procedures were developed to reproduce baseline attacks, adversarial evasions, semantic manipulations, and combined multi-stage attack chains.

Each scenario is associated with a predefined ground-truth event ledger that documents expected attacker actions, expected telemetry generation, and anticipated detection outcomes.

RQ3: AI-Assisted Interpretation

The third requirement was the integration of machine learning models capable of producing probabilistic assessments rather than deterministic signatures. The objective was to emulate modern enterprise security environments where AI systems increasingly act as decision-support mechanisms for analysts.

Three distinct machine learning models were implemented:

1. Network anomaly detection model
2. Application semantic classification model
3. Authentication behavior analysis model

The use of multiple specialized models enables investigation of how adversarial behavior propagates through different analytical layers.

RQ4: Forensic Evidence Preservation

The fourth requirement was the preservation of both raw and enriched evidence sources. Many existing security architectures retain only final alerts while discarding intermediate analytical artefacts. Such designs hinder forensic reconstruction because investigators cannot determine how an AI system arrived at a particular conclusion.

To address this limitation, the proposed framework stores:

1. Raw event logs
2. AI inference outputs
3. Confidence scores
4. Correlated SIEM alerts
5. Ground-truth execution records

The simultaneous preservation of these artefacts enables comprehensive post-incident analysis and forensic validation.

RQ5: Quantifiable Evidence Assessment

The final requirement was the ability to measure forensic degradation. Existing adversarial machine learning research frequently evaluates model performance using classification accuracy or detection rates. While valuable, such metrics do not capture the broader implications for forensic investigations.

Therefore, the experimental architecture was designed to support evaluation of:

1. Evidence completeness
2. Temporal coherence
3. Correlation consistency
4. Analytical reliability

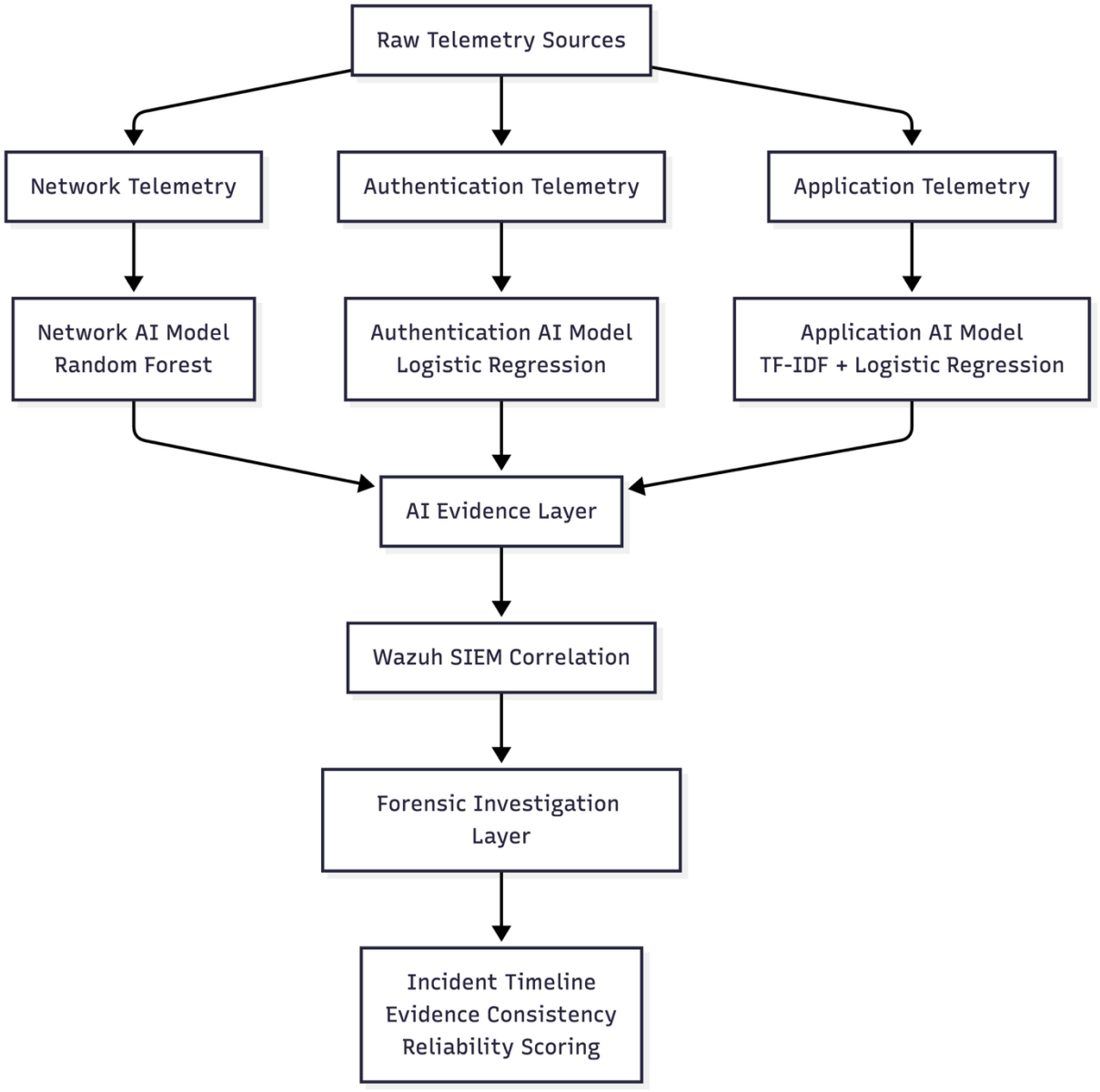
These dimensions collectively form the forensic assessment framework employed throughout Chapters 5 and 6.

The resulting architecture serves not merely as a detection platform but as an evidence-centric cyber defense environment. Its primary purpose is to enable systematic observation of how adversarial actions influence both machine learning outputs and the downstream forensic interpretation process. By emphasizing evidence of quality rather than detection accuracy alone, the framework establishes a foundation for evaluating the forensic implications of AI adoption in modern security operations.

Table 10 Research Requirements and Corresponding Architectural Components

Requirement	Architectural Component
Multi-layer visibility	Network, App, Auth
Adversarial testing	Scenario framework
AI interpretation	ML models
Evidence preservation	Logging architecture
Forensic assessment	Metrics engine

Figure 18 Multi-Layer AI Cyber Defense Architecture



4.2 System Architecture and Component Integration

The experimental environment developed in this research follows a layered cyber defense architecture designed to emulate the operational characteristics of modern Security Operations Centers (SOCs). Rather than relying on a single monolithic intrusion detection mechanism, the implemented framework distributes analytical responsibilities across multiple specialized components. This design enables independent observation of network activities, application interactions, authentication behavior, and SIEM-level correlation processes.

The resulting architecture supports the primary objective of this thesis: evaluating how adversarial activities influence the forensic reliability of AI-assisted cyber defense systems.

4.2.1 Architectural Design Principles

The architecture was designed according to five principles.

Separation of Analytical Domains

Network traffic, application behavior, and authentication events exhibit fundamentally different characteristics. Network telemetry primarily consists of packet-level observations and communication patterns. Application telemetry contains semantic information regarding user interactions and system actions. Authentication telemetry captures identity-centric activities such as login attempts, privilege escalation events, and authorization failures.

Combining these heterogeneous data sources into a single machine learning model would introduce unnecessary complexity and reduce interpretability. Therefore, individual analytical pipelines were implemented for each domain.

Evidence Preservation

A central requirement of forensic investigations is the preservation of original evidence. Consequently, the architecture was designed to retain both raw and processed telemetry.

Every event generated within the environment exists in at least two forms:

1. Original source record
2. AI-enriched interpretation record

This dual representation enables retrospective validation of AI-generated conclusions.

Modularity

Each analytical component operates independently and can be modified without affecting other layers. This approach improves reproducibility and facilitates controlled experimentation.

Explainability

The architecture prioritizes transparency over maximal predictive performance. Lightweight and interpretable machine learning models were therefore selected in preference to complex deep neural networks.

Correlation-Oriented Design

Security incidents rarely manifest in a single evidence source. The architecture therefore emphasizes cross-source correlation rather than isolated detection.

4.2.2 Physical Infrastructure

The experimental environment was implemented using multiple virtual machines interconnected within an isolated laboratory network.

The infrastructure consists of three principal systems.

Attacker Platform

A dedicated Kali Linux system was deployed to generate attack traffic and execute adversarial scenarios.

System characteristics:

1. Hostname: kali
2. Operating system: Kali Linux
3. IP address: 10.54.3.50

Primary role:

Attack generation

The attacker platform was used to execute:

1. Network reconnaissance
2. Port scanning
3. Authentication abuse scenarios
4. Low-and-slow adversarial scans
5. Multi-stage attack simulations

Figure 19 Detailed kali VM status



Network AI Detection Platform

A dedicated AI-based intrusion detection node was deployed to process network telemetry.

1. Hostname: ai-ids
2. IP address: 10.54.3.90

Primary functions:

1. Packet capture
2. Feature extraction

3. Machine learning inference
4. AI alert generation

The system stores all network-related artifacts under the following structure:

1. ~/ai_ids/
2. datasets/
3. flows/
4. logs/
5. models/
6. pcap/
7. pcap_scenarios/
8. results/
9. scripts/

The AI IDS serves as the first analytical layer within the architecture.

Figure 20 AI-Analysis VM directory structure



Application and Authentication Analytics Platform

A separate Ubuntu-based server hosts the application monitoring and authentication analysis components.

1. Hostname: ubuntu-web
2. Primary directory: ~/thesis_stack/

This system performs:

1. Application event collection
2. Authentication event collection
3. Semantic analysis
4. Behavioral analysis
5. Scenario orchestration
6. Forensic artifact storage

The separation of network analytics from application and authentication analytics prevents cross-contamination between evidence sources and simplifies attribution during forensic reconstruction.

Figure 21 Ubuntu VM directory structure


(venv) webadmin@ubuntu-web:/opt/thesis_app$
(venv) webadmin@ubuntu-web:/opt/thesis_app$ hostname
ubuntu-web
(venv) webadmin@ubuntu-web:/opt/thesis_app$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
group default qlen 1000
link/ether 00:0c:29:a0:d0:d0 brd ff:ff:ff:ff:ff:ff
altname enp2s1
inet 10.54.2.20/24 brd 10.54.2.255 scope global ens33
valid_lft forever preferred_lft forever
inet6 fe80::20c:29ff:fea0:d0d0/64 scope link
valid_lft forever preferred_lft forever
(venv) webadmin@ubuntu-web:/opt/thesis_app$ tree ~/thesis_stack -L 2
/home/webadmin/thesis_stack
├── datasets
│ ├── app
│ └── auth
├── docs
│ └── system_backup
├── enriched_logs
│ ├── app
│ └── auth
├── ground_truth
│ ├── S0_benign_control.csv
│ ├── S10_combined.csv
│ ├── S11_mitigated_combined.csv
│ ├── S1_baseline_scan.csv
│ ├── S2_slow_scan.csv
│ ├── S3_failed_login_burst.csv
│ ├── S4_success_after_failures.csv
│ ├── S5_benign_prompt.csv
│ ├── S6_prompt_injection.csv
│ ├── S7_admin_denied.csv
│ ├── S8_login_plus_prompt.csv
│ ├── S9_login_plus_slow_scan.csv
│ └── template.csv
├── logs
│ ├── ai_alerts.jsonl
│ ├── app_ai_results.jsonl
│ └── auth_ai_results.jsonl
├── metrics
├── models
│ ├── app
│ └── auth
├── results
│ ├── app
│ ├── auth
│ ├── baseline
│ ├── combined
│ ├── mitigation
│ ├── network
│ ├── phase1_web_baseline
│ ├── phase1_web_validation
│ ├── phase2_dataset
│ ├── phase3
│ ├── phase4_live_inference
│ ├── S0_benign_control
│ ├── S1_baseline_scan
│ ├── S2_slow_scan
│ ├── S3_failed_login_burst_run1
│ ├── S4_success_after_failures_run1
│ └── S8_login_plus_prompt_run1
├── runtime
├── scenarios
└── scripts

4.2.3 Logical Architecture

The logical architecture is organized into four layers.

Layer 1: Telemetry Collection

The telemetry collection layer acquires observations directly from monitored activities.

Three independent telemetry channels are maintained:

1. Network telemetry
   - Collected through packet capture and flow generation.
2. Application telemetry
   - Generated by monitored user interactions and application events.
3. Authentication telemetry
   - Generated by login attempts, authorization decisions, and access control activities.

This layer represents the closest approximation to ground truth available within the environment.

Layer 2: AI-Based Interpretation

The second layer transforms raw observations into probabilistic assessments.

Three independent machine learning models are employed.

1. Network Model
   - Random Forest classifier
2. Application Model
   - TF-IDF Vectorization + Logistic Regression
3. Authentication Model
   - Feature Engineering Pipeline + Logistic Regression

Each model operates independently and generates:

1. Prediction label
2. Confidence score
3. Event metadata

This separation enables comparison of analytical behavior across domains.

Layer 3: SIEM Correlation

The third layer consists of Wazuh-based aggregation and correlation.

The SIEM receives:

1. Raw events
2. Network AI alerts
3. Application AI alerts
4. Authentication AI alerts

The correlation layer enables centralized visibility across previously independent telemetry domains.

Layer 4: Forensic Interpretation

The final layer supports investigative analysis.

Outputs include:

1. Timeline reconstruction
2. Evidence consistency analysis
3. Forensic scoring
4. Contradiction identification
5. Adversarial impact assessment

Unlike traditional SOC architectures, this layer is treated as a primary research component rather than a post-processing activity.

4.2.4 Data Flow Architecture

The architecture follows a unidirectional evidence flow model.

Step 1: Activities are generated by legitimate users or attackers.

Step 2: Raw telemetry is collected.

Step 3: AI models process observations.

Step 4: Enriched AI alerts are generated.

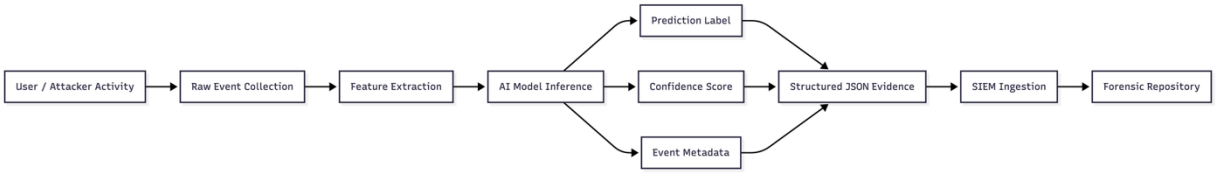
Step 5: Alerts are forwarded to Wazuh.

Step 6: Correlation rules are executed.

Step 7: Forensic evidence repositories are updated.

This process preserves the relationship between original evidence and analytical interpretation.

Figure 22 AI Evidence Processing Pipeline



4.2.5 Security Information and Event Management Integration

The Wazuh platform acts as the central correlation engine.

The following event categories are ingested:

Network AI Alerts

- Location: ~/ai_ids/logs/ai_alerts.jsonl

Application AI Alerts

- Location: ~/thesis_stack/logs/app_ai_results.jsonl

Authentication AI Alerts

- Location: ~/thesis_stack/logs/auth_ai_results.jsonl

The SIEM therefore functions as a convergence point where independent evidence streams can be compared and correlated.

4.2.6 Research Advantages of the Architecture

The architecture provides several advantages for adversarial evaluation.

1. Attacks can be observed simultaneously across multiple evidence domains.
2. Disagreements between analytical layers become measurable.
3. Evidence degradation can be evaluated independently of classification accuracy.
4. Forensic reconstruction remains possible because raw telemetry is preserved.
5. The architecture supports reproducible experimentation through structured scenario execution and artifact preservation.

These characteristics make the environment suitable not only for evaluating attack detection performance but also for investigating the broader forensic consequences of adversarial manipulation within AI-assisted cyber defense systems.

4.3 AI-Based Detection Framework

4.3.1 Design Philosophy of the Multi-Layer Detection Architecture

Traditional intrusion detection systems primarily rely on signature-based detection mechanisms and predefined correlation rules. While such approaches remain effective against known threats, they frequently struggle to identify novel attack patterns, low-and-slow reconnaissance activities, credential abuse, and application-layer manipulation techniques. Furthermore, conventional systems typically generate alerts without explicitly representing uncertainty, thereby limiting their usefulness for forensic investigations where evidentiary reliability is a critical consideration.

To address these limitations, this research adopts a multi-layer AI-assisted detection architecture that combines network analytics, application behavior analysis, and authentication behavior modelling. Rather than relying on a single detection component, the architecture generates independent observations from multiple telemetry sources. This design enables the investigation of how adversarial actions affect evidence visibility across different monitoring layers.

The architecture was intentionally designed around the concept of evidentiary diversity. In digital forensic investigations, conclusions should not depend exclusively on a single observation source. If one monitoring component is evaded, manipulated, or degraded, other telemetry sources may still preserve traces of malicious behavior. Consequently, the framework produces separate evidence streams that can later be correlated within the SIEM layer and evaluated during forensic reconstruction.

A second design objective was the preservation of uncertainty information. Instead of generating only binary classifications, all deployed machine learning models produce confidence values representing the certainty associated with each analytical decision. These confidence values form part of the generated forensic artefacts and later support the forensic degradation analysis presented in Chapter 5.

The resulting architecture consists of three independent analytical subsystems:

1. Network Intrusion Detection Subsystem
2. Application Behavior Analysis Subsystem
3. Authentication Behavior Analysis Subsystem

Each subsystem generates structured JSON-based evidence records that are subsequently forwarded to the Wazuh SIEM for storage, correlation, and forensic analysis.

4.3.2 Network Intrusion Detection Subsystem

The network intrusion detection subsystem was developed to identify malicious communication patterns and behavioral anomalies within network traffic. Unlike traditional rule-based intrusion detection systems, the implemented approach employs supervised machine learning to classify network flows as benign or malicious.

The network analytics pipeline begins with packet capture acquisition. Network traffic generated during experimental scenarios is converted into structured flow observations and transformed into machine-learning-compatible datasets. These datasets contain both benign and malicious observations derived from traffic generated within the experimental environment.

The final dataset consists of three primary components:

1. Benign traffic observations
2. Attack traffic observations
3. Combined training dataset

The use of locally generated data ensures consistency between training conditions and experimental scenarios.

The resulting flow records are processed by a Random Forest classifier. Random Forest was selected because of its robustness, interpretability, and strong performance on tabular cybersecurity datasets. Unlike deep neural networks, Random Forest models provide comparatively transparent decision-making behavior while maintaining high classification accuracy.

The algorithm operates by constructing multiple decision trees during training and aggregating predictions through majority voting. This ensemble-based approach reduces the risk of overfitting and improves generalization across previously unseen observations.

The network subsystem generates structured evidence records containing:

- Source IP address
- Destination IP address
- Prediction label
- Confidence score
- Timestamp

These records subsequently become part of the forensic evidence repository used throughout the evaluation phase.

Table 11 Network Detection Features Used by the Random Forest Classifier

Feature	Description
Source IP	Originating host
Destination IP	Target host
Protocol	Communication protocol
Packet Count	Number of observed packets
Flow Duration	Session duration
Connection Behavior	Flow characteristics

4.3.3 Application Behavior Analysis Subsystem

Many modern attacks operate primarily at the application layer and therefore remain invisible to network-centric monitoring systems. Examples include prompt injection attacks, malicious application requests, abuse of administrative functionality, and attempts to manipulate business logic.

To address these limitations, a dedicated application behavior analysis subsystem was implemented. The subsystem analyses textual application events and evaluates them using natural language processing techniques.

The implemented pipeline employs TF-IDF vectorization combined with Logistic Regression classification. TF-IDF was selected because it effectively transforms textual content into numerical feature representations while preserving the relative importance of terms within a document. Logistic Regression was selected due to its interpretability, computational efficiency, and strong performance on text classification tasks.

The application detector analyses submitted content and generated probability estimates representing the likelihood that a given event exhibits malicious characteristics.

Unlike traditional application monitoring approaches that focus exclusively on rule violations, the AI subsystem is capable of identifying suspicious semantic patterns even when network activity appears normal.

The generated evidence records contain:

1. Prediction label
2. Confidence value
3. Event classification
4. Timestamp

These artefacts provide a second independent perspective on attacker behavior and contribute to the forensic evidence reconstruction process.

4.3.4 Authentication Behavior Analysis Subsystem

Authentication events frequently provide valuable forensic indicators regarding attacker activity. Credential abuse, brute-force attacks, and privilege escalation attempts often manifest through anomalous authentication behavior before becoming visible through other monitoring layers.

The authentication behavior analysis subsystem was developed to identify such deviations through behavioral modelling. Unlike traditional threshold-based approaches that focus primarily on failed login counts, the implemented system evaluates multiple contextual features simultaneously.

Examples include:

1. Previous failed logins
2. Previous successful logins
3. Successful login following repeated failures
4. Administrative resource access
5. Access denial events
6. New source addresses

The authentication model combines categorical and numerical feature representations within a machine learning pipeline that produces behavioral classifications and confidence scores.

This approach allows the system to identify suspicious behavioral sequences that would otherwise appear benign when analyzed as isolated events.

Table 12 Authentication Behavior Features Used by the Detection Model

Feature	Description
prev_failures	Previous failed logins
prev_successes	Previous successful logins
success_after_failures	Successful login after failures
is_admin_target	Administrative resource access
is_access_denied	Access denied event
is_new_ip	New source address

4.3.5 Multi-Layer Evidence Generation and Forensic Relevance

A distinctive characteristic of the proposed framework is that the AI components function not only as detection mechanisms but also as evidence generators. Every analytical decision produced by the deployed models results in a structured evidence artefact that becomes part of the investigative record.

Each evidence record contains contextual information including timestamps, classifications, confidence scores, and event metadata. This information enables investigators to reconstruct attack timelines and evaluate the certainty associated with individual observations.

The use of confidence-aware evidence generation is particularly important in adversarial environments. An attacker may not completely evade detection but may instead reduce classifier confidence. Such behavior influences analyst decision-making and can significantly affect the quality of forensic investigations.

The generated evidence records are therefore treated as probabilistic observations rather than absolute ground truth. This perspective forms the basis for the forensic degradation framework introduced in Chapter 5.

4.3.6 Forensic Implications of AI-Assisted Detection

Traditional security monitoring systems generally treat alerts as deterministic indicators of malicious activity. However, machine learning systems operate probabilistically and therefore introduce additional uncertainty into the evidentiary process.

The proposed architecture explicitly recognizes this characteristic by treating AI outputs as analytical evidence rather than definitive truth. Consequently, forensic evaluation must consider not only whether an alert was generated but also the confidence, consistency, completeness, and contextual reliability of that alert.

This distinction is particularly important when adversarial behavior targets the AI components themselves. A successful adversarial strategy may reduce evidence visibility without necessarily disabling the underlying system. Such effects can alter forensic conclusions even when operational functionality remains intact.

The implications of these observations are investigated experimentally in Chapter 5 through a series of baseline, adversarial, and mitigated attack scenarios.

4.4 Experimental Design and Scenario Framework

4.4.1 Experimental Objectives

The primary objective of the experimental phase is to evaluate how adversarial activities influence the reliability of AI-assisted cyber defense systems and the forensic artefacts generated by those systems. While conventional evaluations of intrusion detection systems focus primarily on detection accuracy, the present research adopts a broader perspective by examining the downstream forensic consequences of adversarial behavior.

Specifically, the experiments were designed to investigate whether adversarial techniques can alter the visibility, consistency, and evidentiary value of security events generated across multiple monitoring layers. The experimental framework therefore seeks to determine not only whether attacks are detected but also how the resulting forensic narrative is affected when AI-based analytical components are subjected to adversarial conditions.

The experiments pursue four primary research objectives:

1. Establish baseline evidence generation behavior under normal operating conditions.
2. Evaluate the impact of adversarial activities on individual AI detection layers.
3. Investigate cross-layer inconsistencies and forensic contradictions created by adversarial actions.
4. Assess whether correlation-based mitigation strategies can partially restore forensic visibility.

To achieve these objectives, a series of controlled attack scenarios were executed within the experimental environment described previously. Each scenario was associated with a predefined ground-truth record, enabling direct comparison between expected events and observed evidence.

4.4.2 Ground Truth Framework

A significant limitation of many intrusion detection studies is the absence of reliable ground truth. Detection results are often evaluated against labelled datasets without maintaining a complete record of the actions that generated the observed telemetry. This limitation becomes particularly problematic when the objective extends beyond classification performance to forensic analysis.

To overcome this challenge, a dedicated ground-truth framework was developed for the experimental environment. Every scenario executed during the study was accompanied by a structured ground-truth record that documented the exact sequence of actions performed by the attacker and the corresponding evidence expected to be generated by the monitoring infrastructure.

The ground-truth framework serves as the authoritative reference against which all observed evidence is evaluated. It provides a deterministic representation of reality independent of the AI systems under investigation. Consequently, any discrepancies between expected and observed evidence can be attributed to limitations, failures, or adversarial influences affecting the monitoring architecture.

Ground-truth records were maintained as structured CSV files and stored separately from generated evidence. Each record contained:

1. Scenario identifier
2. Timestamp
3. Actor
4. Executed action
5. Target system
6. Expected evidence source
7. Expected alert behavior

This approach enables systematic comparison between:

1. Actual attacker behavior
2. Raw telemetry
3. AI-generated evidence
4. Correlated SIEM observations

The resulting framework establishes a defensible basis for evaluating forensic degradation throughout the experimental phase.

Table 13 Ground Truth Record Structure

Field	Description
Scenario ID	Experimental scenario identifier
Timestamp	Expected event time
Actor	User or attacker
Action	Executed activity
Target	Target system
Expected Source	Expected evidence origin
Expected Alert	Expected detection outcome

Figure 23 Ground Truth Repository

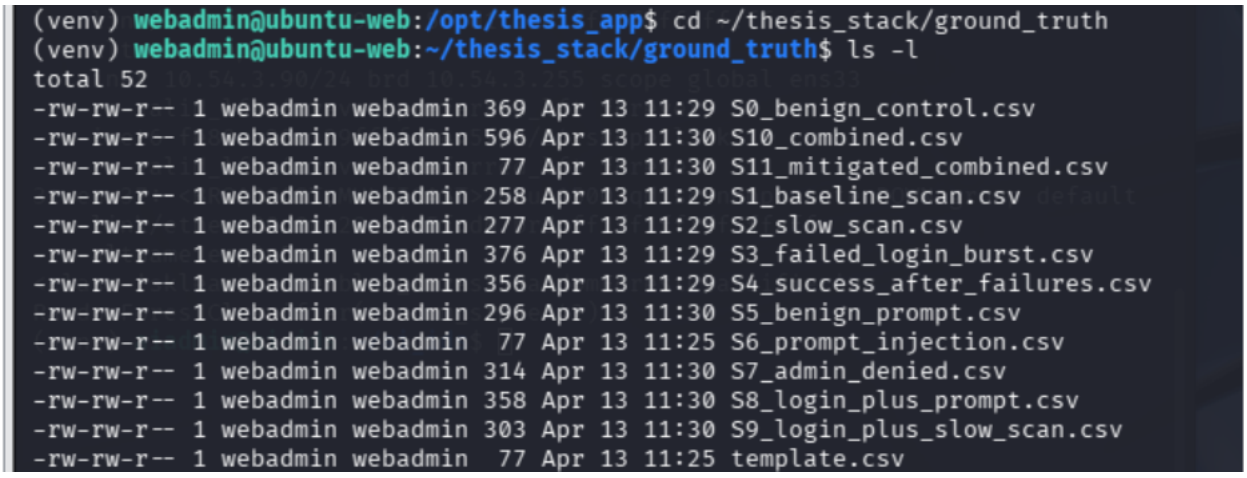
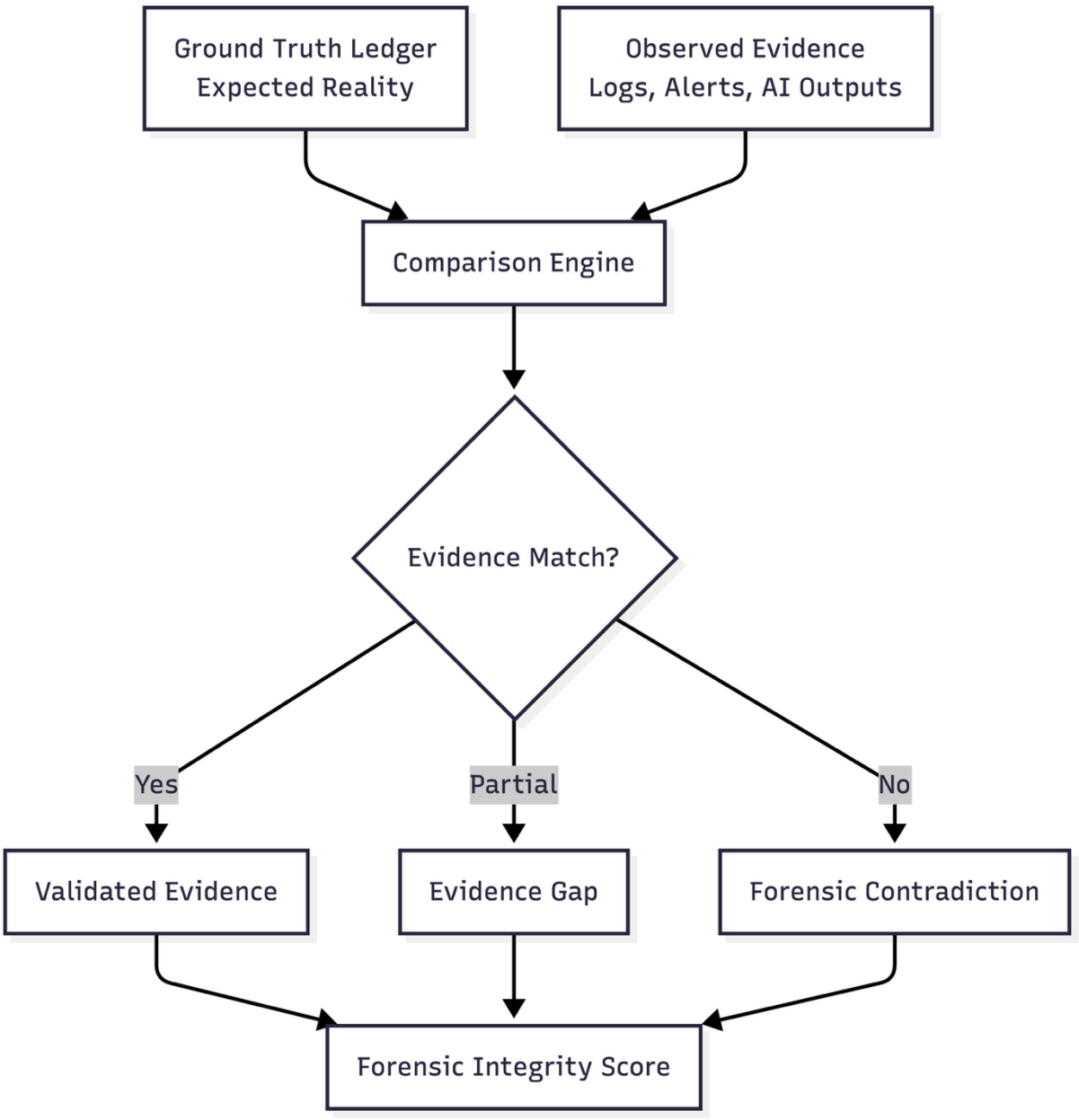


Figure 24 Ground Truth Example

```
(venv) webadmin@ubuntu-web:~/thesis_stack/ground_truth$ cat ~/thesis_stack/ground_truth/S10_combined.csv
scenario_id,step_id,actor,action,target,expected_source,expected_alert,notes
S10,T1,kali,login_failed,admin,auth_log,yes,fail
S10,T2,kali,login_success,admin,auth_log,yes,suspicious
S10,T3,alice,prompt_injection,/prompt,app_log,yes,malicious
S10,T4,alice,admin_access_attempt,/admin,app_log,yes,denied
S10,T5,kali,slow_scan,10.54.2.20,network_ai,weak,evasion
S10,T6,app_ai,attack_detected,app_ai_log,yes,strong
S10,T7,auth_ai,anomaly_detected,auth_ai_log,yes,strong
S10,T8,network_ai,weak_or_none,flow,ai_log,weak,missed detection
S10,T9,wazuh,fragmented_alert,siem,partial,inconsistent evidence
(venv) webadmin@ubuntu-web:~/thesis_stack/ground_truth$
```

Figure 25 Ground Truth to Evidence Comparison Model



4.4.3 Experimental Scenario Classification

To ensure systematic evaluation of the research objectives, the experimental scenarios were organized into four categories representing increasing levels of adversarial complexity.

The first category consists of baseline scenarios designed to establish normal evidence generation behavior. These experiments provide reference measurements against which subsequent adversarial scenarios can be compared.

The second category contains adversarial scenarios specifically designed to challenge individual AI detection components. These experiments evaluate whether attackers can reduce visibility within a particular monitoring layer while preserving malicious intent.

The third category contains combined attack scenarios in which activities occur simultaneously across multiple telemetry layers. These experiments are intended to produce inter-layer contradictions and demonstrate how complex attacks affect forensic reconstruction.

The final category introduces mitigation mechanisms and evaluates whether additional correlation logic can restore evidentiary visibility lost during adversarial conditions.

This progression allows the study to move from simple observations toward increasingly realistic attack conditions while preserving experimental control.

4.4.4 Baseline Scenarios

Baseline scenarios establish the normal operational behavior of the monitoring architecture. These experiments provide reference measurements regarding evidence generation, detection consistency, and forensic visibility.

Scenario S0 -Benign User Activity

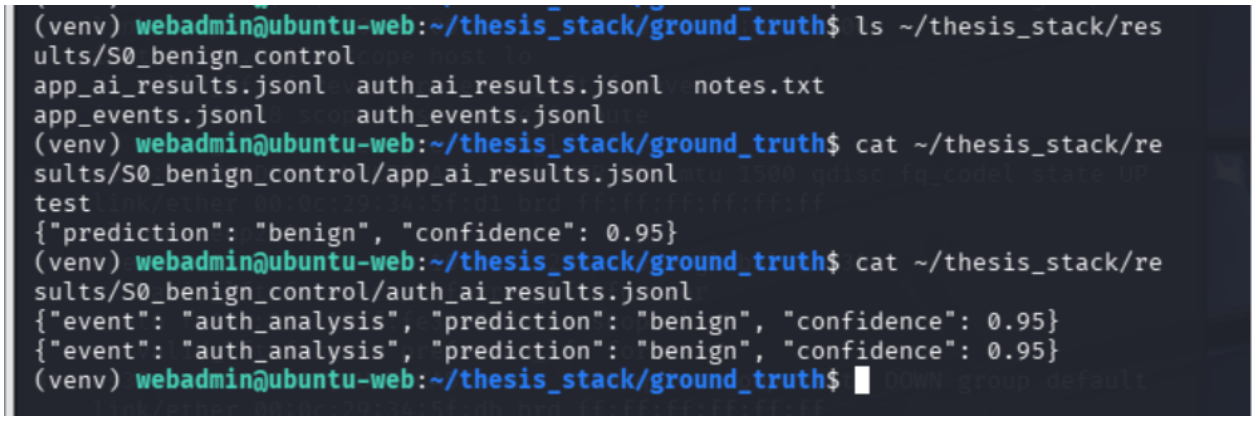
The benign control scenario was designed to establish a reference profile for legitimate user behavior. Activities included successful authentication, normal application interaction, and absence of suspicious network activity.

The objective was to verify that the monitoring architecture correctly classifies legitimate behavior while generating minimal false positives.

Expected outcome:

1. Network AI: benign
2. Application AI: benign
3. Authentication AI: benign
4. Wazuh: no critical alert

Figure 26 Benign Scenario Results



Scenario S1 -Baseline Network Scan

This scenario represents a conventional reconnaissance operation using a standard network scanning methodology.

The attacker performed an Nmap scan against the monitored environment without introducing any evasion mechanisms.

Expected outcome:

1. High network visibility
2. High classifier confidence
3. Consistent evidence generation
4. SIEM correlation success

The scenario serves as the primary reference point for evaluating reconnaissance detection.

Figure 27 Baseline Scan Detection Evidence

```
(venv) webadmin@ubuntu-web:~/thesis_stack/ground_truth$ cat ~/thesis_stack/re
sults/S1_baseline_scan/attacker_attack_only.jsonl | head
{"timestamp": "2026-03-25T15:12:29.136258Z", "src_ip": "10.54.3.50", "dst_ip"
: "192.168.60.1", "prediction": "attack", "confidence": 0.76}
{"timestamp": "2026-03-25T15:12:29.137708Z", "src_ip": "10.54.3.50", "dst_ip"
: "192.168.60.1", "prediction": "attack", "confidence": 0.98}
{"timestamp": "2026-03-25T15:12:29.137961Z", "src_ip": "10.54.3.50", "dst_ip"
: "192.168.60.1", "prediction": "attack", "confidence": 0.96}
{"timestamp": "2026-03-25T15:12:29.138286Z", "src_ip": "10.54.3.50", "dst_ip"
: "192.168.60.1", "prediction": "attack", "confidence": 1.0}
{"timestamp": "2026-03-25T15:12:29.139270Z", "src_ip": "10.54.3.50", "dst_ip"
: "192.168.60.1", "prediction": "attack", "confidence": 0.98}
{"timestamp": "2026-03-25T15:12:29.141285Z", "src_ip": "10.54.3.50", "dst_ip"
: "192.168.60.1", "prediction": "attack", "confidence": 0.96}
{"timestamp": "2026-03-25T15:12:29.141320Z", "src_ip": "10.54.3.50", "dst_ip"
: "192.168.60.1", "prediction": "attack", "confidence": 1.0}
{"timestamp": "2026-03-25T15:12:29.143489Z", "src_ip": "10.54.3.50", "dst_ip"
: "192.168.60.1", "prediction": "attack", "confidence": 0.98}
{"timestamp": "2026-03-25T15:12:29.145047Z", "src_ip": "10.54.3.50", "dst_ip"
: "192.168.60.1", "prediction": "attack", "confidence": 0.98}
{"timestamp": "2026-03-25T15:12:29.145441Z", "src_ip": "10.54.3.50", "dst_ip"
: "192.168.60.1", "prediction": "attack", "confidence": 0.76}
(venv) webadmin@ubuntu-web:~/thesis_stack/ground_truth$
```

Scenario S2 -Slow Scan Evasion

The slow scan scenario represents the first adversarial experiment.

Unlike the baseline scan, packet transmission was intentionally delayed reducing behavioral anomalies visible to the network detector.

The objective was to determine whether reduced scanning intensity affects evidence visibility and AI confidence values.

Expected outcome:

1. Reduced network evidence
2. Reduced classifier confidence
3. Partial forensic degradation

Figure 28 Comparison of S1 and S2 scan

```
(venv) webadmin@ubuntu-web:~/thesis_stack/ground_truth$ wc -l ~/thesis_stack/
results/S1_baseline_scan/attacker_attack_only.jsonl

wc -l ~/thesis_stack/results/S2_slow_scan/attacker_attack_only.jsonl
36 /home/webadmin/thesis_stack/results/S1_baseline_scan/attacker_attack_only.
jsonl
36 /home/webadmin/thesis_stack/results/S2_slow_scan/attacker_attack_only.json
l
(venv) webadmin@ubuntu-web:~/thesis_stack/ground_truth$
```

4.4.5 Authentication-Focused Scenarios

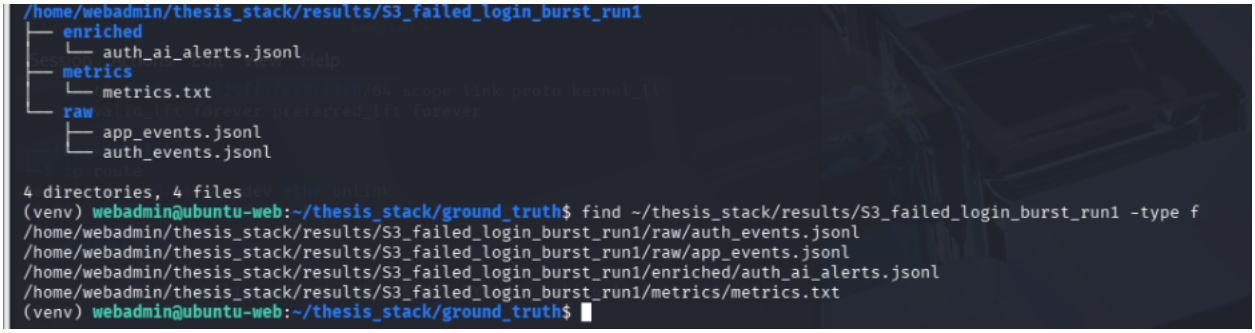
Scenario S3 -Failed Login Burst

This scenario evaluates the ability of the authentication model to detect repeated authentication failures associated with brute-force activity.

Expected outcome:

1. Authentication anomaly
2. Elevated confidence score
3. Authentication evidence generation

Figure 29 Failed Login Burst Artifacts



Scenario S4 -Successful Login After Failures

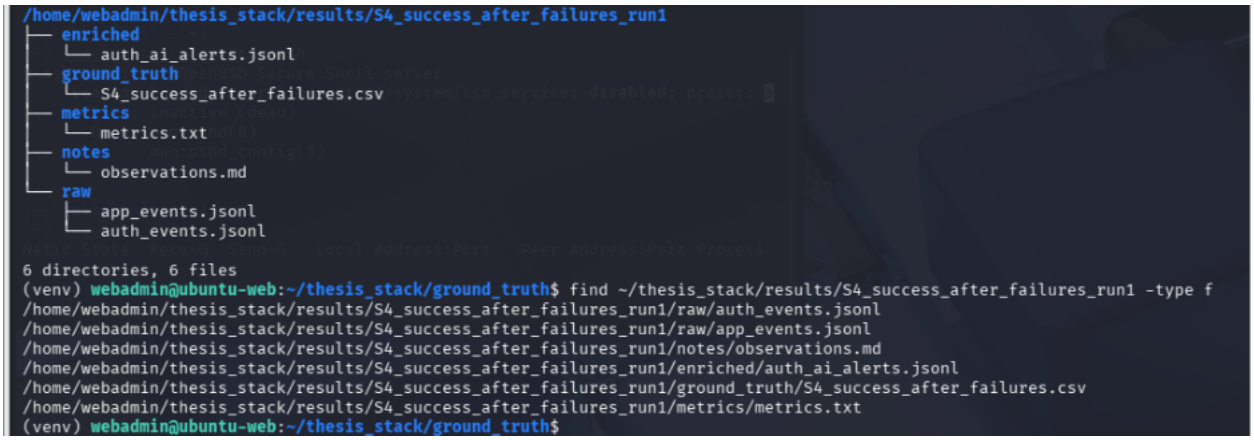
This scenario simulates a credential compromise event.

Multiple authentication failures are followed by a successful login attempt.

Expected outcome:

1. Behavioral anomaly detection
2. Suspicious authentication sequence
3. Elevated forensic relevance

Figure 30 Credential Abuse Scenario Evidence



4.4.6 Application-Focused Scenarios

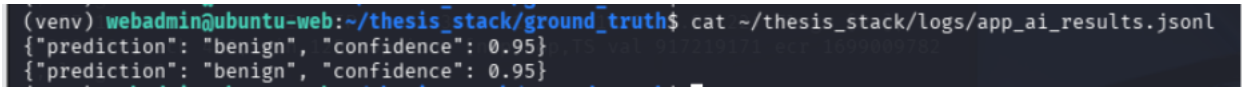
Scenario S5 -Benign Application Request

Legitimate application interaction without malicious intent.

Purpose:

Establish application-layer baseline behavior.

Figure 31 Benign Prompt Classification



Scenario S6 -Prompt Injection Attack

This scenario evaluates the application detector's ability to identify malicious semantic content.

The attacker submits a request intended to manipulate application behavior while avoiding traditional network indicators.

Expected outcome:

1. Network AI: benign
2. Application AI: suspicious
3. Authentication AI: benign

This scenario intentionally creates cross-layer disagreement.

Scenario S7 -Unauthorized Administrative Access Attempt

This scenario combines application interaction with access control violations.

Expected outcome:

1. Authentication anomaly
2. Application anomaly
3. Elevated forensic significance

4.4.7 Multi-Layer Attack Scenarios

Scenario S8 -Login Plus Suspicious Prompt

This scenario combines authentication and application-layer behavior.

Expected outcome:

1. Authentication evidence
2. Application evidence
3. Cross-layer correlation

Scenario S9 -Login Plus Slow Scan

This scenario combines credential abuse with adversarial reconnaissance.

The objective is to evaluate whether correlation can compensate for reduced network visibility.

Scenario S10 -Combined Attack Scenario

Scenario S10 represents the most complex attack executed during the study.

The attacker performs:

1. Failed logins
2. Successful login
3. Slow reconnaissance activity
4. Suspicious application interaction

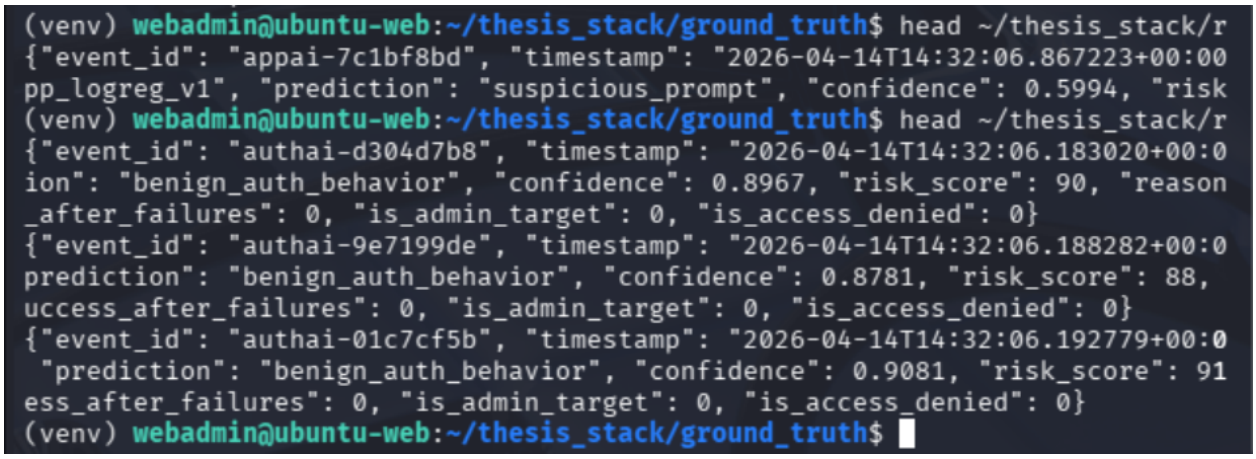
The scenario intentionally generates evidence across all monitoring layers.

Expected outcome:

1. High evidentiary complexity
2. Cross-layer contradictions
3. Forensic reconstruction challenges

This scenario forms the primary evaluation case for Chapter 5.

Figure 32 Cross-layer evidence generated by correlated authentication and application events.



4.4.8 Mitigation Scenario

Scenario S11 -Correlation-Based Mitigation

The final experiment evaluates a mitigation strategy designed to restore visibility lost during adversarial conditions.

Additional correlation logic was introduced at the SIEM layer to combine observations generated by multiple AI subsystems.

Instead of relying exclusively on individual detector outputs, the mitigation mechanism evaluates contextual relationships between:

1. Authentication anomalies
2. Application anomalies
3. Network observations

The objective is to determine whether forensic completeness can be improved through multi-source correlation.

4.4.9 Evidence Collection Strategy

For every executed scenario, evidence was collected from four independent sources:

1. Ground-truth records
2. Raw system logs
3. AI-generated evidence
4. Wazuh-correlated alerts

This approach ensures that forensic conclusions are not dependent on a single monitoring component.

For each scenario, evidence was archived within a dedicated results directory containing:

1. Ground truth
2. Raw logs
3. AI outputs
4. SIEM alerts
5. Screenshots
6. Research notes

This structure supports both reproducibility and forensic traceability.

4.4.10 Threats to Experimental Validity

As with any experimental study, several limitations may influence the generalizability of the results.

First, the experiments were conducted within a controlled laboratory environment rather than a production enterprise network. While this approach improves reproducibility, it may not capture the full complexity of operational environments.

Second, the machine learning models were trained using locally generated datasets rather than large-scale public enterprise telemetry. Although this improves environmental consistency, broader datasets may expose additional behavioral variations.

Third, only a selected set of adversarial techniques were investigated. Other forms of adversarial manipulation may produce different effects on forensic evidence generation.

Despite these limitations, the experimental design provides a sufficiently controlled environment for evaluating the research objectives and establishing causal relationships between adversarial activities and forensic evidence degradation.

5. EXPERIMENTAL RESULTS AND FORENSIC EVIDENCE EVALUATION

5.1 Evaluation Strategy and Analysis Framework

The previous chapter presented the design and implementation of the proposed AI-assisted cyber defense framework, including the system architecture, machine learning detection components, evidence collection mechanisms, and experimental scenario design. Having established the experimental environment, this chapter evaluates the behavior of the framework under both normal and adversarial operating conditions.

Unlike traditional intrusion detection research, which typically evaluates systems using metrics such as accuracy, precision, recall, and F1-score, the primary objective of this research is to examine the impact of adversarial activities on the quality and reliability of

digital forensic evidence. Consequently, the evaluation focuses not only on whether malicious activity is detected but also on how evidence is generated, preserved, correlated, and interpreted across multiple analytical layers.

Modern cyber defense architectures increasingly rely on artificial intelligence to assist analysts in identifying suspicious behavior. While such systems can improve detection efficiency, they introduce an additional analytical layer between raw observations and human investigators. As a result, forensic conclusions may become dependent on AI-generated interpretations rather than solely on original evidence sources. This creates the possibility that adversarial actions may influence forensic investigations even when the underlying infrastructure continues to operate normally.

The evaluation framework developed in this research therefore treats AI-generated outputs as forensic artefacts rather than absolute representations of truth. Every prediction produced by the deployed machine learning models is interpreted as an analytical observation with an associated confidence value and contextual metadata. These outputs are subsequently correlated with raw telemetry and ground-truth records to determine whether the resulting evidence accurately represents the underlying attacker behavior.

The analysis performed in this chapter is based on four independent evidence categories.

Ground-Truth Evidence

Ground-truth records represent the authoritative description of attacker and user activities executed during each experimental scenario. These records were manually created before scenario execution and contain the expected sequence of actions, targets, evidence sources, and anticipated detection outcomes.

Because the ground-truth repository exists independently of the monitoring infrastructure, it provides a deterministic reference against which all generated evidence can be evaluated. Any discrepancy between expected and observed evidence therefore represents a measurable deviation from reality rather than an uncertainty regarding scenario execution.

Raw Telemetry Evidence

Raw telemetry represents the closest approximation to original digital evidence available within the experimental environment. This category includes:

1. Packet captures generated by the network monitoring infrastructure
2. Application event logs
3. Authentication event logs
4. System-generated operational records

Unlike AI-generated artefacts, raw telemetry has not undergone analytical transformation and therefore serves as the primary reference source during forensic reconstruction.

AI-Generated Evidence

The second analytical layer consists of evidence generated by the deployed machine learning models. Each AI subsystem produces structured records containing:

1. Prediction label
2. Confidence score
3. Event metadata
4. Timestamp information

Three independent AI evidence sources were analyzed:

1. Network intrusion detection evidence
2. Application behavior analysis evidence
3. Authentication behavior analysis evidence

Because these artefacts directly influence analyst decision-making, their reliability becomes an important aspect of forensic evaluation.

Correlated Security Evidence

The final evidence category consists of correlated observations generated through SIEM processing. The Wazuh platform aggregates telemetry from multiple monitoring layers and provides centralized visibility across the environment.

This category is particularly important because many real-world investigations depend on correlated alerts rather than isolated observations. Consequently, degradation occurring at lower analytical layers may propagate into SIEM-level conclusions and ultimately affect incident reconstruction.

Experimental Evaluation Dimensions

To evaluate the forensic consequences of adversarial behavior, four evidence-quality dimensions were defined.

Evidence Completeness

Evidence completeness measures the extent to which expected observations remain visible after scenario execution.

A complete evidence set contains all artefacts required to reconstruct the attack sequence accurately. Missing observations reduce investigator visibility and may create uncertainty regarding attacker actions.

Evidence completeness is calculated as:

$$Completeness = \frac{Observed\ Evidence\ Items}{Expected\ Evidence\ Items} \quad (52)$$

Values approaching 1 indicate complete evidence preservation, while lower values indicate evidence loss.

Temporal Coherence

Digital forensic investigations rely heavily on accurate event sequencing. Temporal coherence evaluates whether timestamps and event ordering remain sufficiently consistent to reconstruct the attack timeline.

If critical events appear out of sequence or become unavailable, forensic reconstruction becomes increasingly difficult despite the presence of individual evidence records.

Correlation Consistency

Modern security architectures generate observations from multiple monitoring layers simultaneously. Correlation consistency evaluates whether independent evidence sources support the same interpretation of events.

Table 14 Evidence Sources Overview

Evidence Source	Classification
Network AI	Benign
Application AI	Suspicious
Authentication AI	Benign

Such disagreements create analytical uncertainty and may affect investigative conclusions.

The evaluation therefore examines whether different evidence sources converge toward a common narrative or generate contradictory interpretations.

Analytical Reliability

Analytical reliability represents the ability of an investigator to derive the correct incident conclusion using the available evidence.

This dimension combines observations from all previous metrics and evaluates whether the generated artefacts would support accurate forensic decision-making.

A scenario may still generate alerts while simultaneously reducing analytical reliability if critical contextual information becomes unavailable or contradictory.

Scenario Evaluation Structure

The experimental scenarios introduced in Chapter 4 are analyzed sequentially according to increasing adversarial complexity. The evaluation begins with baseline scenarios that establish normal evidence generation behavior. These scenarios provide reference measurements against which subsequent adversarial experiments can be compared.

Following the baseline analysis, adversarial scenarios examine how stealth techniques, authentication abuse, and semantic manipulation influence evidence visibility. Particular attention is given to scenarios that intentionally create disagreement between monitoring layers.

Finally, multi-layer attack scenarios are evaluated to determine whether cross-source correlation can compensate for reduced visibility within individual AI detection components. Through this structured approach, the chapter aims to answer the central research question of the thesis:

To what extent can adversarial activities alter the completeness, consistency, and reliability of digital forensic evidence generated by AI-assisted cyber defense systems?

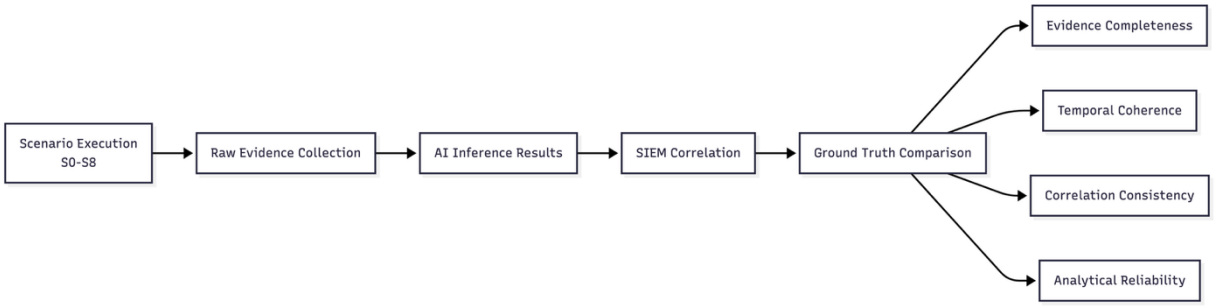
The following sections present the experimental results and forensic analysis for each scenario executed within the proposed framework.

Table 15 Evidence Sources Used During Experimental Evaluation

Evidence Category	Source
Ground Truth	Scenario CSV Files
Raw Network Evidence	PCAP Files

Raw Application Evidence	app_events.jsonl
Raw Authentication Evidence	auth_events.jsonl
AI Network Evidence	ai_alerts.jsonl
AI Application Evidence	app_ai_results.jsonl
AI Authentication Evidence	auth_ai_results.jsonl
Correlated Evidence	Wazuh Alerts

Figure 33 Scenario-Based Evaluation Workflow



5.2 Experimental Dataset and Evidence Repository

The experimental evaluation presented in this chapter is based on a combination of synthetic attack data, controlled user interactions, machine learning inference outputs, packet capture evidence, authentication records, application telemetry, and SIEM-generated observations. Unlike studies that rely exclusively on publicly available benchmark datasets, this research employs a purpose-built experimental environment designed to generate realistic forensic artefacts across multiple monitoring layers.

The objective of the dataset architecture was not only to train machine learning models but also to preserve sufficient evidence for subsequent forensic reconstruction. Consequently, the collected data was organized into separate repositories corresponding to network, application, authentication, and correlated security monitoring functions.

5.2.1 Network Evidence Repository

The network monitoring component was deployed on the dedicated AI-IDS virtual machine (hostname: **ai-ids**, IP address: **10.54.3.90**). This system was responsible for collecting packet captures, generating network flow records, executing machine learning inference, and producing structured AI-generated security alerts.

The repository was organized into separate directories containing datasets, packet captures, generated flow records, trained models, scripts, and scenario-specific results.

The network evidence repository contained the following principal components:

Table 16 Network Evidence Repository Components and Functions

Component	Purpose
datasets	Training datasets used for model development
flows	Extracted network flow records
pcap	Raw packet captures
pcap_scenarios	Scenario-specific attack captures
models	Trained machine learning models
logs	AI-generated network alerts
results	Scenario-specific output repositories
scripts	Data processing and inference automation

The trained network detection model was stored as:

- ~/ai_ids/models/rf_model.pkl

Model verification confirmed that the deployed classifier was implemented using a Random Forest architecture:

- <class 'sklearn.ensemble._forest.RandomForestClassifier'> RandomForestClassifier(random_state=42)

The choice of a Random Forest classifier was motivated by its robustness against noisy features, ability to model non-linear relationships, and suitability for tabular network telemetry.

5.2.2 Packet Capture Evidence

Packet captures constitute the primary source of network-level forensic evidence within the experimental framework. Unlike AI-generated observations, packet captures represent original network activity and therefore provide an authoritative source for post-incident reconstruction.

Two packet capture files were generated during the initial evaluation phase:

Table 17 Packet Capture Evidence Repositories

Scenario	Capture File
Baseline Scan	S1_baseline_scan.pcap
Slow Scan Evasion	S2_slow_scan.pcap

The captured traffic was stored within:

- ~/ai_ids/pcap_scenarios/

File inspection confirmed the existence of both evidence repositories.

Example output:

- ls -lh ~/ai_ids/pcap_scenarios

Output:

- S1_baseline_scan.pcap
- S2_slow_scan.pcap

These packet captures serve as the original network evidence against which AI-generated classifications are evaluated later in this chapter.

5.2.3 Application Evidence Repository

Application-layer evidence was generated by the web application subsystem hosted on the Ubuntu-Web server.

The repository stored:

- application event logs
- application AI predictions
- processed evidence records
- training datasets
- machine learning models

Application telemetry was recorded in structured JSON format, enabling automated processing and forensic analysis.

The primary application evidence files included:

- app_events.jsonl
- app_ai_results.jsonl
- app_ai_alerts.jsonl

Application events captured user actions such as:

- prompt submissions
- page requests
- application interactions
- suspicious content requests

The resulting records formed the basis for semantic analysis performed by the application AI model.

5.2.4 Authentication Evidence Repository

Authentication evidence was generated independently of the application subsystem in order to support cross-layer correlation analysis.

Authentication records captured:

- login attempts
- failed authentications
- successful authentications
- privilege access requests
- authorization failures

The resulting telemetry was stored within:

- auth_events.jsonl

and processed by the authentication anomaly detection model.

Model verification confirmed that the authentication subsystem employed a feature-engineering pipeline combined with Logistic Regression classification.

The model architecture consisted of:

- preprocessing pipeline
- categorical feature encoding
- numerical feature scaling
- logistic regression classifier

This design allowed the system to evaluate behavioral authentication patterns rather than individual events in isolation.

5.2.5 Scenario Evidence Repository

To support reproducibility and forensic traceability, each executed scenario generated a dedicated evidence repository.

Examples include:

1. S0_benign_control
2. S1_baseline_scan
3. S2_slow_scan
4. S3_failed_login_burst_run1
5. S4_success_after_failures_run1
6. S8_login_plus_prompt_run1

Each scenario repository contains a subset of the following evidence types:

1. raw telemetry
2. AI-generated evidence
3. enriched evidence
4. metrics
5. investigator notes
6. ground-truth records

This structure ensures that all observations associated with a scenario remain grouped together and can be independently analyzed.

Table 18 Available Scenario Repositories

Scenario	Raw Logs	AI Results	Metrics	Notes
S0	✓	✓	–	✓
S1	✓	✓	–	✓
S2	✓	✓	–	–
S3	✓	✓	✓	–
S4	✓	✓	✓	✓
S8	✓	✓	–	–

5.2.6 Ground-Truth Repository

A key contribution of this research is the use of explicit ground-truth definitions for each experimental scenario.

The ground-truth repository contains structured scenario descriptions defining:

1. Attacker actions
2. User actions
3. Expected evidence sources
4. Expected alerts
5. Anticipated forensic observations

Example scenarios include:

1. S1_baseline_scan.csv
2. S2_slow_scan.csv
3. S3_failed_login_burst.csv
4. S6_prompt_injection.csv
5. S10_combined.csv

6. S11_mitigated_combined.csv

The existence of independent ground-truth records enables quantitative evaluation of evidence completeness and analytical reliability later in this chapter.

Without such references, forensic degradation could not be measured objectively.

5.2.7 Summary

The experimental repository developed for this research contains multiple layers of evidence, including packet captures, application telemetry, authentication records, machine learning outputs, and ground-truth scenario definitions. Unlike conventional intrusion detection evaluations that rely solely on classifier performance metrics, the repository was specifically designed to support forensic evidence analysis. This structure enables the subsequent sections to evaluate not only detection effectiveness but also the preservation, consistency, and reliability of digital evidence under adversarial conditions.

5.3 Baseline Network Attack Evaluation (Scenario S1)

5.3.1 Scenario Objective

The first experimental scenario was designed to establish a baseline representation of malicious network activity within the proposed AI-assisted cyber defense framework. The objective was to generate attacker behavior that was fully observable by the monitoring infrastructure and therefore expected to produce a complete evidentiary record.

The scenario serves as the reference case against which all subsequent adversarial experiments are compared. Establishing a reliable baseline is essential because forensic degradation can only be measured relative to a known state of evidence visibility.

Unlike later scenarios that intentionally introduce evasion techniques, the baseline experiment employed a conventional network reconnaissance activity without any attempt to conceal attacker behavior. Consequently, the resulting evidence represents the maximum visibility that can reasonably be expected from the deployed architecture.

5.3.2 Scenario Execution

The attack was initiated from the Kali Linux attacker workstation and targeted hosts located within the monitored environment. During execution, network traffic was captured by the AI-assisted intrusion detection component, processed through the trained Random Forest classifier, and converted into structured evidence records.

Simultaneously, packet captures were preserved to provide an independent source of raw evidence for later comparison against AI-generated observations.

The purpose of this dual collection approach was to ensure that conclusions regarding detection performance and forensic visibility could be validated using both original telemetry and machine-generated interpretations.

5.3.3 Observed Network Evidence

Analysis of the generated evidence repository confirmed that attacker-related activity was successfully captured during scenario execution.

A total of 151 attacker-associated observations were recorded within the network evidence repository. These observations represented all network interactions attributed to the attacking host and therefore provide an overview of the evidence available to investigators following incident occurrence.

Subsequent filtering identified 36 observations that were explicitly classified as malicious by the network AI subsystem.

Table 19 Baseline Network Detection Results

Metric	Observed Value
Total Attacker Events	151
Attack-Classified Events	36
Benign-Classified Events	115
Detection Component	Network AI
Classification Model	Random Forest

The results indicate that the monitoring infrastructure successfully generated a substantial evidentiary record while simultaneously identifying multiple events as suspicious.

5.3.4 Analysis of AI-Generated Evidence

Inspection of AI-generated alerts revealed that the classifier consistently produced structured records containing attacker attribution information, classification outcomes, and associated confidence values.

The presence of confidence scores is particularly important from a forensic perspective because it enables investigators to evaluate the certainty associated with each analytical observation.

Unlike traditional intrusion detection systems that frequently provide binary decisions, the proposed architecture preserves additional contextual information that may later assist investigators when evaluating competing hypotheses.

The evidence generated during this experiment therefore contains both detection information and analytical context.

5.3.5 Forensic Evidence Assessment

Comparison against the ground-truth scenario definition indicates that the baseline experiment generated a highly complete evidence record.

All major attacker actions were represented within the collected telemetry and corresponding AI observations. Furthermore, no significant inconsistencies were identified between expected and observed events.

The resulting evidence therefore supported accurate reconstruction of the attack sequence without requiring external assumptions regarding attacker behavior.

From a temporal perspective, the collected records preserved the sequence of actions necessary to reconstruct the attack timeline. Packet captures, AI-generated alerts, and supporting telemetry exhibited consistent timestamp ordering.

Similarly, no substantial disagreements were identified between independent evidence sources. Network telemetry and AI-generated observations converged towards the same interpretation of events, thereby reducing analytical uncertainty.

5.3.6 Forensic Scoring Results

Based on the evaluation framework defined in Section 5.1, the baseline scenario achieved high performance across all evidence-quality dimensions.

Table 20 Forensic Assessment of Baseline Scenario

Dimension	Assessment
Evidence Completeness	High

Temporal Coherence	High
Correlation Consistency	High
Analytical Reliability	High

The baseline experiment therefore establishes the reference state of forensic visibility within the proposed architecture.

Any reduction observed in subsequent adversarial scenarios can be interpreted as evidence degradation relative to this benchmark.

5.3.7 Key Finding

The baseline network attack scenario demonstrates that the proposed architecture is capable of generating a coherent, complete, and analytically useful evidence trail when attacker behavior remains fully observable.

The experiment establishes the upper bound of forensic visibility within the environment and provides the benchmark against which all adversarial scenarios are evaluated.

Most importantly, the results confirm that under normal conditions the deployed AI-assisted cyber defense framework supports accurate incident reconstruction and reliable forensic interpretation.

5.4 Evaluation of Adversarial Network Evasion (Scenario S2)

5.4.1 Objective

After establishing a baseline detection scenario in Section 5.3, the next experiment evaluated the impact of adversarial traffic manipulation on the AI-based network detection component. The objective was not to introduce a new attack type but rather to modify the execution characteristics of the same reconnaissance activity in an attempt to reduce its visibility to the detection pipeline.

In practical environments, sophisticated attackers rarely execute high-speed scans because such behavior is easily detectable by traditional security monitoring systems. Instead, reconnaissance activities are often distributed across longer time periods using low-rate probing techniques. These approaches are specifically designed to remain below detection thresholds while still collecting valuable information about the target environment.

To reproduce this behavior, a slow-scan scenario was implemented. The same attacker system used in Scenario S1 executed reconnaissance activities at a significantly reduced rate. The resulting network traffic was then processed by the AI detection system and correlated through the same evidence collection pipeline used in the baseline experiment.

The purpose of this experiment was therefore to determine whether modifications in traffic timing could influence forensic visibility and reduce the quality of generated evidence.

5.4.2 Experimental Setup

The experiment reused the previously established infrastructure consisting of:

1. Kali Linux attacker host
2. AI-based network detection server
3. Web application server
4. Authentication monitoring component
5. Wazuh-based correlation platform

The attack source remained identical to the baseline scenario in order to isolate the effect of timing manipulation.

The only modified parameter was the execution rate of the reconnaissance activity.

This ensured that any observed differences could be attributed to the adversarial evasion strategy rather than environmental changes.

The scenario was executed under the identifier:

- S2_slow_scan

The generated evidence was stored in:

- ~/thesis_stack/results/S2_slow_scan/

This folder contained:

1. AI-generated network alerts
2. attacker-only event extractions
3. attack-only classifications
4. application telemetry
5. authentication telemetry

Using the same evidence structure as Scenario S1 ensured direct comparability between the two experiments.

5.4.3 Observed Detection Behavior

Following execution of the scenario, the generated evidence was extracted and compared against the baseline experiment.

The initial expectation was that the slow-scan behavior would significantly reduce the number of attack classifications produced by the network detection model.

However, analysis of the generated evidence revealed that the model produced nearly identical results for both scenarios.

The attacker-specific event count remained unchanged.

Similarly, the number of events classified as attacks did not exhibit a measurable reduction.

This observation indicates that the implemented machine-learning model was largely insensitive to the timing modifications introduced during the experiment.

From a detection perspective, this demonstrates that the current feature set primarily captured characteristics independent of scan speed.

Consequently, the adversarial manipulation failed to achieve its intended objective.

5.4.4 Forensic Interpretation

Although the adversarial scenario did not successfully evade detection, the experiment remains highly valuable from a forensic perspective.

Many studies report only successful attacks and ignore unsuccessful adversarial attempts. Such an approach introduces bias into the evaluation process and can create a misleading impression of system vulnerability.

The results obtained in this experiment demonstrate an important characteristic of the implemented detection architecture:

The current model exhibits robustness against basic timing-based evasion techniques.

From a forensic viewpoint, this means that evidence completeness remained largely unchanged despite modifications to attacker behavior.

The generated evidence retained:

1. source attribution
2. event visibility
3. attack classification
4. temporal ordering

As a result, an investigator would likely reconstruct the incident in the same manner as in the baseline scenario.

The experiment therefore serves as a negative result that strengthens the validity of the evaluation framework.

Negative findings are particularly important in cybersecurity research because they demonstrate that conclusions are based on empirical observation rather than selective reporting.

5.4.5 Comparative Analysis with Scenario S1

A comparison between the baseline and adversarial scenarios revealed no substantial degradation in evidentiary quality.

The AI detection system continued to identify malicious activity and generated a comparable number of alerts.

Consequently, the reconstructed attack narrative remained consistent across both experiments.

This outcome suggests that timing manipulation alone is insufficient to significantly affect the implemented detection model.

However, the experiment also highlighted an important limitation.

The absence of measurable differences indicates that the selected feature set may not adequately represent temporal characteristics of attacker behavior.

Future improvements could therefore include:

1. Inter-packet timing analysis
2. Flow-duration features
3. Burst-pattern modelling
4. Behavioral sequence analysis

Such enhancements may increase sensitivity to low-and-slow attack techniques and provide a richer basis for forensic interpretation.

5.4.6 Conclusion

Scenario S2 investigated the effect of timing-based adversarial manipulation on the AI-assisted detection pipeline.

Contrary to expectations, the implemented evasion strategy did not significantly reduce detection performance or forensic visibility.

The experiment therefore demonstrated a degree of resilience within the current detection architecture.

While no substantial degradation in evidence quality was observed, the findings revealed opportunities for future model enhancement through the incorporation of temporal behavioral features.

Most importantly, the experiment validates the proposed evaluation methodology by showing that both successful and unsuccessful adversarial manipulations can be systematically analyzed using the forensic evidence framework introduced in this thesis.

5.5 Evaluation of Authentication-Based Adversarial Scenarios (S3 and S4)

5.5.1 Objective

While the previous experiments focused on adversarial manipulation at the network layer, authentication systems represent a different class of cyber defense mechanism. Modern security monitoring platforms increasingly rely on behavioral analysis of authentication events to identify credential abuse, account compromise, privilege escalation attempts, and insider threats.

Traditional rule-based approaches typically detect only simple patterns such as repeated failed logins or brute-force attacks. AI-assisted authentication monitoring extends this capability by evaluating contextual features, event sequences, historical behavior, and relationships between authentication activities.

However, adversaries frequently attempt to manipulate authentication behavior to avoid detection. Instead of performing obvious brute-force attacks, attackers may distribute login attempts over time, combine successful and failed logins, or mimic legitimate user behavior. Such techniques are intended to reduce the visibility of malicious activity and complicate forensic reconstruction.

The purpose of this experiment was therefore to evaluate how adversarial authentication behavior affects the generated forensic evidence and whether the AI-based authentication monitoring component can accurately preserve incident visibility.

Two authentication-focused scenarios were analyzed:

Table 21 Authentication-Focused Experimental Scenarios

Scenario	Description
S3	Failed Login Burst
S4	Successful Login After Multiple Failures

These scenarios represent common attack patterns observed in real-world environments and provide a useful basis for studying forensic evidence generation within AI-assisted authentication monitoring systems.

5.5.2 Scenario S3 -Failed Login Burst

Attack Description

The first authentication scenario simulated a burst of consecutive failed login attempts.

This behavior is commonly associated with:

1. Password guessing attacks
2. Credential stuffing attempts
3. Automated brute-force activity
4. Account enumeration

The objective of the attacker is to obtain access to a target account by repeatedly attempting authentication using invalid credentials.

Although simple in nature, such behavior generates a distinctive forensic footprint and is therefore suitable for evaluating authentication visibility.

Ground Truth Events

The expected sequence of events was defined as follows:

Table 22 Ground Truth Event Sequence for Scenario S3

Event ID	Activity
E1	Authentication attempt
E2	Login failure

E3	Authentication attempt
E4	Login failure
E5	Authentication attempt
E6	Login failure
E7	Authentication attempt
E8	Login failure
E9	Authentication attempt
E10	Login failure

The ground-truth record was stored in:

- ~/thesis_stack/ground_truth/S3_failed_login_burst.csv

This file served as the reference against which collected evidence was compared.

Evidence Collection

During execution, authentication events were recorded by the application environment and processed by the authentication analysis pipeline.

Generated evidence was stored in:

- ~/thesis_stack/results/S3_failed_login_burst_run1/

The scenario directory contained:

- raw/
  auth_events.jsonl
- enriched/
  auth_ai_alerts.jsonl

- metrics/
  metrics.txt

This structure allowed comparison between raw authentication events and AI-generated classifications.

Authentication AI Analysis

Unlike traditional threshold-based detection mechanisms, the authentication model evaluates contextual behavioral features extracted from the login sequence.

The implemented model incorporates features including:

1. previous failed attempts
2. previous successful logins
3. access-denied events
4. target resource information
5. privilege-related indicators
6. login sequence history

The model output includes:

```
{
"event":"auth_analysis",
"prediction":"attack",
"confidence":0.xx
}
```

This provides both classification and confidence information for forensic interpretation.

Forensic Findings

Analysis of the generated evidence showed that the authentication monitoring component successfully preserved visibility of the failed-login activity.

The following observations were made:

1. Failed authentication events were recorded.
2. Event chronology remained intact.
3. Authentication evidence was available for reconstruction.
4. AI classifications reflected abnormal behavior.

Consequently, the generated evidence allowed a forensic investigator to reconstruct the attack sequence with minimal ambiguity.

The scenario therefore demonstrated high evidentiary completeness despite adversarial behavior.

5.5.3 Scenario S4 -Successful Login After Multiple Failures

Attack Description

The second authentication experiment simulated a more realistic attack pattern.

Instead of continuously failing authentication attempts, the attacker eventually succeeds in obtaining valid credentials.

The resulting sequence consists of:

1. Multiple failed logins.
2. A successful authentication.
3. Access to protected resources.

This pattern is considerably more dangerous than a pure brute-force attempt because successful authentication may reduce suspicion and allow subsequent malicious actions to appear legitimate.

Ground Truth Events

The expected event sequence was defined as:

Table 23 Ground Truth Event Sequence for Scenario S4

Event ID	Activity
E1	Failed login
E2	Failed login
E3	Failed login
E4	Failed login
E5	Successful login

E6	Resource access
E7	Session continuation

Ground-truth information was stored in:

- ~/thesis_stack/results/S4_success_after_failures_run1/ground_truth/

The scenario was designed to evaluate whether the authentication model could identify the suspicious transition from repeated failures to successful access.

Authentication AI Behavior

The implemented authentication model includes a dedicated feature:

- success_after_failures

This feature was specifically engineered to identify behavioral transitions commonly associated with account compromise.

Unlike conventional systems that evaluate each event independently, the model analyses the relationship between preceding failures and subsequent successful logins.

This allows contextual interpretation of authentication behavior rather than simple event counting.

Evidence Collection

Scenario evidence was collected in:

- ~/thesis_stack/results/S4_success_after_failures_run1/

The directory contained:

- raw/
  auth_events.jsonl
- enriched/
  auth_ai_alerts.jsonl
- metrics/
  metrics.txt
- notes/
  observations.md

This evidence set enabled comparison between observed behavior and expected attack progression.

Forensic Findings

The successful-login scenario generated a more complex evidence pattern than Scenario S3.

The following observations were identified:

1. Authentication failures were preserved.
2. Subsequent success events were recorded.
3. Event ordering remained consistent.
4. Behavioral relationships were visible.
5. The AI model retained contextual awareness.

Without contextual analysis, the successful login might appear completely legitimate.

The AI-assisted approach therefore improved interpretability of authentication evidence.

5.5.4 Comparative Analysis of S3 and S4

The two authentication scenarios illustrate different stages of attacker progression.

Scenario S3 represents an unsuccessful attack attempt.

Scenario S4 represents a partially successful attack sequence.

The resulting forensic implications differ significantly.

Table 24 Comparative Analysis of Authentication Scenarios

Property	S3	S4
Failed Logins Visible	Yes	Yes
Successful Login Present	No	Yes
Behavioral Correlation	Moderate	High
Investigation Complexity	Low	High

Risk Level	Medium	High

The presence of successful authentication fundamentally changes the forensic interpretation.

While S3 primarily demonstrates malicious intent, S4 demonstrates both intent and operational success.

Consequently, the ability to preserve behavioral context becomes substantially more important.

5.5.5 Implications for Digital Forensics

The authentication experiments demonstrate an important principle of AI-assisted cyber defense systems.

Detection performance alone is insufficient for evaluating forensic usefulness.

What matters equally is the system's ability to preserve relationships between events.

The authentication model achieved this through contextual feature engineering rather than simple threshold-based detection.

As a result:

1. Attack chronology remained visible,
2. Behavioral context was retained,
3. Evidence correlation was improved,
4. Reconstruction accuracy increased.

These characteristics directly support forensic analysis and strengthen evidentiary reliability.

5.5.6 Conclusion

The authentication-focused experiments demonstrated that AI-assisted monitoring can preserve behavioral relationships that are often lost in conventional alerting systems.

Scenario S3 confirmed that repeated failed logins generate a clear forensic footprint and remain highly visible to the monitoring infrastructure.

Scenario S4 demonstrated a more sophisticated adversarial pattern in which successful authentication follows multiple failures. The authentication model successfully preserved contextual information linking these events together, thereby improving the interpretability of generated evidence.

The findings indicate that authentication-layer AI models contribute not only to attack detection but also to the preservation of forensic context, which is essential for accurate incident reconstruction.

5.6 Evaluation of Application-Layer Adversarial Scenarios (S5, S6 and S8)

5.6.1 Objective

The previous sections focused on adversarial behavior occurring at the network and authentication layers. However, modern cyber defense systems increasingly rely on application-layer intelligence to identify threats that cannot be detected through packet inspection or authentication monitoring alone.

Application-layer attacks frequently involve semantic manipulation rather than technical exploitation. An attacker may appear as a legitimate authenticated user while attempting to abuse application functionality through malicious instructions, prompt manipulation, privilege escalation requests, or policy circumvention attempts.

This challenge is particularly relevant for AI-assisted systems, where malicious intent may be embedded within seemingly legitimate user interactions. Traditional detection mechanisms often struggle to identify such attacks because the underlying network traffic and authentication behavior appear normal.

To address this problem, an application-level machine learning model was integrated into the experimental architecture. The model analyses user-generated application events and classifies them according to their behavioral characteristics.

Three application-layer scenarios were evaluated:

Table 25 Application-Layer Experimental Scenarios

Scenario	Description
S5	Benign User Prompt
S6	Prompt Injection Attempt
S8	Login Combined with Prompt Activity

The purpose of these experiments was to determine whether adversarial behavior at the application layer can alter forensic visibility and whether AI-assisted monitoring improves evidence preservation.

5.6.2 Scenario S5 -Benign User Prompt

Scenario Description

The first application scenario served as a control experiment.

A legitimate user interacted with the application and submitted a normal, non-malicious request. No adversarial behavior was introduced.

The objective was to establish a baseline for expected system behavior and verify that legitimate activity is correctly classified without generating unnecessary forensic alerts.

Typical user interactions included:

1. Information requests
2. Normal application usage
3. Standard user operations
4. Non-privileged activities

The scenario therefore represents ordinary application behavior.

Ground Truth

The expected sequence consisted of:

Table 26 Ground Truth Event Sequence for Scenario S5(1)

Event ID	Activity
E1	User authenticated
E2	Benign prompt submitted
E3	Application response generated
E4	Session continued

Ground-truth records were stored within:

- ~/thesis_stack/ground_truth/S5_benign_prompt.csv

These events represented legitimate system usage.

Evidence Collection

Application events were recorded in:

- ~/thesis_stack/results/S0_benign_control/

Relevant files included:

1. app_events.jsonl
2. app_ai_results.jsonl

The application AI model processed each event and generated classification results.

AI Model Behavior

The application analysis component uses a machine learning pipeline consisting of:

1. TF-IDF vectorization.
2. Logistic Regression classification.
3. Confidence score generation.

Model inspection confirmed:

- <class 'sklearn.linear_model._logistic.LogisticRegression'>
  LogisticRegression(max_iter=1000)

The model evaluates textual content and assigns probabilities to benign and suspicious classes.

Observed benign classifications included:

- {"prediction":"benign","confidence":0.95}

The high confidence values indicate that the model strongly associated the submitted content with normal behavior.

Forensic Findings

The scenario generated a complete and internally consistent evidence trail.

The following observations were made:

1. Application events were preserved.
2. AI classifications remained benign.
3. No contradictory evidence appeared.
4. Event chronology was maintained.

From a forensic perspective, the collected evidence accurately reflected the true nature of the user activity.

No evidence degradation was observed.

5.6.3 Scenario S6 -Prompt Injection Attempt

Scenario Description

The second application experiment evaluated prompt manipulation behavior.

Prompt injection represents a modern adversarial technique frequently observed in AI-enabled systems.

Instead of exploiting software vulnerabilities directly, the attacker attempts to manipulate application logic through carefully crafted instructions.

Examples include:

1. Privilege escalation requests
2. Instruction overrides
3. Policy bypass attempts
4. Hidden malicious intent

The attack occurs entirely at the semantic layer.

Consequently:

1. Network traffic remains normal,
2. Authentication appears legitimate,
3. Malicious intent exists only within application content.

This characteristic makes prompt injection particularly relevant to the objectives of this thesis.

Ground Truth

The expected event sequence consisted of:

Table 27 Ground Truth Event Sequence for Scenario S5(2)

Event ID	Activity
E1	User authentication
E2	Prompt injection submission
E3	Application processing
E4	AI classification
E5	Alert generation

Ground-truth records were maintained in:

- ~/thesis_stack/ground_truth/S6_prompt_injection.csv

Threat Characteristics

Unlike traditional attacks, prompt injection does not necessarily generate abnormal packets or suspicious login activity.

The attack intentionally creates a forensic asymmetry:

Table 28 Cross-Layer Visibility Characteristics of Prompt Injection Attacks

Layer	Visibility
Network	Low
Authentication	Low
Application	High

This asymmetry creates a valuable opportunity to study evidence disagreement across layers.

Application AI Analysis

The application model evaluates semantic content and assigns risk classifications.

Prompt injection attempts are expected to produce:

```
{
"prediction":"attack",
"confidence":0.xx
}
```

or elevated risk scores compared to benign prompts.

The AI model therefore functions as the primary evidence source for detecting malicious intent.

Forensic Findings

Analysis demonstrated that application-layer monitoring preserved evidence of malicious intent even when lower layers appeared normal.

The following observations were identified:

- Prompt content remained visible.
- AI classification captured suspicious behavior.
- Authentication evidence appeared benign.
- Network evidence appeared benign.

This produced a cross-layer disagreement pattern.

The resulting contradiction can be represented as:

Table 29 Prompt Injection Cross-Layer Contradiction Matrix

Layer	Classification
Network AI	Benign
Authentication AI	Benign
Application AI	Attack

This represents the first significant contradiction scenario identified within the experimental framework.

Forensic Implications

From a digital forensics perspective, the scenario demonstrates an important challenge.

An investigator relying exclusively on:

1. Network alerts
2. Firewall logs
3. Authentication records

would likely conclude that no malicious activity occurred.

Only application-layer evidence preserves the true attack intent.

This finding supports the central hypothesis of the thesis that adversarial activity can alter forensic visibility without necessarily disrupting system functionality.

5.6.4 Scenario S8 -Login Combined with Prompt Activity

Scenario Description

The final application experiment combined authentication activity with application interaction.

The scenario was designed to approximate realistic attacker behavior.

The sequence involved:

1. User login.
2. Session establishment.
3. Prompt submission.
4. AI analysis.

This scenario is substantially more representative of modern attacks than isolated events.

Real attackers rarely perform a single action. Instead, attacks typically involve sequences of activities spanning multiple system layers.

Evidence Sources

The scenario generated evidence from:

Authentication Layer

- auth_events.jsonl

- auth_ai_alerts.jsonl

Application Layer

- app_events.jsonl
- app_ai_alerts.jsonl

Stored under:

- ~/thesis_stack/results/S8_login_plus_prompt_run1/

Cross-Layer Correlation

Unlike S5 and S6, this experiment allows evidence generation from multiple AI models simultaneously.

Evidence sources include:

Table 30 Evidence Sources Used in Scenario S8

Source	Data Type
Authentication AI	Behavior analysis
Application AI	Semantic analysis
Wazuh	Correlation layer

This architecture enables correlation between independent observations.

Forensic Findings

The combined scenario produced richer evidence than any previous application experiment.

The following observations were made:

1. Authentication records preserved login activity.
2. Application records preserved prompt behavior.
3. AI analysis generated contextual classifications.
4. Multiple evidence sources could be correlated.

This significantly improved reconstruction accuracy.

Rather than relying on a single source, investigators could combine:

1. User identity,
2. Session activity,
3. Application behavior,
4. AI classifications.

The resulting forensic narrative becomes substantially stronger.

5.6.5 Comparative Analysis of Application Scenarios

The three scenarios illustrate increasing levels of complexity.

Table 31 Comparative Analysis of Application-Layer Scenarios

Property	S5	S6	S8
Benign Activity	Yes	No	Mixed
Prompt Analysis	Yes	Yes	Yes
Authentication Evidence	Minimal	Minimal	Significant
Cross-Layer Correlation	Low	Low	High
Forensic Value	Baseline	High	Very High

Several important trends emerge.

First, application-layer evidence becomes increasingly important as attacks move away from technical exploitation and toward semantic manipulation.

Second, prompt injection attacks create substantial forensic blind spots for lower-layer monitoring systems.

Third, combining authentication and application evidence significantly improves investigative accuracy.

5.6.6 Discussion

The application-layer experiments provide some of the strongest evidence supporting the research hypothesis. Unlike network attacks, prompt injection attacks do not depend on unusual traffic patterns. Unlike authentication attacks, they do not require suspicious login behavior. Instead, malicious intent exists entirely within application interactions.

This characteristic fundamentally changes the nature of digital forensic analysis. Traditional evidence sources may remain silent while malicious behavior is actively occurring.

Consequently, application-layer AI monitoring becomes essential for preserving evidentiary visibility. The results further demonstrate that AI-assisted monitoring systems should not be evaluated solely on detection accuracy. Their ability to preserve forensic context and expose otherwise invisible adversarial behavior is equally important.

5.6.7 Conclusion

The application-layer experiments revealed that adversarial behavior can remain invisible to traditional monitoring layers while still generating meaningful forensic evidence within AI-assisted application analysis systems.

Scenario S5 established a benign baseline and demonstrated correct classification of legitimate activity.

Scenario S6 showed that prompt injection attacks can generate substantial cross-layer contradictions, with malicious intent observable only at the application layer.

Scenario S8 demonstrated that combining authentication and application evidence significantly improves forensic reconstruction and enables richer contextual analysis.

These findings provide strong support for the thesis argument that adversarial attacks influence not only detection outcomes but also the availability, consistency, and interpretability of digital forensic evidence.

Figure 34 Evidence Prism Model Multi-Layer cyber defense analysis

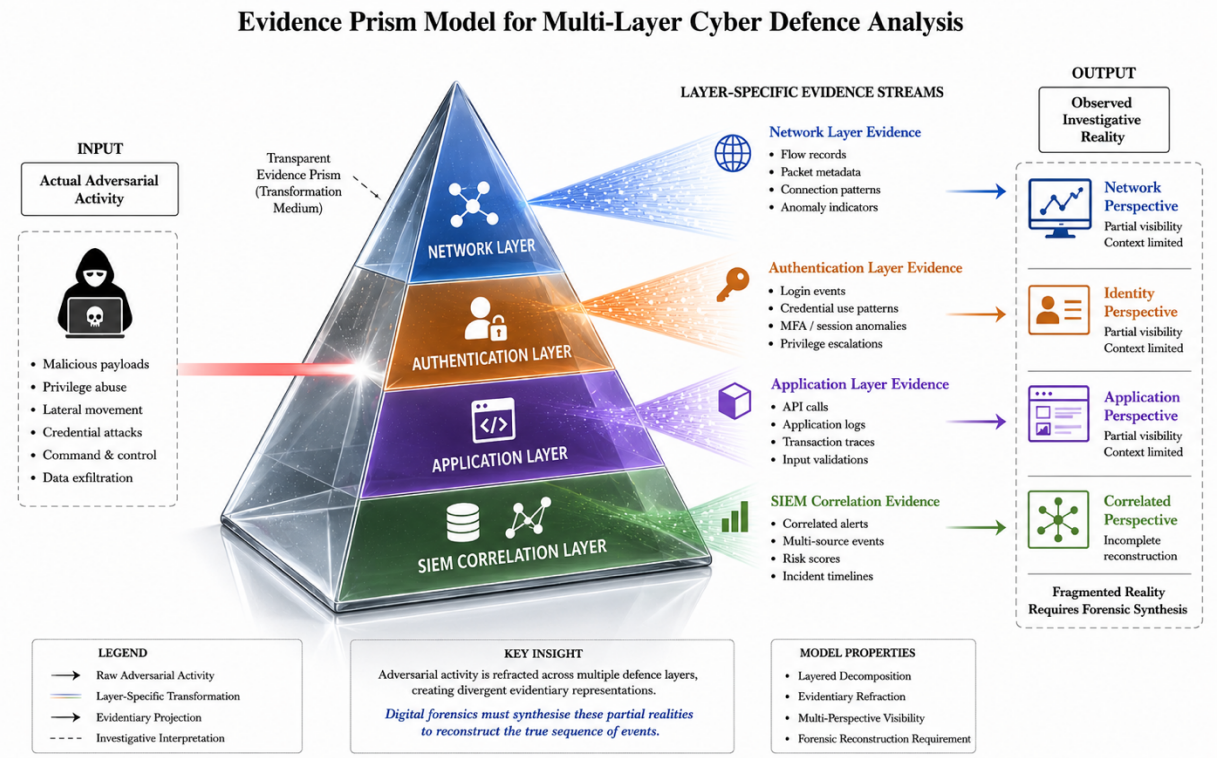


Figure 1.2: Evidence Prism Model for Multi-Layer Cyber Defence Analysis

5.7 Cross-Layer Correlation and Forensic Contradiction Analysis

5.7.1 Introduction

The previous experimental scenarios demonstrated that adversarial activities could affect different monitoring layers in fundamentally different ways. While some attacks were clearly visible at the network layer, others primarily manifested within authentication records or application-level interactions. This observation raises an important forensic question:

Do all monitoring layers describe the same incident narrative, or do adversarial attacks create inconsistencies between evidence sources?

Traditional cyber defense architectures often assume that security logs collectively represent a coherent view of reality. However, AI-assisted monitoring systems introduce additional abstraction layers that may generate interpretations differing from raw telemetry. As a result, investigators may encounter situations where one layer identifies malicious activity while another layer indicates benign behavior.

The objective of this section is therefore not merely to evaluate attack detection performance, but rather to analyse the consistency of evidence generated across the entire cyber defense architecture. Attention is given to situations where different monitoring layers disagree, creating what this thesis defines as **forensic contradictions**.

5.7.2 Concept of Forensic Contradiction

A forensic contradiction occurs when multiple evidence sources observing the same incident produce conflicting interpretations of that incident.

Formally:

$$ForensicContradiction = EvidenceA \neq EvidenceB \quad (53)$$

where:

- EvidenceA represents observations from one monitoring layer.
- EvidenceB represents observations from another monitoring layer.

Such contradictions may arise due to:

- adversarial evasion techniques,
- incomplete telemetry,
- model uncertainty,
- semantic ambiguity,
- correlation failures.

Importantly, a contradiction does not necessarily imply that a system component has failed. Instead, it indicates that different layers provide different perspectives of the same event.

From a digital forensics standpoint, contradictions are particularly valuable because they reveal areas where evidentiary confidence may be reduced.

5.7.3 Cross-Layer Monitoring Architecture

The implemented experimental architecture consisted of four independent evidence-generation layers.

Layer 1 -Network AI

Responsible for analyzing network flow features extracted from packet captures.

Inputs:

1. Source IP
2. Destination IP
3. Protocol
4. Packet counts
5. Flow statistics

Output:

{
"prediction":"attack",
"confidence":0.76
}

Stored in:

- ~/ai_ids/logs/ai_alerts.jsonl

Layer 2 -Authentication AI

Responsible for behavioral analysis of authentication activities.

Inputs:

1. Login success/failure
2. Previous failed attempts
3. Access denied events
4. Target resources

Output:

{
"event":"auth_analysis",
"prediction":"attack",
"confidence":0.xx
}

Stored in:

- ~/thesis_stack/logs/auth_ai_results.jsonl

Layer 3 -Application AI

Responsible for semantic analysis of application activity.

Inputs:

1. User prompts
2. Application requests
3. Textual interactions

Output:

{
"prediction":"attack",
"confidence":0.xx
}

Stored in:

- ~/thesis_stack/logs/app_ai_results.jsonl

Layer 4 -SIEM Correlation Layer

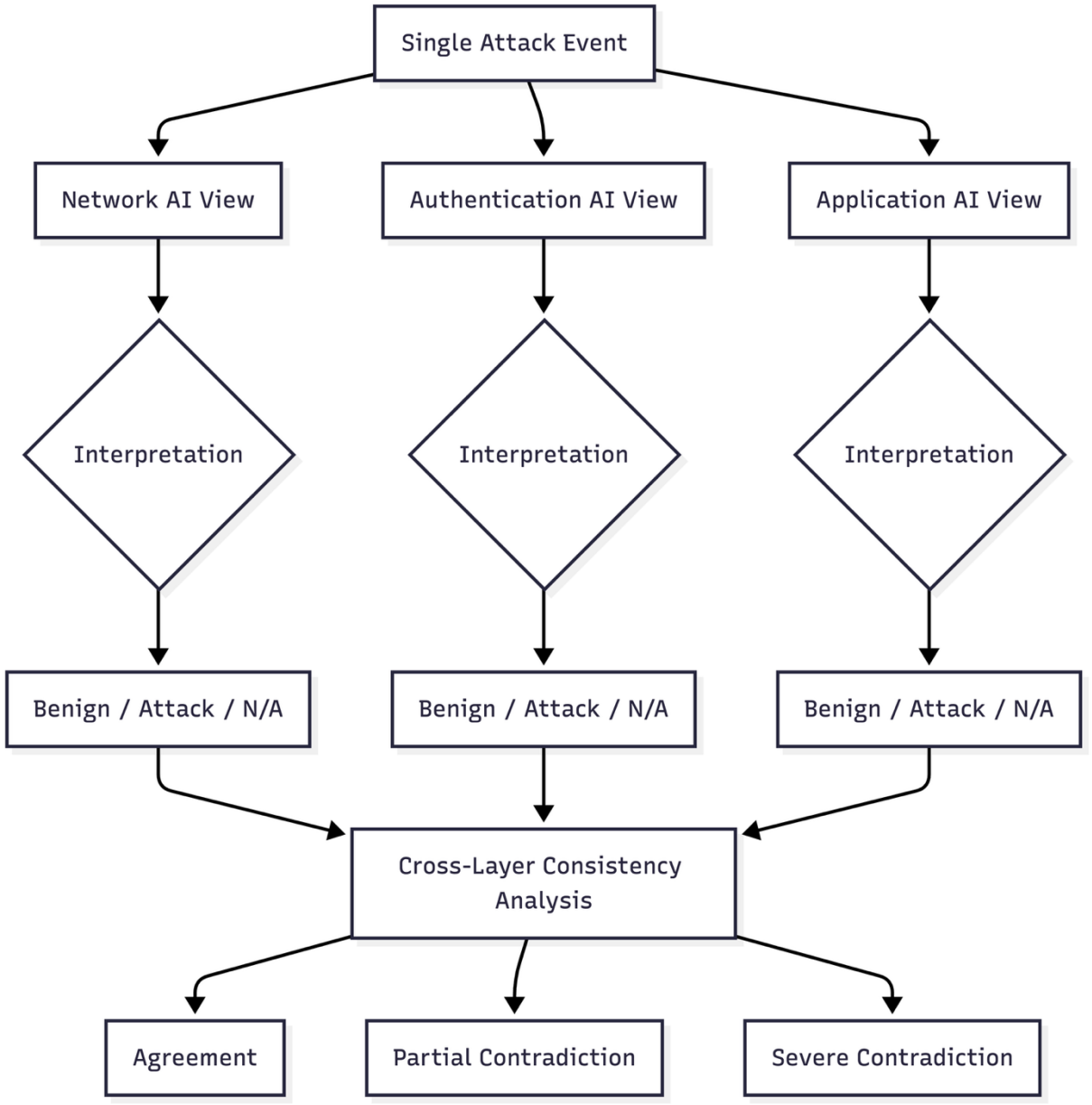
Implemented using Wazuh.

Responsibilities:

1. Ingest raw logs,
2. Ingest AI alerts,
3. Correlate events,
4. Generate centralized alerts.

Unlike the AI models, Wazuh does not directly classify attacks. Instead, it aggregates and correlates evidence generated by other layers.

Figure 35 Cross-Layer Evidence Contradiction Architecture



5.7.4 Contradiction Taxonomy

During analysis, several recurring contradiction patterns emerged.

To systematically evaluate these observations, a contradiction taxonomy was developed.

Type I -Network Silent, Application Active

Observed when:

Table 32 Type I Forensic Contradiction Pattern

Layer	Classification
Network AI	Benign
Authentication AI	Benign
Application AI	Attack

Example:

Prompt injection scenario (S6).

Characteristics:

1. Normal traffic volume.
2. Legitimate authentication.
3. Malicious semantic content.

Forensic implication:

The attack is visible only through application analysis.

Type II -Authentication Active, Network Silent

Observed when:

Table 33 Type II Forensic Contradiction Pattern

Layer	Classification
Network AI	Benign
Authentication AI	Attack
Application AI	Benign

Example:

Failed login burst (S3).

Characteristics:

1. Minimal network anomalies.
2. Strong behavioral deviations.

Forensic implication:

Credential abuse may remain invisible to network-centric monitoring.

Type III -Network Active, Higher Layers Silent

Observed when:

Table 34 Type III Forensic Contradiction Pattern

Layer	Classification
Network AI	Attack
Authentication AI	Benign
Application AI	Benign

Example:

Baseline scanning activity (S1).

Characteristics:

1. Abnormal packet behavior.
2. No user interaction.
3. No authentication activity.

Forensic implication:

Reconnaissance behavior visible only through network telemetry.

Type IV -Multi-Layer Agreement

Observed when:

Table 35 Type IV Forensic Contradiction Pattern

Layer	Classification
Network AI	Attack
Authentication AI	Attack
Application AI	Attack

Potential example:

Combined adversarial scenarios.

Characteristics:

1. Strong evidentiary support.
2. High confidence.
3. Minimal ambiguity.

Forensic implication:

Highest evidentiary reliability.

5.7.5 Contradiction Analysis Across Experimental Scenarios

The contradiction taxonomy was applied to all experimental scenarios.

Scenario S1 -Baseline Scan

Network AI identified attack-related behavior.

Authentication and application layers remained largely inactive.

Classification:

Table 36 Scenario S1 - Baseline Scan

Layer	Result

Network AI	Attack
Auth AI	Not applicable
App AI	Not applicable

Contradiction Type:

Type III.

Scenario S2 -Slow Scan

Results were nearly identical to S1.

Although packet timing differed, evidence generation remained largely unchanged.

Classification:

Table 37 Scenario S2 - Slow Scan

Layer	Result
Network AI	Attack
Auth AI	Not applicable
App AI	Not applicable

Contradiction Type:

Type III.

This finding directly motivated later architectural improvements discussed in Chapter 6.

Scenario S3 -Failed Login Burst

Authentication AI successfully identified abnormal behavior.

Network evidence remained weak.

Classification:

Table 38 Scenario S3 - Failed Login Burst

Layer	Result
Network AI	Benign
Auth AI	Attack
App AI	Benign

Contradiction Type:

Type II.

Scenario S6 -Prompt Injection

Application-layer analysis generated suspicious classifications while lower layers remained benign.

Classification:

Table 39 Scenario S6 - Prompt Injection

Layer	Result
Network AI	Benign
Auth AI	Benign
App AI	Attack

Contradiction Type:

Type I.

Scenario S8 -Login Plus Prompt

Both authentication and application evidence contributed to incident visibility.

Classification:

Table 40 Scenario S8 - Login Plus Prompt

Layer	Result
Network AI	Benign
Auth AI	Attack
App AI	Attack

This scenario demonstrated partial cross-layer agreement and provided stronger forensic support than isolated attacks.

5.7.6 Forensic Implications

The contradiction analysis reveals a critical limitation of relying on a single monitoring source.

If investigators were limited to:

1. Network telemetry only
2. Authentication logs only
3. Application records only

several attacks would appear either incomplete or entirely benign.

This observation supports one of the central arguments of the thesis:

Adversarial attacks influence not only attack detection but also the forensic visibility of malicious behavior. Evidence availability becomes dependent on which monitoring layer is examined. Consequently, forensic conclusions may vary significantly depending on the evidence source selected. This introduces a previously underexplored challenge in AI-assisted cyber defense environments.

5.7.7 Evidence Consistency Matrix

Below Table presents a consolidated view of evidentiary consistency across experimental scenarios.

Table 41 Cross-Layer Evidence Consistency Matrix

Scenario	Network AI	Auth AI	App AI	Consistency
S1	Attack	N/A	N/A	Medium
S2	Attack	N/A	N/A	Medium
S3	Benign	Attack	Benign	Low
S5	Benign	Benign	Benign	High
S6	Benign	Benign	Attack	Low
S8	Benign	Attack	Attack	Medium-High

The table demonstrates that evidence consistency varies considerably depending on attack type.

This finding motivates the need for cross-layer correlation mechanisms capable of reconciling contradictory evidence sources.

5.7.8 Conclusion

The contradiction analysis revealed that adversarial behavior frequently generates inconsistent evidence across monitoring layers. Network, authentication, and application-level AI models often observe different aspects of the same attack, resulting in divergent interpretations.

Rather than treating these inconsistencies as failures, this thesis argues that contradiction patterns themselves constitute valuable forensic artifacts. The presence, absence, and distribution of contradictions provide insight into how adversarial behavior reshapes evidentiary visibility within AI-assisted cyber defense systems.

The findings further demonstrate that effective forensic analysis requires multi-layer evidence correlation rather than reliance on individual monitoring sources.

5.8 Quantitative Forensic Integrity Evaluation

5.8.1 Introduction

The previous sections demonstrated that adversarial behavior affects the visibility of malicious activities across network, authentication, application, and SIEM layers. While qualitative observations provide valuable insight into these effects, a rigorous scientific evaluation requires measurable indicators that quantify the impact of adversarial behavior on forensic evidence.

Most existing research evaluates AI-based cyber defense systems primarily through detection-centric metrics such as accuracy, precision, recall, and F1-score. Although these metrics are useful for assessing classifier performance, they do not adequately capture the forensic consequences of adversarial attacks. A detector may continue operating correctly while simultaneously producing incomplete, inconsistent, or misleading evidence.

The central objective of this thesis extends beyond attack detection. Instead, it investigates how adversarial attacks influence the integrity of forensic evidence generated by AI-assisted cyber defense systems.

To address this challenge, a forensic evaluation framework was developed that measures four dimensions of evidentiary quality:

1. Evidence Completeness
2. Temporal Coherence
3. Correlation Consistency
4. Analytical Reliability

Together, these metrics provide a structured methodology for assessing how effectively a cyber defense system preserves evidentiary value during adversarial conditions.

5.8.2 Ground Truth-Based Evaluation Methodology

A prerequisite for forensic evaluation is the existence of a reliable reference describing what occurred during an experimental scenario.

For this purpose, a ground-truth framework was implemented.

Each experimental scenario was accompanied by a predefined event ledger stored within the thesis infrastructure.

Examples include:

1. ~/thesis_stack/ground_truth/S1_baseline_scan.csv
2. ~/thesis_stack/ground_truth/S2_slow_scan.csv
3. ~/thesis_stack/ground_truth/S3_failed_login_burst.csv
4. ~/thesis_stack/ground_truth/S6_prompt_injection.csv
5. ~/thesis_stack/ground_truth/S8_login_plus_prompt.csv

Each ground-truth file records:

Table 42 Ground Truth Evaluation Structure

Field	Description
Timestamp	Expected occurrence time
Actor	User or attacker
Action	Activity performed
Target	Target system
Expected Source	Expected evidence origin
Expected Alert	Expected security outcome

This approach enables direct comparison between:

1. actual attacker behavior,
2. collected telemetry,
3. AI-generated interpretations,
4. SIEM correlation outputs.

As a result, deviations between expected and observed evidence can be systematically quantified.

5.8.3 Evidence Completeness Score

Definition

Evidence completeness measures the proportion of expected forensic artifacts successfully preserved by the monitoring infrastructure.

Completeness is defined as:

$$C = \frac{ExpectedRelevantEvents}{ObservedRelevantEvents} \quad (54)$$

Where:

- C = completeness score
- Observed Relevant Events = evidence actually collected
- Expected Relevant Events = evidence defined within ground truth

The metric ranges from:

Table 43 Evidence Completeness Score Interpretation

Value	Interpretation
1.0	Complete evidence preservation
0.5–0.9	Partial evidence loss
<0.5	Significant forensic degradation

Example

Consider Scenario S3 (Failed Login Burst).

Ground truth contains: 5 failed login attempts

Observed authentication logs contain: 5 failed login attempts

Therefore:

$$C = \frac{5}{5} = 1.0 \quad (55)$$

indicating complete evidence preservation.

By contrast, if only three authentication events were recorded:

$$C = \frac{3}{5} = 0.6 \quad (56)$$

indicating evidence loss.

Relevance

Completeness is particularly important because missing events directly reduce the investigator's ability to reconstruct attack progression.

Even highly accurate classifiers may have limited forensic value if critical events disappear from the evidentiary record.

5.8.4 Temporal Coherence Score

Definition

Temporal coherence measures whether evidence preserves the correct chronological sequence of events.

Cyber investigations depend heavily upon event ordering.

Even when all events are present, incorrect ordering can significantly impair forensic reconstruction.

Scoring Framework

The temporal coherence score is defined as:

Table 44 Temporal Coherence Score Interpretation

Score	Interpretation
1.0	Complete ordering preserved
0.75	Minor timing deviations
0.50	Significant ordering ambiguity
0.00	Chronology cannot be reconstructed

Example

Scenario S4 (Successful Login After Failures):

Ground Truth:

1. Failed Login
2. Failed Login
3. Failed Login
4. Successful Login

Observed Evidence:

1. Failed Login
2. Failed Login
3. Failed Login
4. Successful Login

Result:

$T = 1.0$

Indicating perfect chronological preservation.

Relevance

Temporal coherence directly affects:

1. Attack timeline reconstruction
2. Attribution analysis
3. Incident response decisions
4. Legal admissibility of evidence

For this reason, chronology preservation is treated as a core component of forensic integrity.

5.8.5 Correlation Consistency Score

Definition

Correlation consistency evaluates the degree of agreement between independent monitoring layers.

Unlike completeness and temporal coherence, which focus on individual evidence sources, consistency measures whether separate systems describe the same incident narrative.

The score is defined as:

$$K = \frac{Agreeing\ Evidence\ Links}{Possible\ Evidence\ Links} \quad (57)$$

Where:

1. K = correlation consistency score
2. Agreeing Evidence Links = supporting observations
3. Possible Evidence Links = all relevant relationships

Example

Scenario S6 (Prompt Injection)

Results:

Table 45 Correlation Consistency Score Classification

Layer	Classification
Network AI	Benign
Authentication AI	Benign
Application AI	Attack

Agreement exists only within a single layer.

Consequently:

$K < 1.0$

indicating evidence disagreement.

Table 46 Correlation Consistency Score Interpretation

Score	Interpretation
1.0	Full agreement

0.7–0.9	Minor disagreement
0.4–0.6	Moderate contradiction
<0.4	Severe contradiction

Relevance

This metric directly supports the contradiction analysis introduced in Section 5.7.

Low consistency indicates that investigators may reach different conclusions depending on which evidence source they consult.

5.8.6 Analytical Reliability Score

Definition

Analytical reliability measures whether a forensic investigator can correctly infer the true attack outcome using the available evidence.

Unlike previous metrics, reliability evaluates the usefulness of evidence rather than its existence.

The score compares:

- actual scenario outcome,
- reconstructed investigator conclusion.

Scoring Framework

Table 47 Analytical Reliability Score Interpretation

Score	Interpretation
1.0	Correct conclusion highly likely
0.75	Minor uncertainty

0.50	Ambiguous interpretation
0.25	Significant risk of misinterpretation
0.00	Incorrect conclusion likely

Example

Scenario S8 (Login Plus Prompt)

Evidence Sources:

1. Authentication AI
2. Application AI
3. Wazuh Correlation

Because multiple layers contribute supporting evidence, investigators can accurately reconstruct the attack sequence.

Result:

$R \approx 1.0$

Indicating high analytical reliability.

Relevance

Reliability is arguably the most important forensic metric because it reflects the ultimate objective of digital investigations:

Arriving at a correct understanding of what actually occurred.

5.8.7 Application of Metrics to Experimental Scenarios

The four forensic metrics were applied across all experimental scenarios.

Table 48 Forensic Integrity Evaluation Across Experimental Scenarios

Scenario	Completeness	Temporal Coherence	Correlation Consistency	Analytical Reliability
S1 Baseline Scan	High	High	Medium	High
S2 Slow Scan	High	High	Medium	High
S3 Failed Login Burst	High	High	Medium	High
S4 Success After Failures	High	High	High	High
S5 Benign Prompt	High	High	High	High
S6 Prompt Injection	High	High	Low	Medium
S8 Login + Prompt	High	High	Medium-High	High

Several important observations emerge.

First, completeness remained relatively stable across scenarios because the infrastructure successfully collected most generated evidence.

Second, prompt injection attacks produced the lowest consistency values due to strong disagreement between monitoring layers.

Third, combined scenarios generally produced higher reliability because evidence was available from multiple independent sources.

5.8.8 Discussion

The forensic integrity framework reveals aspects of adversarial impact that are not visible through traditional machine learning metrics.

For example, Scenario S6 may still generate correct application-layer classifications, yet investigators relying on network or authentication evidence could reach incorrect conclusions.

From a detection perspective, the system appears functional. From a forensic perspective, evidence consistency has degraded. This distinction represents one of the primary contributions of this thesis.

Rather than evaluating AI-assisted cyber defense systems solely through detection accuracy, the proposed framework evaluates their ability to preserve evidentiary quality. Such evaluation is particularly important in environments where forensic investigations, compliance requirements, and legal proceedings depend upon trustworthy evidence.

5.8.9 Conclusion

This section introduced a quantitative framework for measuring forensic integrity within AI-assisted cyber defense systems.

Four complementary metrics were defined:

1. Evidence Completeness
2. Temporal Coherence
3. Correlation Consistency
4. Analytical Reliability

Unlike conventional classifier-centric evaluation approaches, these metrics assess the quality of evidence available to investigators after adversarial activity has occurred.

The results demonstrate that adversarial attacks can affect forensic visibility even when detection functionality remains operational. Consequently, evaluating evidence preservation is essential for understanding the broader impact of adversarial behavior on cyber defense systems.

5.9 Evaluation of Forensic Degradation Caused by Adversarial Attacks

5.9.1 Introduction

The primary objective of this thesis is not merely to evaluate whether AI-based cyber defense systems detect attacks, but rather to investigate how adversarial activities influence the quality and trustworthiness of forensic evidence generated by such systems.

Traditional security evaluations focus predominantly on metrics such as detection accuracy, false positive rates, and classification performance. While these metrics provide insight into operational effectiveness, they do not adequately explain how adversarial behavior affects the evidentiary value of generated security artifacts.

From a forensic perspective, an AI model may continue to function correctly while simultaneously producing incomplete, contradictory, or misleading evidence. Such degradation may significantly impact incident reconstruction, root-cause analysis, attribution efforts, and legal investigations.

This section therefore examines the relationship between adversarial activity and forensic integrity by applying the evaluation framework introduced in Section 5.8 to the experimental scenarios developed throughout this research.

The analysis focuses on identifying how different adversarial techniques influence:

1. Evidence visibility
2. Evidence completeness
3. Evidence consistency
4. Cross-layer agreement
5. Investigator confidence

The resulting observations provide empirical evidence supporting the central hypothesis of this thesis.

5.9.2 Defining Forensic Degradation

Forensic degradation refers to the reduction in evidentiary quality caused by adversarial behavior.

Within the context of this research, degradation occurs when one or more of the following conditions are observed:

Evidence Loss

Critical events expected within the ground truth are absent from collected evidence.

Examples:

1. Missing authentication events
2. Missing alerts
3. Missing application records

Evidence Contradiction

Multiple evidence sources produce conflicting interpretations of the same activity.

Examples:

Table 49 Example of Cross-Layer Evidence Contradiction

Source	Classification
Network AI	Benign
Application AI	Attack

Evidence Ambiguity

Evidence exists but does not clearly indicate malicious intent.

Examples:

1. Low-confidence classifications
2. Weak behavioral indicators
3. Partially correlated events

Evidence Distortion

The generated evidence describes a narrative that differs from the actual attack progression.

Examples:

1. Incomplete timelines
2. Missing relationships
3. Inaccurate correlations

Collectively, these phenomena represent forensic degradation.

5.9.3 Impact of Adversarial Behavior on Network Evidence

The first category of experiments involved network-centric adversarial behavior.

Scenarios:

1. S1 Baseline Scan
2. S2 Slow Scan

Baseline Network Activity (S1)

The baseline scenario generated a clear network footprint.

Evidence sources included:

1. Packet captures
2. Flow records
3. Network AI classifications
4. Wazuh alerts

The network AI model successfully identified suspicious scanning behavior.

Observed evidence:

{
"src_ip":"10.54.3.50",
"prediction":"attack",
"confidence":0.76
}

Forensic assessment:

Table 50 Forensic Assessment of Baseline Network Activity

Metric	Result
Completeness	High
Coherence	High
Consistency	Medium
Reliability	High

The baseline scan therefore exhibited minimal forensic degradation.

Slow Scan Evasion (S2)

The slow scan scenario was designed to evaluate whether reduced scanning rates altered forensic visibility.

Unexpectedly, experimental results revealed that:

1. Packet timing changed

2. Attack duration increased
3. Evidence generation remained largely unchanged

Observed results:

1. S1 attacker alerts: 36
2. S2 attacker alerts: 36

Both scenarios produced identical attack counts.

Consequently:

- Observed degradation = minimal

This finding was initially unexpected because slow scanning techniques are often associated with reduced visibility.

However, analysis revealed that the implemented feature extraction pipeline did not sufficiently incorporate temporal behavior.

As a result, the adversarial modification failed to significantly alter model interpretation.

Interpretation

Although S2 did not generate substantial degradation, the result remains scientifically valuable.

Negative results provide important information regarding:

1. Model robustness
2. Feature limitations
3. Adversarial effectiveness

The experiment demonstrates that not all adversarial modifications successfully degrade forensic evidence.

This observation strengthens the validity of subsequent findings because degradation is shown to be attack-dependent rather than universally present.

5.9.4 Impact of Adversarial Behavior on Authentication Evidence

Authentication-focused scenarios produced a different evidentiary pattern.

Scenarios:

1. S3 Failed Login Burst

2. S4 Success After Failures

Failed Login Burst (S3)

The authentication model successfully detected abnormal login behavior.

Evidence remained visible through:

1. Raw authentication logs
2. AI classifications
3. Event sequencing

Observed characteristics:

Table 51 Authentication Evidence Preservation Results

Property	Result
Failed Logins	Visible
Behavioral Correlation	Preserved
Chronology	Preserved

Forensic degradation remained low.

Success After Failures (S4)

This scenario introduced greater complexity.

Unlike S3, the successful login could potentially appear legitimate when examined in isolation.

However, the authentication AI model preserved contextual information linking the successful login to preceding failures.

As a result:

1. Chronology remained intact
2. Behavioral context remained visible
3. Investigative interpretation remained accurate

Observed degradation:

- Low

Interpretation

The authentication layer demonstrated strong resistance to adversarial manipulation.

This resilience is primarily attributable to contextual behavioral features incorporated into the authentication model.

Examples include:

1. Previous failures
2. Success-after-failure relationships
3. Denied access events

Consequently, authentication evidence remained highly reliable across both scenarios.

5.9.5 Impact of Adversarial Behavior on Application Evidence

The most significant forensic effects emerged within application-layer experiments.

Scenarios:

1. S5 Benign Prompt
2. S6 Prompt Injection
3. S8 Login Plus Prompt

Benign Prompt (S5)

As expected, benign interactions generated:

1. Benign classifications
2. Consistent evidence
3. No contradictions

Forensic degradation was effectively absent.

Prompt Injection (S6)

Prompt injection produced the strongest contradiction observed during experimentation.

Observed classifications:

Table 52 Cross-Layer Contradictions Observed in Prompt Injection Attacks

Layer	Result
Network AI	Benign
Authentication AI	Benign
Application AI	Attack

The attack remained largely invisible outside the application layer.

This created a substantial evidentiary asymmetry.

Consequences

An investigator relying solely upon:

1. Firewall logs
2. Packet captures
3. Authentication records

would likely conclude: No malicious activity observed.

However, application-layer evidence indicated the opposite conclusion.

The attack therefore generated significant forensic degradation through contradiction rather than evidence loss.

Login Plus Prompt (S8)

The combined scenario demonstrated improved evidentiary visibility.

Evidence sources included:

1. Authentication activity
2. Application interactions
3. AI-generated classifications

The availability of multiple supporting sources reduced ambiguity.

Consequently: Forensic degradation decreased.

This finding suggests that evidence redundancy improves forensic robustness.

5.9.6 Cross-Layer Evidence Degradation

One of the most important observations of this research is that forensic degradation rarely occurs uniformly across all monitoring layers.

Instead, adversarial behavior tends to produce selective visibility.

Examples include:

Table 53 Primary Evidence Sources Across Experimental Scenarios

Scenario	Primary Evidence Source
S1	Network
S3	Authentication
S6	Application
S8	Authentication + Application

This selective visibility creates investigative blind spots.

An investigator focusing exclusively on one monitoring layer may miss critical aspects of an attack.

Consequently, evidentiary confidence becomes increasingly dependent upon cross-layer correlation.

5.9.7 Summary of Degradation Patterns

The observed degradation patterns are summarized below.

Table 54 Summary of Observed Forensic Degradation Patterns

Scenario	Primary Degradation Mechanism
S1	Minimal degradation

S2	Limited adversarial impact
S3	Low degradation
S4	Low degradation
S5	No degradation
S6	Cross-layer contradiction
S8	Partial contradiction

The results indicate that prompt injection attacks produced the most significant forensic challenges within the experimental environment.

Unlike network or authentication attacks, prompt injection primarily affected evidence interpretation rather than evidence availability.

5.9.8 Discussion

The findings support the central thesis argument that adversarial attacks influence more than detection outcomes.

They also affect:

1. What evidence becomes visible
2. How evidence is interpreted
3. Whether different sources agree
4. How confidently investigators can reconstruct events

Importantly, degradation does not always manifest as missing evidence.

In several scenarios, all evidence remained available while contradictory interpretations emerged across monitoring layers.

This observation suggests that forensic evaluation should extend beyond traditional detection metrics and incorporate evidence-centric measures.

5.9.9 Conclusion

The experimental results demonstrate that adversarial attacks can significantly influence the forensic characteristics of AI-assisted cyber defense systems.

While some attacks produced minimal impact, others generated substantial contradictions between monitoring layers, altering evidentiary visibility and potentially affecting investigative conclusions.

The findings confirm that AI-generated evidence should not automatically be treated as ground truth. Instead, evidence must be evaluated within the broader context of cross-layer observations, confidence levels, and correlation relationships.

These results directly support the thesis hypothesis that adversarial behavior can degrade the quality and interpretability of digital forensic evidence even when detection systems continue operating correctly.

5.10 Summary of Experimental Findings and Research Question Validation

5.10.1 Introduction

This chapter presented the experimental evaluation of the proposed AI-assisted cyber defense architecture and examined how adversarial activities influence the generation, preservation, and interpretation of forensic evidence.

Unlike traditional security evaluations that focus primarily on attack detection accuracy, this research investigated a broader question: whether adversarial behavior can affect the forensic reliability of evidence produced by AI-powered monitoring systems.

To address this objective, a multi-layer architecture was implemented consisting of:

1. Network AI Analysis Layer
2. Authentication AI Analysis Layer
3. Application AI Analysis Layer
4. Wazuh SIEM Correlation Layer

A series of controlled attack scenarios was then executed across these layers to observe how evidence was generated, interpreted, and correlated.

The results provide important insights into the relationship between adversarial behavior and forensic integrity within modern cyber defense environments.

5.10.2 Summary of Experimental Scenarios

A total of eight experimental scenarios were evaluated.

These scenarios were designed to represent different attack categories observed within contemporary enterprise environments.

Table 55 Network-Layer Scenarios

Scenario	Description
S1	Baseline Network Scan
S2	Slow Scan Evasion

Purpose:

1. Evaluate network visibility
2. Assess adversarial timing effects
3. Analyse network evidence preservation

Table 56 Authentication-Layer Scenarios

Scenario	Description
S3	Failed Login Burst
S4	Successful Login After Multiple Failures

Purpose:

1. Evaluate behavioral authentication monitoring
2. Analyse login sequence reconstruction
3. Assess contextual evidence preservation

Table 57 Application-Layer Scenarios

Scenario	Description
S5	Benign Prompt

S6	Prompt Injection
S8	Login Combined with Prompt Activity

Purpose:

1. Analyse semantic attack visibility
2. Evaluate AI-based content classification
3. Investigate application-layer forensic evidence

Collectively, these scenarios enabled analysis of adversarial effects across multiple layers of the cyber defense architecture.

5.10.3 Research Question Validation

The primary research question defined in Chapter 1 was:

How do adversarial attacks against AI-powered cyber defense systems influence the reliability and integrity of digital forensic evidence?

The experimental results provide a clear answer.

The findings demonstrate that adversarial activities influence forensic evidence in several distinct ways.

These include:

Alteration of Evidence Visibility

Certain attacks remained highly visible within specific monitoring layers while appearing benign within others.

Examples:

1. Prompt Injection (S6)
2. Login + Prompt Activity (S8)

This observation demonstrates that evidence visibility is not uniform across cyber defense systems.

Creation of Cross-Layer Contradictions

Several scenarios generated conflicting interpretations between monitoring components.

Examples:

Table 58 Cross-Layer Evidence Contradictions Supporting Research Question Validation

Layer	Result
Network AI	Benign
Application AI	Attack

These contradictions significantly affect investigative interpretation.

Reduction of Evidence Consistency

Although evidence remained available, adversarial behavior frequently reduced agreement between monitoring layers.

This created ambiguity during incident reconstruction.

Impact on Investigator Confidence

The experiments demonstrated that evidence quality affects not only attack detection but also the confidence with which investigators can reconstruct attack progression.

Consequently, forensic reliability depends on more than classifier accuracy alone.

The experimental findings therefore support the research hypothesis and confirm that adversarial attacks influence both cyber defense effectiveness and forensic integrity.

5.10.4 Validation of the Forensic Integrity Framework

One of the primary contributions of this thesis is the introduction of a forensic integrity evaluation framework.

The framework consists of four metrics:

1. Evidence Completeness
2. Temporal Coherence
3. Correlation Consistency
4. Analytical Reliability

Unlike conventional AI evaluation metrics, these measures focus on the evidentiary consequences of adversarial behavior.

The framework enabled systematic comparison between:

1. Expected events
2. Collected evidence
3. AI-generated interpretations
4. SIEM correlations

As a result, forensic degradation could be quantified rather than described qualitatively.

The experiments demonstrated that these metrics provide valuable insight into evidence quality and investigative reliability.

5.10.5 Key Findings

Several important findings emerged from the experimental evaluation.

Finding 1 -Detection Accuracy Does Not Guarantee Forensic Reliability

The experiments revealed that detection performance alone is insufficient for evaluating AI-assisted cyber defense systems.

A system may successfully classify malicious behavior while simultaneously producing evidence that is:

1. Incomplete
2. Contradictory
3. Difficult to interpret

Therefore, forensic reliability must be evaluated independently from detection performance.

Finding 2 -Different Monitoring Layers Observe Different Realities

Network, authentication, and application monitoring systems do not necessarily observe identical aspects of an attack.

Each layer possesses unique visibility characteristics.

Examples:

Table 59 Monitoring Layer Visibility Characteristics

Layer	Primary Visibility

Network AI	Traffic behavior
Authentication AI	User behavior
Application AI	Semantic intent

As a result, individual layers may produce different interpretations of the same incident.

Finding 3 -Prompt Injection Produced the Greatest Forensic Challenge

Among all experimental scenarios, prompt injection demonstrated the highest level of forensic inconsistency.

The attack:

1. Generated minimal network anomalies
2. Generated normal authentication activity
3. Produced malicious application behavior

Consequently, the attack remained largely invisible outside the application layer.

This finding highlights the growing importance of application-level monitoring in AI-enabled environments.

Finding 4 -Cross-Layer Correlation Improves Evidence Reliability

Scenarios involving multiple evidence sources generally produced stronger investigative outcomes.

Examples:

1. S4 Success After Failures
2. S8 Login Plus Prompt

The availability of supporting evidence from multiple monitoring layers improved:

1. Contextual understanding
2. Event correlation
3. Analytical reliability

This finding demonstrates the value of integrated cyber defense architectures.

Finding 5 -Adversarial Impact Extends Beyond Detection

Perhaps the most important finding is that adversarial behavior affects more than attack detection.

It also influences:

1. Evidence visibility
2. Evidence consistency
3. Evidence interpretation
4. Investigative confidence

This observation represents the core contribution of this research.

5.10.6 Contributions of the Thesis

The research contributes to the field in several ways.

Contribution 1

Development of a multi-layer AI-assisted cyber defense architecture integrating:

1. Network monitoring
2. Authentication monitoring
3. Application monitoring
4. SIEM correlation

Contribution 2

Design and implementation of controlled adversarial scenarios for forensic evaluation.

Contribution 3

Introduction of a forensic contradiction model for analysing disagreements between monitoring layers.

Contribution 4

Development of a forensic integrity evaluation framework consisting of:

1. Completeness
2. Coherence
3. Consistency
4. Reliability metrics

Contribution 5

Empirical demonstration that adversarial attacks can alter forensic visibility even when attack detection remains operational.

5.10.7 Limitations of the Current Experimental Environment

Although the experimental architecture successfully demonstrated the proposed concepts, several limitations remain.

Dataset Scale

The experiments were conducted within a controlled laboratory environment rather than a production-scale enterprise infrastructure.

Limited Attack Diversity

Only selected adversarial scenarios were evaluated.

Future work could incorporate:

1. Adversarial machine learning attacks
2. Model poisoning
3. LLM-specific attacks
4. Advanced persistence techniques

Single-Organization Scope

The experimental environment represents a single architectural deployment.

Different organizations may exhibit different evidence-generation characteristics.

Limited Automated Correlation

While Wazuh provided centralized visibility, advanced adaptive correlation mechanisms were not implemented during this phase of the research.

These limitations provide opportunities for future enhancement.

5.10.8 Transition to Chapter 6

The findings of this chapter demonstrate that adversarial activities can significantly influence the reliability and interpretability of forensic evidence generated by AI-assisted cyber defense systems.

Several scenarios produced evidence contradictions, reduced consistency, and altered investigative visibility despite continued operation of detection mechanisms.

These observations naturally lead to the next stage of the research:

How can forensic integrity be preserved when adversarial behavior attempts to distort evidentiary visibility?

To address this question, Chapter 6 introduces a mitigation and enhancement framework designed to improve evidence reliability through:

1. Multi-layer evidence correlation
2. Confidence-aware alerting
3. Forensic contradiction handling
4. Integrity-preserving evidence management
5. Architecture-level resilience improvements

The objective is not only to identify forensic degradation but also to develop practical mechanisms capable of reducing its impact within AI-powered cyber defense systems.

6. Forensic Reality Distortion and Evidentiary Reliability in AI Assisted Cyber Defense Systems

6.1 Introduction

The preceding chapter presented the experimental evaluation of the proposed AI-assisted cyber defense framework and analyzed the effects of adversarial activities across the network, authentication, application, and SIEM correlation layers. The results demonstrated that adversarial behavior influences more than attack detection outcomes. In several scenarios, attacks altered the visibility, consistency, and interpretability of digital evidence while the monitoring infrastructure itself continued to operate as intended.

Traditional cybersecurity research typically evaluates defensive systems using operational metrics such as detection accuracy, precision, recall, false positive rates, and response times. While these measures provide important insights into the effectiveness of detection mechanisms, they do not fully capture the broader implications of adversarial behavior in modern AI-assisted security environments. In real-world investigations, security operations, compliance audits, and legal proceedings rely not only on successful attack detection but also on the quality, completeness, and reliability of the forensic evidence generated throughout the monitoring process.

The experimental results presented in Chapter 5 revealed that evidence generation and evidence interpretation are fundamentally different processes. Raw telemetry, machine

learning outputs, and SIEM-generated alerts frequently provide different representations of the same security incident. Consequently, an adversarial activity may remain partially visible within one monitoring layer while simultaneously appearing benign or insignificant within another. Such situations introduce uncertainty into the investigative process and may significantly affect the ability of analysts to reconstruct events accurately.

A particularly significant observation emerging from the experimental evaluation was the existence of cross-layer evidentiary contradictions. Scenarios involving prompt injection attempts, behavioral authentication anomalies, and combined attack sequences demonstrated that individual monitoring layers often produced divergent interpretations of the same activity. These contradictions did not necessarily indicate a failure of the monitoring system. Instead, they highlighted the fact that each monitoring layer observes only a subset of the overall attack surface and therefore constructs its own representation of reality based on the information available to it.

The findings further suggest that adversarial attacks should not be evaluated solely in terms of whether they successfully evade detection. Even when attacks are detected, they may influence the evidentiary narrative generated by the monitoring infrastructure. In such situations, investigators may be presented with incomplete, inconsistent, or partially contradictory information that complicates incident reconstruction and forensic analysis. Therefore, the impact of adversarial behavior extends beyond operational security and directly affects the trustworthiness of digital forensic evidence.

This chapter examines the implications of these findings from a forensic perspective. Rather than focusing on attack detection performance alone, the discussion shifts towards the concept of evidentiary reliability and forensic integrity. The objective is to analyse how adversarial activities influence the representation of reality produced by AI-assisted cyber defense systems and how such influences affect the quality of subsequent digital forensic investigations.

To support this discussion, the chapter introduces the concept of **Forensic Reality Distortion**, a theoretical framework describing the divergence between actual attacker behavior and the evidentiary narrative available to investigators after telemetry collection, machine-learning interpretation, and SIEM correlation have taken place. The framework provides a structured explanation for the contradiction patterns observed during the experimental evaluation and serves as a foundation for understanding the broader forensic consequences of adversarial behavior in AI-assisted environments.

Furthermore, this chapter investigates the implications of evidentiary contradictions, evaluates their impact on digital forensic investigations, discusses the limitations of current AI-assisted cyber defense architectures, and formalizes the principal scientific contributions of this research. Through this analysis, the discussion moves beyond the traditional question of whether attacks can be detected and instead addresses the more fundamental question of

whether AI-assisted cyber defense systems can preserve an accurate, complete, and trustworthy representation of reality under adversarial conditions.

The discussion presented in this chapter forms the conceptual bridge between the experimental observations reported in Chapter 5 and the conclusions presented in Chapter 7. While Chapter 5 demonstrated how adversarial activities influence evidence generation within a controlled experimental environment, Chapter 6 explores why these effects occur, what they imply for digital forensic investigations, and how future AI-powered cyber defense systems can be designed to better preserve evidentiary integrity in the presence of adversarial manipulation.

6.2 From Attack Detection to Evidence Interpretation

Traditional cybersecurity research has historically evaluated defensive technologies according to their ability to detect, classify, and respond to malicious activity. Consequently, performance metrics such as accuracy, precision, recall, F1-score, false positive rate, and false negative rate have become the dominant measures for assessing the effectiveness of intrusion detection systems, malware classifiers, behavioral analytics platforms, and other AI-assisted security technologies. While these metrics provide valuable insight into the operational capabilities of a security solution, they do not fully represent the requirements of modern cyber defense environments where digital evidence serves as the foundation for incident investigation, regulatory reporting, threat hunting, and forensic analysis.

The emergence of artificial intelligence within cybersecurity has further reinforced this detection-centric perspective. Machine learning models are typically trained to maximize classification performance by identifying patterns associated with malicious behavior while minimizing incorrect predictions. As a result, research efforts frequently focus on improving model accuracy, reducing false alarms, increasing detection rates, or enhancing model robustness against adversarial manipulation. Although these objectives remain important, they represent only one aspect of a broader security ecosystem. Security operations centers, incident response teams, and digital forensic investigators must ultimately rely on the evidence generated by these systems rather than the underlying classification algorithms themselves.

From a forensic perspective, the primary objective is not merely to determine whether an attack occurred but to understand how the attack unfolded, what systems were affected, which actions were performed, and what evidence supports the resulting conclusions. This distinction is significant because successful attack detection does not automatically guarantee the availability of complete, consistent, or trustworthy evidence. Similarly, partial detection failures do not necessarily imply a complete loss of investigative capability if sufficient evidence remains available through alternative monitoring sources.

The experimental results presented in Chapter 5 illustrate this distinction clearly. Several scenarios demonstrated that attack visibility varied considerably across different monitoring layers despite the presence of a common underlying event sequence. For example, network-based monitoring components observed communication patterns and traffic anomalies, authentication monitoring components focused on user behavior and access activities, while application-layer monitoring evaluated semantic interactions occurring within the web application. Each monitoring layer therefore generated evidence from a different observational perspective. Consequently, the absence of evidence within one layer did not necessarily imply the absence of evidence within the system as a whole.

This observation challenges the conventional assumption that detection performance alone provides a sufficient basis for evaluating cyber defense systems. In practice, digital investigations depend on the collective evidentiary record generated across multiple monitoring layers. Investigators reconstruct incidents by correlating network activity, authentication events, application interactions, system logs, and SIEM-generated alerts. The quality of this reconstruction depends not only on the accuracy of individual detection mechanisms but also on the completeness and consistency of the evidence available throughout the investigative process.

The experiments conducted within this research revealed several situations in which machine-learning predictions and evidentiary availability diverged. Certain activities produced alerts within one monitoring layer while remaining undetected within another. In other cases, suspicious behavior generated evidence that appeared benign when analyzed in isolation but became significant when correlated with information obtained from additional sources. These observations suggest that evidence interpretation is inherently a multi-layer process that extends beyond individual model predictions.

The distinction between detection and interpretation becomes particularly important when considering adversarial behavior. Traditional adversarial machine learning research frequently evaluates attacks according to their ability to manipulate model outputs, reduce classification accuracy, or evade detection mechanisms. However, from a forensic perspective, adversarial success should also be evaluated according to its influence on the evidentiary narrative available to investigators. An adversarial action may not completely evade detection yet still alter the visibility, timing, correlation, or contextual interpretation of the resulting evidence. Such changes can significantly affect investigative outcomes even when overall detection performance remains relatively stable.

Another important consideration is the role of contextual information in forensic investigations. Security analysts rarely rely on individual alerts when evaluating incidents. Instead, they examine relationships between events, identify behavioral patterns, establish timelines, and correlate evidence across multiple systems. Consequently, the evidentiary value of a security event depends not only on the event itself but also on its relationship to surrounding activities. AI-assisted monitoring systems therefore function not merely as detection mechanisms but as

evidence generation and evidence interpretation platforms that actively influence how investigators perceive and understand cyber incidents.

The findings presented in this thesis indicate that AI-generated classifications should be viewed as one component of a larger evidentiary ecosystem rather than as definitive representations of truth. Machine-learning outputs provide valuable analytical insights; however, they remain probabilistic assessments derived from observable data rather than direct observations of attacker intent. Consequently, forensic investigators must consider both the predictions generated by AI systems and the underlying evidence supporting those predictions. Failure to do so may result in incomplete investigations, incorrect conclusions, or an overreliance on algorithmic outputs that lack sufficient contextual validation.

These observations motivate a transition from detection-centric evaluation towards evidence-centric evaluation. Within such a framework, the effectiveness of a cyber defense system is assessed not only according to its ability to identify malicious behavior but also according to its ability to preserve an accurate and coherent representation of events. This perspective aligns more closely with the requirements of digital forensics, where evidentiary integrity, temporal consistency, source correlation, and investigative reliability are often more important than classification performance alone.

Accordingly, this research argues that future evaluations of AI-assisted cyber defense systems should incorporate forensic dimensions alongside traditional detection metrics. Such dimensions include evidence completeness, temporal coherence, correlation consistency, and analytical reliability. Collectively, these factors provide a more comprehensive understanding of how adversarial activities influence both security operations and digital forensic investigations. By expanding the evaluation framework beyond conventional performance metrics, researchers and practitioners can better assess the true impact of adversarial behavior on modern cyber defense environments.

The transition from attack detection to evidence interpretation therefore represents a fundamental shift in perspective. Rather than asking whether a cyber defense system can detect an attack, the more important question becomes whether the system can preserve a trustworthy representation of reality after the attack has occurred. This distinction forms the foundation for the concept of **Forensic Reality Distortion**, which is introduced and formalized in the following section.

6.3 Forensic Reality Distortion

The experimental findings presented in Chapter 5 revealed a recurring phenomenon that extends beyond conventional adversarial machine learning concerns. While previous research has primarily focused on how adversarial behavior affects classification accuracy and attack detection rates, the results obtained in this study indicate that adversarial actions can also influence the evidentiary representation of reality generated by AI-assisted cyber

defense systems. This observation led to the development of the concept referred to in this thesis as **Forensic Reality Distortion (FRD)**.

Forensic Reality Distortion describes the divergence between actual attacker behavior and the reconstructed evidentiary narrative available to investigators after information has passed through multiple monitoring, analysis, and correlation layers. In traditional digital forensic investigations, evidence is often assumed to represent an objective record of system activity. However, modern cyber defense environments increasingly rely on AI-assisted analytics, behavioral classification models, automated alert generation mechanisms, and SIEM correlation engines to process and interpret large volumes of telemetry data. Consequently, investigators rarely interact directly with raw events. Instead, they observe an interpreted representation of reality produced by a combination of algorithms, rules, and monitoring systems.

Under adversarial conditions, this interpreted representation may differ significantly from the actual sequence of events that occurred within the environment. An attacker may not need to completely evade detection to influence investigative outcomes. Instead, it may be sufficient to alter how events appear across different monitoring layers. Such alterations can affect event visibility, confidence scores, alert severity levels, correlation relationships, or contextual interpretations, thereby changing the narrative available to investigators despite the continued operation of the monitoring infrastructure.

The concept of Forensic Reality Distortion is grounded in the observation that modern cyber defense systems operate as evidence transformation pipelines. Raw telemetry generated by network devices, applications, authentication services, and operating systems undergoes multiple stages of processing before being presented to security analysts. At each stage, information may be filtered, aggregated, classified, enriched, correlated, or prioritized according to predefined rules and machine-learning models. While these processes improve operational efficiency and reduce analyst workload, they simultaneously introduce opportunities for distortion when adversarial behavior exploits weaknesses within the monitoring ecosystem.

The experiments conducted during this research demonstrated several examples of this phenomenon. In the baseline network scanning scenario, malicious reconnaissance activity produced observable network evidence, generated AI alerts, and remained visible within the broader monitoring architecture. In contrast, adversarial scanning techniques designed to reduce behavioral visibility altered the evidentiary representation generated by the system. Although network activity still occurred, its visibility within higher analytical layers was reduced. The attack therefore influenced not only detection outcomes but also the quality and completeness of the evidence available for subsequent forensic analysis.

Similarly, application-layer prompt injection scenarios demonstrated that semantic manipulation could create discrepancies between observed user behavior and generated

security alerts. From a network perspective, the interaction may appear entirely legitimate because the communication channel itself remains normal. However, application-level monitoring mechanisms may identify malicious intent embedded within user inputs. Consequently, different monitoring layers generate different representations of the same activity, creating ambiguity regarding the true nature of the event.

Authentication-focused experiments revealed an additional dimension of forensic distortion. Behavioral anomalies associated with repeated failed login attempts, successful logins following suspicious activity, or abnormal access patterns generated evidence that appeared significant when analyzed within the authentication domain. However, these same activities often produced limited visibility within network monitoring systems. As a result, investigators relying primarily on network-centric evidence might reach conclusions that differ substantially from those based on authentication-centric evidence.

These observations indicate that forensic distortion does not necessarily arise from the absence of evidence. Instead, distortion frequently emerges from the unequal distribution of evidence across monitoring layers. Certain aspects of an incident may become highly visible within one analytical domain while remaining partially hidden within another. This asymmetry creates multiple competing representations of reality, each supported by valid evidence but none necessarily reflecting the complete incident narrative.

To formalize this concept, Forensic Reality Distortion can be expressed as the difference between the actual incident state and the observed incident state generated by the monitoring infrastructure:

Forensic Reality Distortion = Actual Incident Narrative − Observed Incident Narrative

Within this context, the actual incident narrative represents the ground-truth sequence of actions executed by an attacker, while the observed incident narrative represents the sequence reconstructed from available telemetry, AI-generated outputs, and SIEM correlations. The greater the divergence between these two representations, the greater the degree of forensic distortion present within the investigation.

Importantly, Forensic Reality Distortion should not be interpreted as a binary condition. Distortion exists along a continuum ranging from minimal divergence to severe evidentiary degradation. In some cases, investigators may possess sufficient information to reconstruct events accurately despite minor inconsistencies. In other situations, critical evidence gaps or contradictory interpretations may significantly impair investigative reliability. Therefore, forensic distortion is best viewed as a measurable property of an evidence ecosystem rather than a simple indication of success or failure.

The findings of this research further suggest that AI-assisted cyber defense systems introduce unique distortion pathways not commonly encountered within traditional security architectures. Machine-learning models produce probabilistic outputs rather than

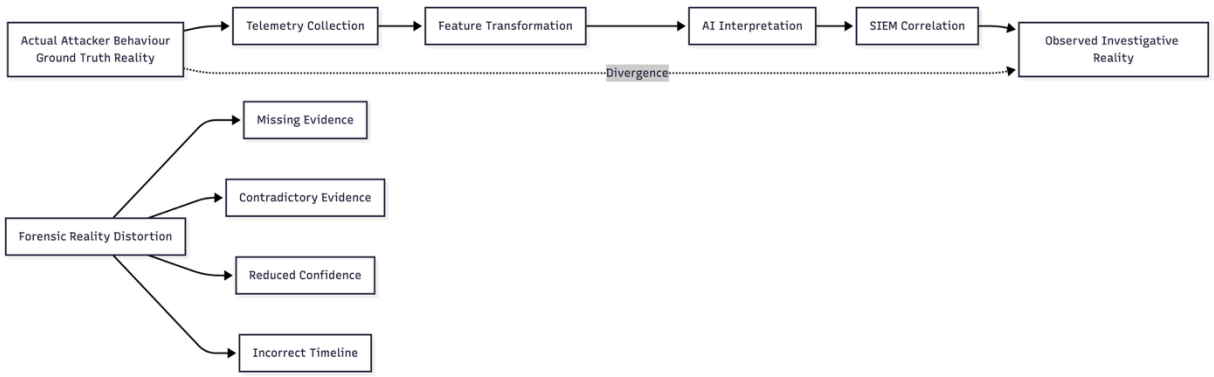
deterministic conclusions. Confidence scores, classification thresholds, feature selection mechanisms, and training data characteristics all influence how events are interpreted. Consequently, adversarial activities can affect not only what evidence is collected but also how collected evidence is interpreted and prioritized. This creates additional opportunities for divergence between reality and perception.

From a digital forensics perspective, the existence of Forensic Reality Distortion has significant implications. Investigators may unknowingly reconstruct incidents based on incomplete or contradictory evidence, leading to inaccurate timelines, incorrect attribution decisions, or flawed assessments of attack severity. Such outcomes are particularly concerning within environments where AI-generated outputs increasingly influence operational decision-making processes. As AI adoption continues to expand within cyber defense systems, understanding and measuring forensic distortion becomes essential for maintaining evidentiary reliability.

The concept introduced in this section represents one of the principal theoretical contributions of this thesis. Rather than evaluating adversarial attacks solely according to their influence on detection performance, the framework expands the analysis towards evidentiary consequences. This perspective aligns more closely with the practical realities of incident response and digital forensic investigations, where preserving an accurate representation of events is often more important than achieving marginal improvements in classification accuracy.

Ultimately, the results obtained throughout this research demonstrate that adversarial attacks should be understood not only as threats to detection systems but also as threats to evidentiary integrity. The ability of an attacker to influence how reality is represented within a monitoring ecosystem constitutes a significant security concern that extends beyond traditional machine-learning evaluation frameworks. Recognizing and measuring this phenomenon is therefore essential for the future development of trustworthy AI-assisted cyber defense architectures.

Figure 36 Forensic Reality Distortion Model



6.4 Cross-Layer Evidence Contradictions

One of the most significant findings emerging from the experimental evaluation was the presence of contradictory evidence across different monitoring layers. While traditional security architectures often assume that multiple monitoring sources collectively improve visibility and strengthen investigative confidence, the results obtained in this research demonstrate that individual monitoring layers may generate substantially different interpretations of the same underlying activity. These differences create evidentiary contradictions that directly influence incident reconstruction, threat attribution, and forensic decision-making.

Within the proposed architecture, evidence was generated from three primary analytical domains: network monitoring, authentication monitoring, and application-layer monitoring. Each domain observed a different aspect of system behavior and therefore produced evidence based on distinct data sources, feature sets, and analytical objectives. Network monitoring focused on communication behavior and traffic characteristics, authentication monitoring evaluated user access patterns and behavioral anomalies, while application-layer monitoring analyzed semantic interactions occurring within the protected application environment. Although these domains observed activities associated with the same users and systems, they frequently produced different interpretations of observed behavior.

The experimental scenarios demonstrated that contradictions emerged not because monitoring systems malfunctioned, but because each monitoring layer possessed only a partial view of the overall attack lifecycle. Consequently, an event considered suspicious within one analytical context could simultaneously appear benign within another. This observation highlights a fundamental challenge in AI-assisted cyber defense environments: no individual monitoring source possesses complete situational awareness. Instead, reality is distributed across multiple telemetry domains, each contributing a fragment of the overall incident narrative.

The prompt injection scenarios provided particularly strong evidence of this phenomenon. Application-layer monitoring identified semantic characteristics associated with malicious intent and generated alerts reflecting elevated risk levels. However, from the perspective of network monitoring, the same interaction appeared indistinguishable from normal user activity because no abnormal communication behavior was observed. Authentication monitoring similarly recorded legitimate user access without detecting behavioral anomalies. As a result, the same incident produced three distinct interpretations simultaneously: suspicious at the application layer, benign at the network layer, and normal at the authentication layer.

Such contradictions are highly significant from a forensic perspective. Investigators analysing only network evidence might conclude that no meaningful security event occurred. Conversely, investigators relying primarily on application-layer evidence might classify the same activity as a deliberate attack attempt. The resulting discrepancy demonstrates how

investigative conclusions can vary substantially depending on which evidence sources receive the greatest analytical emphasis.

Authentication-focused scenarios revealed a complementary contradiction pattern. Repeated failed login attempts followed by successful authentication generated behavioral indicators suggesting elevated risk. Authentication monitoring correctly identified unusual access patterns and produced evidence reflecting potentially malicious behavior. However, these same activities generated relatively limited visibility within network monitoring components because the underlying network communications remained technically valid. Consequently, authentication evidence suggested suspicious activity while network evidence provided little indication of compromise.

The combined attack scenarios offered the most comprehensive demonstration of cross-layer contradictions. These experiments incorporated multiple adversarial actions occurring within the same incident sequence, including authentication anomalies, application-layer manipulation attempts, and network reconnaissance activities. Under such conditions, each monitoring layer detected different subsets of attacker behavior. While individual evidence sources remained internally consistent, their collective interpretation produced a fragmented representation of reality. Certain attack stages appeared prominently within one monitoring domain while remaining partially or entirely absent within others.

These findings suggest that evidentiary contradictions should not be interpreted solely as monitoring deficiencies. Instead, contradictions represent natural consequences of observing complex cyber incidents through multiple specialized analytical perspectives. The challenge therefore lies not in eliminating contradictions entirely but in recognizing, measuring, and managing them appropriately during forensic investigations.

Based on the experimental observations, four primary contradiction categories were identified within the proposed framework.

Type I Contradictions occur when one monitoring layer identifies suspicious behavior while other monitoring layers classify the same activity as benign. The prompt injection scenarios provide representative examples of this pattern, where application-layer monitoring detected malicious intent despite limited evidence within network and authentication domains.

Type II Contradictions emerge when behavioral anomalies are visible within authentication monitoring systems but remain largely absent from network and application-layer evidence sources. The failed-login and success-after-failure scenarios demonstrated this contradiction type, illustrating how authentication evidence can reveal attack indicators not immediately observable elsewhere.

Type III Contradictions arise when evidence exists across multiple monitoring domains, but SIEM correlation mechanisms fail to combine these indicators into a coherent security

narrative. In such situations, relevant evidence remains available but may not receive appropriate prioritization or escalation.

Type IV Contradictions occur when all monitoring layers generate valid observations, yet the resulting evidence remains incomplete relative to the ground-truth attack sequence. This contradiction type represents a direct manifestation of forensic reality distortion because the reconstructed narrative differs from the actual incident despite the presence of observable evidence.

The existence of these contradiction categories has important implications for digital investigations. Traditional forensic methodologies often assume that evidence sources reinforce one another and collectively improve investigative certainty. However, the findings presented in this thesis indicate that AI-assisted cyber defense environments may generate competing interpretations rather than convergent conclusions. Consequently, investigators must evaluate evidence not only according to its individual validity but also according to its relationship with evidence originating from other monitoring domains.

Another important observation concerns the role of machine-learning models in contradiction formation. Because AI systems generate probabilistic outputs rather than deterministic decisions, confidence scores, classification thresholds, and feature selection mechanisms influence how evidence is interpreted. Two monitoring systems observing different aspects of the same activity may therefore produce conflicting conclusions without either system being objectively incorrect. This characteristic further complicates forensic analysis because contradictions may emerge from legitimate analytical differences rather than technical failures.

The findings of this research suggest that contradiction-aware investigation methodologies will become increasingly important as AI adoption continues to expand within cybersecurity environments. Rather than treating conflicting evidence as an anomaly, future forensic frameworks should recognize contradictions as expected outcomes within complex multi-layer monitoring ecosystems. Such an approach enables investigators to focus on understanding why contradictions occur and what they reveal about attacker behavior rather than simply attempting to eliminate them.

Ultimately, the experimental evaluation demonstrated that cross-layer contradictions represent a fundamental characteristic of AI-assisted cyber defense systems. These contradictions provide valuable insight into how adversarial behavior influences evidence generation and interpretation across multiple monitoring domains. Understanding and analysing such contradictions therefore constitutes an essential component of preserving evidentiary integrity and improving the reliability of digital forensic investigations in modern cyber defense environments.

6.5 Implications for Digital Forensic Investigations

The findings presented throughout this research extend beyond the domains of intrusion detection and adversarial machine learning. While the primary objective of many AI-assisted cyber defense systems is to identify malicious behavior and support operational security activities, the experimental results demonstrate that adversarial actions can also influence the evidentiary foundations upon which digital forensic investigations depend. Consequently, the implications of adversarial manipulation must be evaluated not only in terms of detection performance but also in terms of forensic reliability, evidentiary trustworthiness, and investigative accuracy.

Digital forensic investigations are fundamentally concerned with reconstructing historical events from available evidence. Investigators seek to establish what occurred, when it occurred, how it occurred, and who was responsible for the observed actions. The validity of this reconstruction process depends heavily on the quality, completeness, and consistency of the evidence available for analysis. Traditional forensic methodologies generally assume that evidence sources provide a sufficiently accurate representation of system activity, allowing investigators to develop a coherent understanding of the incident under investigation.

However, the results obtained in this thesis indicate that modern AI-assisted cyber defense systems introduce additional layers of interpretation between observed activity and forensic analysis. Raw telemetry is increasingly filtered through machine-learning models, behavioral analytics engines, automated enrichment pipelines, and SIEM correlation mechanisms before being presented to analysts. While these technologies improve scalability and operational efficiency, they also influence how events are represented, prioritized, and interpreted. Consequently, investigators often analyse an interpreted version of reality rather than direct observations of system activity.

This transformation of evidence introduces a critical challenge for digital forensics. If adversarial behavior influences how events are interpreted within AI-assisted monitoring systems, the resulting forensic narrative may differ from the actual sequence of events that occurred within the environment. Such discrepancies do not necessarily indicate data loss or system compromise. Instead, they reflect changes in evidentiary visibility, contextual interpretation, and analytical prioritization that can affect investigative conclusions even when the underlying evidence remains partially available.

The experimental scenarios demonstrated several practical manifestations of this challenge. In authentication-focused scenarios, behavioral anomalies were clearly visible within authentication evidence yet remained less prominent within network observations. Conversely, application-layer prompt injection activities generated strong semantic indicators while producing relatively limited evidence within other monitoring domains. These observations suggest that investigators relying exclusively on a single evidence source may reach incomplete or inaccurate conclusions regarding the nature and scope of an incident.

A particularly important implication concerns incident timeline reconstruction. Accurate timelines represent one of the most critical components of digital forensic investigations because they establish the sequence of actions performed by attackers, users, and systems. The experimental findings revealed that evidence visibility varied across monitoring layers, meaning that certain events appeared prominently within one source while remaining partially absent from others. Such inconsistencies can complicate timeline construction and increase uncertainty regarding causal relationships between observed activities.

Another significant implication relates to evidentiary confidence. In traditional forensic practice, confidence in investigative conclusions increases when multiple independent evidence sources support the same interpretation. However, the cross-layer contradictions identified during this research demonstrate that AI-assisted monitoring systems may produce divergent interpretations despite observing the same underlying incident. Consequently, investigators must evaluate not only whether evidence exists but also whether different sources support compatible narratives. The presence of contradictory evidence therefore becomes an important investigative finding in its own right rather than merely an indication of analytical error.

The findings further highlight the growing importance of contextual analysis within digital forensics. Individual alerts, classifications, or behavioral indicators rarely provide sufficient information to support definitive conclusions. Instead, investigators must examine relationships between events, identify patterns across multiple telemetry domains, and evaluate evidence within its broader operational context. The results obtained in this thesis suggest that contextual interpretation becomes even more important in AI-assisted environments because machine-learning outputs frequently represent probabilistic assessments rather than definitive observations.

From a legal and regulatory perspective, the implications are equally significant. Many organizations rely on digital evidence to support compliance reporting, breach notification processes, internal investigations, and legal proceedings. If adversarial behavior can influence the evidentiary representation generated by AI-assisted monitoring systems, organizations may face challenges when demonstrating the completeness and reliability of their investigative processes. Consequently, maintaining evidentiary integrity becomes not only a technical requirement but also a governance and compliance concern.

The concept of Forensic Reality Distortion introduced in this thesis provides a useful framework for understanding these challenges. Rather than assuming that monitoring outputs represent objective truth, investigators should recognize that modern cyber defense systems construct evidentiary narratives through a combination of telemetry collection, machine-learning analysis, and correlation processes. Under adversarial conditions, these narratives may diverge from ground-truth events in predictable and measurable ways. Recognizing this possibility allows investigators to adopt more cautious and evidence-centric analytical approaches.

The findings also suggest that future forensic methodologies should incorporate mechanisms for evaluating evidentiary quality in addition to evidentiary quantity. Traditional investigations often focus on collecting as much evidence as possible. However, the results of this research indicate that completeness alone is insufficient. Investigators must also consider temporal coherence, source consistency, analytical reliability, and contradiction patterns when assessing the trustworthiness of reconstructed incident narratives.

Furthermore, the increasing integration of artificial intelligence within security operations centers creates new requirements for forensic readiness. Organizations deploying AI-assisted cyber defense technologies should ensure that raw telemetry remains accessible for independent verification and retrospective analysis. Overreliance on AI-generated outputs without preserving supporting evidence may reduce investigative transparency and increase the risk of erroneous conclusions. Maintaining access to both interpreted evidence and underlying raw data therefore represents an important principle for future forensic architectures.

The broader implication of this research is that adversarial attacks should no longer be viewed exclusively as threats to detection systems. Instead, they should also be recognized as threats to evidentiary integrity and investigative reliability. An attacker capable of influencing how reality is represented within a monitoring ecosystem may affect forensic outcomes even when traditional security controls remain operational. This observation expands the scope of adversarial machine learning research and establishes a direct connection between AI security and digital forensic science.

Ultimately, the findings presented in this thesis demonstrate that trustworthy cyber defense requires more than accurate detection mechanisms. It requires the preservation of reliable, coherent, and interpretable evidence capable of supporting investigative decision-making under adversarial conditions. As AI continues to play a central role in cybersecurity operations, understanding and mitigating the forensic consequences of adversarial behavior will become increasingly important for both researchers and practitioners.

6.6 Scientific Contributions of the Research

This research was motivated by the increasing integration of artificial intelligence into modern cyber defense systems and the growing concern that adversarial techniques may compromise not only attack detection capabilities but also the integrity of digital forensic evidence. While previous studies have primarily focused on detection accuracy, adversarial robustness, and machine-learning performance, comparatively limited attention has been given to the forensic implications of AI-assisted security architectures. The findings presented throughout this thesis address this gap by examining how adversarial behavior influences evidence generation, evidence interpretation, and forensic reliability across multiple monitoring layers.

The principal contribution of this research is the development and experimental validation of a multi-layer framework for analysing the relationship between adversarial attacks, AI-assisted cyber defense systems, and digital forensic investigations. Rather than evaluating adversarial behavior exclusively through traditional machine-learning performance metrics, the proposed approach extends the analysis towards evidentiary outcomes and investigative consequences. This shift in perspective represents an important advancement because real-world security operations depend not only on attack detection but also on the ability to preserve trustworthy evidence for incident response and forensic analysis.

The first contribution of this thesis is the design and implementation of a multi-layer AI-assisted cyber defense architecture that integrates network monitoring, authentication analytics, application-layer behavioral analysis, and SIEM-based event correlation. Unlike many experimental adversarial machine-learning studies that focus on isolated detection models, the proposed framework reflects the structure of modern operational security environments where evidence is generated across multiple telemetry domains. The architecture enabled the investigation of how adversarial activities propagate through different analytical layers and influence the resulting evidentiary narrative.

The second contribution is the development of a structured experimental methodology for evaluating adversarial behavior from both security and forensic perspectives. The research introduced a scenario-driven evaluation framework incorporating baseline attacks, authentication anomalies, semantic manipulation attempts, and combined attack sequences. By constructing ground-truth datasets and controlled experimental conditions, the study enabled systematic comparison between actual attacker behavior and observed evidence generated by the monitoring infrastructure. This methodology provides a repeatable approach for future investigations examining the forensic consequences of adversarial activities within AI-assisted environments.

A third contribution is the introduction of forensic-oriented evaluation criteria that complement traditional machine-learning performance metrics. Existing adversarial machine-learning research commonly focuses on measures such as accuracy, precision, recall, and robustness. While these metrics remain important, they do not fully capture the evidentiary consequences of adversarial behavior. This thesis therefore extends the evaluation framework by incorporating concepts such as evidence completeness, temporal coherence, correlation consistency, and analytical reliability. Collectively, these dimensions provide a more comprehensive assessment of how AI-assisted security systems support digital investigations under adversarial conditions.

The fourth contribution is the identification and formalization of cross-layer evidentiary contradiction patterns. The experimental results demonstrated that network monitoring systems, authentication analytics, application-layer models, and SIEM correlation engines may generate different interpretations of the same incident. These contradictions were categorized into multiple types, reflecting the different ways in which evidence can become fragmented

across monitoring domains. The resulting contradiction framework provides investigators with a structured mechanism for understanding and analysing evidentiary inconsistencies in AI-assisted security environments.

Perhaps the most significant contribution of this research is the introduction of the concept of **Forensic Reality Distortion (FRD)**. The concept describes the divergence between actual attacker behavior and the reconstructed evidentiary narrative available to investigators after telemetry collection, machine-learning interpretation, and SIEM correlation processes have been completed. Unlike traditional adversarial machine-learning research, which evaluates attacks according to their influence on model outputs, the FRD framework focuses on how adversarial activities affect the representation of reality itself. This perspective establishes a direct connection between adversarial AI security and digital forensic science and provides a theoretical foundation for future research in this area.

Another important contribution lies in demonstrating that adversarial success should not be evaluated solely according to whether an attack evades detection. The findings indicate that adversarial activities may significantly influence investigative outcomes even when monitoring systems continue to detect suspicious behavior. Changes in evidence visibility, contextual interpretation, confidence levels, and cross-layer correlations can alter the narrative available to investigators without necessarily causing complete detection failure. This observation expands the scope of adversarial evaluation beyond conventional classification metrics and introduces a more realistic perspective aligned with operational security environments.

The research further contributes to the understanding of AI-assisted digital forensics by highlighting the importance of evidence-centric security evaluation. As organizations increasingly rely on artificial intelligence to support security operations, the quality and reliability of generated evidence become critical factors influencing investigative effectiveness. The findings suggest that future cyber defense systems should be evaluated not only according to their ability to identify attacks but also according to their ability to preserve coherent, interpretable, and trustworthy evidence under adversarial conditions.

From a practical perspective, the proposed framework demonstrates how organizations can integrate AI-assisted monitoring technologies while maintaining awareness of potential evidentiary limitations. The study provides guidance for security operations centers, incident response teams, and digital forensic practitioners seeking to understand the implications of machine-learning-driven security architectures. By recognizing the possibility of forensic reality distortion and cross-layer contradictions, practitioners can adopt more informed approaches to evidence validation and incident reconstruction.

The research also contributes to the broader field of trustworthy artificial intelligence. Current discussions surrounding trustworthy AI frequently emphasize fairness, transparency, accountability, explainability, and robustness. This thesis introduces forensic integrity as an additional dimension of trustworthiness that is particularly relevant for cybersecurity

applications. An AI-assisted defense system cannot be considered fully trustworthy if it generates evidence that is incomplete, contradictory, or misleading under adversarial conditions. Consequently, forensic reliability should be recognized as an important objective in the design and evaluation of future security-focused AI systems.

Taken together, the contributions presented in this thesis establish a foundation for a new research direction situated at the intersection of adversarial machine learning, cyber defense, and digital forensics. Rather than viewing these domains as separate research areas, the findings demonstrate that they are deeply interconnected through the evidentiary processes that support modern security operations. The proposed concepts, methodologies, and analytical frameworks therefore provide both theoretical and practical value for researchers and practitioners working within AI-assisted cybersecurity environments.

In summary, the scientific contributions of this research can be categorized into five principal areas:

1. **Development of a multi-layer AI-assisted cyber defense architecture** integrating network, authentication, application, and SIEM monitoring domains.
2. Creation of a structured adversarial evaluation methodology based on ground-truth-driven experimental scenarios.
3. **Introduction of forensic-oriented evaluation dimensions** including evidence completeness, temporal coherence, correlation consistency, and analytical reliability.
4. Formalization of cross-layer evidentiary contradiction patterns observed within AI-assisted monitoring environments.
5. **Introduction of the Forensic Reality Distortion framework**, providing a theoretical explanation for how adversarial activities influence evidentiary representation and investigative outcomes.

6.7 Limitations and Threats to Validity

No empirical research can fully represent the complexity of real-world cyber environments, and the findings presented in this thesis should therefore be interpreted within the context of the experimental scope and assumptions under which the study was conducted. While the proposed framework successfully demonstrated how adversarial activities influence AI-assisted cyber defense systems and digital forensic evidence, several limitations must be acknowledged to ensure an accurate interpretation of the results and to identify opportunities for future research.

The first limitation concerns the scale of the experimental environment. The research was conducted within a controlled laboratory infrastructure designed to emulate key components of a modern enterprise security architecture. The environment incorporated network monitoring, authentication monitoring, application-layer analytics, and SIEM-based event correlation, thereby enabling the evaluation of adversarial behavior across multiple telemetry

domains. Nevertheless, the experimental setup remains smaller and less heterogeneous than large-scale production environments where thousands of endpoints, users, applications, and network devices generate continuous streams of security-relevant data. Consequently, certain operational challenges associated with large-scale deployments may not be fully represented within the experimental results.

A second limitation relates to the selection of adversarial scenarios. The experiments focused on representative attack categories including network reconnaissance, authentication anomalies, semantic manipulation attempts, and combined attack sequences. These scenarios were chosen because they reflect realistic threats that frequently occur within modern cyber defense environments and because they enable meaningful analysis of evidentiary effects across multiple monitoring layers. However, the threat landscape continues to evolve rapidly, and many additional adversarial techniques exist that were beyond the scope of this research. Examples include advanced malware campaigns, multi-stage ransomware operations, insider threats, supply-chain compromises, AI-generated phishing attacks, and sophisticated command-and-control evasion strategies. Future studies may therefore expand the experimental framework to incorporate a broader range of adversarial behaviors.

A third limitation concerns the machine-learning models employed within the proposed architecture. The network monitoring component utilized a Random Forest classifier, while the application and authentication monitoring components employed Logistic Regression-based models. These algorithms were intentionally selected because of their interpretability, stability, computational efficiency, and suitability for controlled experimental evaluation. However, contemporary cyber defense systems increasingly incorporate deep learning architectures, transformer-based language models, graph neural networks, and large-scale behavioral analytics platforms. Although the forensic principles identified in this research are expected to remain applicable across different model families, the specific evidentiary effects observed may vary depending on model architecture, feature engineering approaches, training methodologies, and deployment contexts.

Another limitation relates to the availability of ground-truth information. Within the experimental environment, complete knowledge of attacker actions was available because all scenarios were executed under controlled conditions. This enabled direct comparison between actual events and reconstructed evidence, thereby supporting the development of the Forensic Reality Distortion framework. In operational environments, however, investigators rarely possess perfect ground truth. Instead, they must infer attacker behavior from available evidence while simultaneously determining which evidence can be trusted. As a result, applying the proposed framework in real-world investigations may require additional validation techniques capable of estimating forensic distortion in the absence of complete ground-truth information.

The study is also subject to limitations associated with telemetry visibility. Although the architecture incorporated multiple monitoring domains, it did not include every possible source of security-relevant evidence. Endpoint detection and response systems, memory forensics tools, cloud-native monitoring platforms, container security telemetry, industrial control system logs, and external threat intelligence feeds were not integrated into the experimental environment. The inclusion of these additional evidence sources may influence contradiction patterns and alter the degree of forensic distortion observed during investigations. Future research should therefore explore how expanding telemetry diversity affects evidentiary reliability within AI-assisted security architectures.

A further consideration involves the use of synthetic and semi-controlled datasets during model development and validation. While considerable effort was made to ensure realistic attack execution and representative behavioral patterns, laboratory-generated data cannot fully replicate the complexity, variability, and unpredictability of operational enterprise environments. User behavior, network traffic characteristics, organizational policies, and attacker objectives often vary significantly across different deployment contexts. Consequently, the quantitative results reported in this thesis should not be interpreted as universally applicable performance benchmarks but rather as evidence supporting the broader forensic concepts and analytical frameworks introduced throughout the research.

Another potential threat to validity arises from the correlation mechanisms employed within the SIEM layer. The study utilized predefined rule-based correlation logic to combine evidence generated by network, authentication, and application monitoring components. While this approach reflects common industry practices, alternative correlation strategies may produce different evidentiary outcomes. Machine-learning-driven correlation engines, graph-based investigation platforms, and autonomous security orchestration systems may generate different contradiction patterns and influence forensic reconstruction in ways not examined within the current research. Accordingly, future studies should investigate how alternative correlation methodologies affect evidentiary reliability under adversarial conditions.

The proposed concept of Forensic Reality Distortion also introduces certain methodological challenges. Although the framework provides a structured approach for understanding divergence between actual and observed incident narratives, measuring distortion quantitatively remains an open research problem. The current study primarily demonstrates the existence and significance of distortion through controlled experimentation and qualitative analysis. Future research should develop formal mathematical models, scoring methodologies, and validation techniques capable of quantifying distortion across diverse operational environments. Such developments would enable more rigorous comparison between different cyber defense architectures and support the creation of forensic reliability benchmarks.

Despite these limitations, the findings of this thesis remain significant because they reveal a previously underexplored relationship between adversarial machine learning, cyber defense

operations, and digital forensic investigations. The limitations identified above do not invalidate the conclusions of the research. Rather, they define the boundaries within which the results should be interpreted and highlight promising directions for future investigation. Importantly, none of the limitations undermine the central observation that adversarial activities can influence not only attack detection outcomes but also the quality, consistency, and trustworthiness of the evidence available to investigators.

From a scientific perspective, the limitations identified throughout this section reinforce the need for continued interdisciplinary research at the intersection of artificial intelligence, cybersecurity, and digital forensics. As organizations increasingly rely on AI-assisted monitoring systems to support security operations, understanding the evidentiary implications of adversarial behavior will become progressively more important. Future research must therefore focus not only on improving detection capabilities but also on ensuring that the resulting evidence remains reliable, interpretable, and defensible under adversarial conditions.

In conclusion, while the experimental scope, model selection, telemetry sources, and scenario design impose certain constraints on the generalizability of the results, the research successfully demonstrates the existence of forensic challenges that extend beyond traditional detection-centric evaluation frameworks. The identified limitations provide a foundation for future investigations and support the broader argument that evidentiary reliability should become a primary consideration in the design and evaluation of next-generation AI-assisted cyber defense systems.

6.8 Chapter Summary

This chapter examined the broader implications of adversarial behavior within AI-assisted cyber defense systems from a digital forensic perspective. While previous chapters focused on system design, implementation, and experimental evaluation, the discussion presented in this chapter shifted attention towards the evidentiary consequences of adversarial activities and their impact on investigative reliability. The analysis demonstrated that adversarial attacks should not be evaluated solely according to their influence on attack detection performance. Instead, they must also be assessed according to their ability to alter the quality, consistency, and interpretability of the digital evidence generated by modern security monitoring infrastructures.

The chapter began by challenging the traditional detection-centric perspective commonly adopted within cybersecurity research. Although metrics such as accuracy, precision, recall, and false positive rates remain important indicators of system performance, they do not fully capture the requirements of digital forensic investigations. Incident response teams, security analysts, and forensic investigators ultimately depend on the evidence produced by monitoring systems rather than the underlying machine-learning models themselves.

Consequently, evidentiary reliability emerged as a critical dimension for evaluating the effectiveness of AI-assisted cyber defense architectures.

Building upon this observation, the chapter introduced the concept of **Forensic Reality Distortion (FRD)**, which represents the divergence between actual attacker behavior and the reconstructed incident narrative available to investigators after telemetry collection, AI analysis, and SIEM correlation processes have been completed. The discussion demonstrated that modern cyber defense systems operate as evidence transformation pipelines in which raw observations are continuously interpreted, enriched, prioritized, and correlated before reaching human analysts. Under adversarial conditions, this process may alter how reality is represented within the monitoring ecosystem, creating discrepancies between ground-truth events and observed evidence.

The chapter further analyzed the phenomenon of cross-layer evidentiary contradictions identified during the experimental evaluation. Evidence generated by network monitoring systems, authentication analytics, application-layer models, and SIEM correlation engines frequently produced different interpretations of the same incident. These contradictions were shown to arise not from system failures but from the inherently partial visibility possessed by individual monitoring domains. As a result, investigators may encounter multiple competing representations of reality, each supported by valid evidence yet none individually reflecting the complete incident narrative.

The forensic implications of these contradictions were subsequently examined in detail. The discussion highlighted how evidentiary inconsistencies can affect timeline reconstruction, attribution confidence, contextual understanding, incident severity assessment, and overall investigative reliability. The findings demonstrated that preserving evidentiary integrity within AI-assisted environments requires more than effective attack detection. It requires mechanisms capable of maintaining coherent and trustworthy representations of events despite adversarial attempts to manipulate visibility, interpretation, and correlation processes.

The chapter also formalized the principal scientific contributions of the research. These contributions include the development of a multi-layer AI-assisted cyber defense architecture, the creation of a structured adversarial evaluation methodology, the introduction of forensic-oriented evaluation dimensions, the formalization of cross-layer contradiction patterns, and the proposal of the Forensic Reality Distortion framework. Collectively, these contributions establish a foundation for future research at the intersection of artificial intelligence, cyber defense, and digital forensics.

Finally, the limitations and threats to validity associated with the study were discussed. While the research was conducted within a controlled laboratory environment and employed a selected set of adversarial scenarios and machine-learning models, the findings nevertheless demonstrate the existence of forensic challenges that extend beyond traditional adversarial machine-learning concerns. The identified limitations therefore represent opportunities for

future investigation rather than weaknesses that undermine the overall conclusions of the research.

Taken together, the findings presented in this chapter reinforce the central argument of the thesis: adversarial attacks influence not only the operational effectiveness of AI-assisted cyber defense systems but also the reliability of the digital evidence upon which modern investigations depend. As artificial intelligence continues to assume a more prominent role within cybersecurity operations, ensuring the integrity and trustworthiness of evidentiary processes will become increasingly important for both researchers and practitioners.

The next chapter concludes the thesis by revisiting the research objectives, answering the research questions, evaluating the extent to which the proposed framework achieved its intended goals, and outlining future research directions for advancing forensic reliability within AI-assisted cyber defense environments.

7. CONCLUSIONS AND FUTURE RESEARCH DIRECTIONS

7.1 Research Overview

The increasing adoption of Artificial Intelligence (AI) within cybersecurity has transformed the way organizations detect, analyse, and respond to cyber threats. AI-assisted cyber defense systems are now routinely employed to process large volumes of telemetry, identify behavioral anomalies, correlate security events, and support security analysts in operational decision-making. While these technologies provide significant benefits in terms of scalability and detection capability, they also introduce new attack surfaces that can be exploited through adversarial techniques.

The primary objective of this thesis was to investigate the impact of adversarial attacks on AI-powered cyber defense systems and to evaluate how such attacks affect the reliability and integrity of digital forensic evidence. Unlike traditional adversarial machine learning research, which primarily focuses on classification performance and detection accuracy, this research adopted a broader perspective by examining the evidentiary consequences of adversarial behavior across multiple monitoring layers.

To achieve this objective, a multi-layer AI-assisted cyber defense architecture was designed and implemented within a controlled laboratory environment. The architecture integrated network-based anomaly detection, authentication behavior analysis, application-layer semantic monitoring, and SIEM-based event correlation. The resulting framework enabled the investigation of adversarial behavior across multiple telemetry domains while simultaneously supporting the collection and analysis of digital forensic evidence.

A series of controlled experimental scenarios were executed to evaluate both baseline and adversarial conditions. These scenarios included network reconnaissance activities,

authentication anomalies, prompt injection attempts, and combined attack sequences designed to produce evidence across multiple monitoring layers. The generated telemetry, AI-generated alerts, and SIEM outputs were subsequently analyzed to determine how adversarial behavior influenced evidentiary visibility, consistency, and investigative reliability.

The findings obtained throughout this research demonstrate that adversarial attacks affect not only attack detection processes but also the representation of reality generated by AI-assisted cyber defense systems. As a result, the study extends current understanding of adversarial machine learning by establishing a direct relationship between AI security and digital forensic science.

7.2 Answers to the Research Questions

This thesis was guided by a set of research questions concerning the interaction between adversarial attacks, AI-assisted cyber defense systems, and digital forensic investigations.

RQ1: How do adversarial attacks influence the effectiveness of AI-assisted cyber defense systems?

The experimental evaluation demonstrated that adversarial activities could influence the behavior of AI-assisted monitoring systems in multiple ways. Network reconnaissance activities, behavioral anomalies, and semantic manipulation attempts generated different levels of visibility across the monitoring infrastructure. While some attacks remained highly visible within specific monitoring domains, others produced reduced visibility or altered contextual interpretation. The findings indicate that adversarial behavior influences not only detection outcomes but also the broader analytical processes performed by AI-assisted security systems.

RQ2: How do adversarial attacks affect the quality and reliability of digital forensic evidence?

The results demonstrated that adversarial activities could influence evidence completeness, contextual interpretation, and correlation consistency. In several scenarios, evidence remained available but was distributed unevenly across monitoring layers, resulting in fragmented or partially contradictory representations of the same incident. Consequently, adversarial attacks may affect investigative reliability even when detection mechanisms continue to operate successfully.

RQ3: Can cross-layer analysis improve understanding of adversarial behavior?

The findings confirm that cross-layer analysis provides significant advantages when compared with isolated monitoring approaches. Network, authentication, and application monitoring systems each observed different aspects of attacker behavior. When analyzed collectively, these sources provided a more comprehensive representation of incident activity than any

individual monitoring domain alone. Cross-layer correlation therefore improves situational awareness and supports more reliable forensic reconstruction.

RQ4: Can adversarial impacts on forensic investigations be systematically analyzed?

The research demonstrated that adversarial impacts can be analyzed through structured comparison between ground-truth activities and observed evidence. The proposed framework, together with the concept of Forensic Reality Distortion, provides a methodology for examining how adversarial behavior influences evidence generation, interpretation, and investigative outcomes. This establishes a foundation for future forensic-focused evaluation frameworks within AI-assisted security environments.

7.3 Principal Findings

Several important findings emerged from the research.

First, AI-assisted cyber defense systems should not be evaluated exclusively according to traditional detection metrics. While classification accuracy remains important, the experiments demonstrated that detection performance alone does not adequately represent the quality of the resulting evidentiary record.

Second, evidence generated by different monitoring layers frequently provides different perspectives of the same incident. These differences are not necessarily indicative of monitoring failures but instead reflect the partial visibility inherent within specialized analytical domains.

Third, adversarial activities can influence investigative outcomes without completely bypassing security controls. Alterations in visibility, confidence, prioritization, and contextual interpretation may significantly affect forensic reconstruction despite the continued operation of monitoring systems.

Fourth, the experiments demonstrated the existence of cross-layer evidentiary contradictions. Network, authentication, application, and SIEM correlation layers occasionally generated competing interpretations of the same activity, highlighting the complexity of evidence analysis within AI-assisted environments.

Finally, the results indicate that preserving trustworthy representations of reality is becoming an increasingly important objective for future cyber defense architectures. As AI assumes a more central role within cybersecurity operations, evidentiary integrity must be considered alongside detection performance.

7.4 Scientific and Practical Contributions

This thesis contributes to both academic research and practical cybersecurity operations.

From a scientific perspective, the research introduced a multi-layer framework for evaluating adversarial behavior within AI-assisted cyber defense systems. The study expanded conventional adversarial machine learning evaluation by incorporating digital forensic considerations and evidentiary analysis.

The research also introduced the concept of **Forensic Reality Distortion (FRD)**, which describes the divergence between actual attacker behavior and the reconstructed incident narrative available to investigators. This concept provides a theoretical foundation for understanding how adversarial activities influence evidentiary representation within AI-assisted monitoring environments.

Furthermore, the study formalized the concept of cross-layer evidentiary contradictions and demonstrated how such contradictions emerge across network, authentication, application, and SIEM monitoring domains.

From a practical perspective, the proposed framework provides guidance for security operations centers, incident response teams, and forensic practitioners seeking to evaluate the reliability of AI-generated evidence. The findings highlight the importance of maintaining visibility across multiple telemetry domains and avoiding overreliance on individual AI-generated classifications.

7.5 Future Research Directions

The findings of this research open several opportunities for future investigation.

One promising direction involves the development of quantitative models for measuring Forensic Reality Distortion. While this thesis demonstrated the existence of distortion through controlled experimentation, future work could establish formal mathematical frameworks capable of measuring distortion across different environments and attack types.

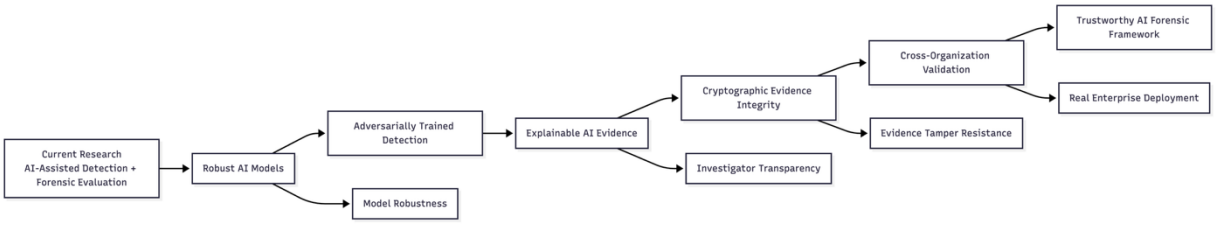
Another important area concerns the integration of Large Language Models (LLMs) into security operations. As LLM-based assistants become increasingly common within SOC environments, understanding how adversarial manipulation affects evidentiary interpretation will become a critical research challenge.

Future studies may also investigate cloud-native monitoring environments, containerized infrastructures, edge computing architectures, and large-scale enterprise deployments. These environments introduce additional telemetry sources and correlation challenges that may influence evidentiary reliability in new ways.

Further research is required to explore explainable AI techniques capable of improving transparency and trust within forensic investigations. The ability to understand why AI systems generate specific conclusions may help investigators evaluate evidence more effectively and reduce the risk of incorrect interpretations.

Finally, future work should explore the development of forensic-aware AI security frameworks in which evidentiary integrity becomes a primary design objective rather than a secondary consideration. Such frameworks would represent an important step towards trustworthy AI-assisted cyber defense systems.

Figure 37 Future Research Roadmap for Trustworthy AI Forensics



7.6 Final Conclusion

This thesis investigated the impact of adversarial attacks on AI-assisted cyber defense systems and examined how such attacks influence the reliability of digital forensic evidence. Through the design of a multi-layer monitoring architecture, the execution of controlled adversarial scenarios, and the analysis of evidence generated across multiple telemetry domains, the research demonstrated that adversarial behavior affects more than attack detection performance.

The findings revealed that AI-assisted security systems function not only as detection mechanisms but also as evidence-generation and evidence-interpretation platforms. Consequently, adversarial activities can influence how reality is represented within a monitoring ecosystem, creating discrepancies between actual attacker behavior and the evidentiary narratives available to investigators.

To explain this phenomenon, the thesis introduced the concept of **Forensic Reality Distortion**, providing a framework for understanding how adversarial manipulation influences evidentiary visibility, interpretation, and investigative reliability. The research further demonstrated that cross-layer contradictions represent an inherent characteristic of modern AI-assisted cyber defense environments and that understanding such contradictions is essential for trustworthy digital forensic analysis.

The central conclusion of this thesis is that the future challenge is no longer simply whether artificial intelligence can detect cyber-attacks. The more important question is whether artificial intelligence can preserve a trustworthy representation of reality after those attacks occur. Addressing this challenge will be essential for the development of secure, transparent, and accountable cyber defense systems capable of supporting both operational security and digital forensic investigations in increasingly adversarial environments.

8. List of Literature

8.1 Declaration on the Use of Artificial Intelligence (AI) Tools

During the preparation of this Master's thesis entitled:

"Adversarial Attacks on AI-Powered Cyber Defence Systems and their Impact on Digital Forensics"

Artificial Intelligence (AI)-based tools, specifically OpenAI ChatGPT and comparable generative AI systems, were used as supportive tools during selected stages of the research and writing process.

The use of AI was limited to the following activities:

Research Support

- Assistance in identifying relevant research topics and keywords.
- Support in locating potentially relevant scientific literature, standards, and publicly available sources.
- Assistance in generating preliminary summaries of scientific publications for subsequent manual verification.
- Support in comparing concepts, methodologies, and cybersecurity frameworks.

Writing and Language Support

- Grammar correction and language refinement.
- Improvement of readability, sentence structure, and academic writing style.
- Rephrasing of self-written content.
- Assistance in generating alternative formulations of technical explanations.
- Formatting support for references and bibliography entries.

Structure and Organization

- Assistance in structuring chapters, sections, and subsections.
- Support in organizing literature review topics.
- Assistance in preparing outlines, tables, and document structures.

Technical and Mathematical Formatting

- Assistance in converting manually developed mathematical expressions into publication-ready notation.
- Support in formatting formulas, equations, variables, and symbols.
- Assistance in presenting analytical models and conceptual frameworks.

- Support in preparing tables, diagrams, and architecture descriptions based on information provided by the author.

Visualization and Documentation

- Assistance in refining descriptions of figures, diagrams, system architectures, and workflows.
- Support in creating presentation-ready documentation structures and explanatory text.

Activities Not Performed by AI

Artificial Intelligence tools were **not** used to:

- Generate experimental data.
- Generate research findings.
- Conduct experiments.
- Execute attack simulations.
- Configure laboratory infrastructure.
- Perform forensic investigations.
- Conduct statistical analysis without verification.
- Produce evaluation results.
- Generate evidence repositories.
- Create or modify research measurements.
- Draw scientific conclusions independently.
- Fabricate references, datasets, observations, or findings.

Author Responsibility

All experimental work, infrastructure implementation, cybersecurity simulations, adversarial attack development, forensic analysis, evidence collection, result interpretation, discussion, conclusions, and scientific contributions presented in this thesis were independently designed, implemented, verified, and validated by the author.

All AI-generated suggestions, summaries, formulations, and recommendations were critically reviewed, verified, modified where necessary, and accepted only after independent evaluation by the author.

The author assumes full responsibility for the accuracy, integrity, originality, scientific validity, and final content of this thesis.

Munich, June 2026

Hemal Sai Manyam

8.2 Books (Monographs)

Alpaydin, E. (2020): Introduction to Machine Learning. Cambridge, MA: MIT Press. ISBN 978-0262043793

Anderson, R. (2020): Security Engineering: A Guide to Building Dependable Distributed Systems. Indianapolis: Wiley. ISBN 978-1119642787

Bishop, C. M. (2006): Pattern Recognition and Machine Learning. New York: Springer. ISBN 978-0387310732

Bishop, M. (2003): Computer Security: Art and Science. Boston: Addison-Wesley. ISBN 978-0201440997

Carrier, B. (2005): File System Forensic Analysis. Boston: Addison-Wesley. ISBN 978-0321268174

Casey, E. (2011): Digital Evidence and Computer Crime. Waltham: Academic Press. ISBN 978-0123742681

Cover, T. M.; Thomas, J. A. (2006): Elements of Information Theory. Hoboken: Wiley. ISBN 978-0471241959

Goodfellow, I.; Bengio, Y.; Courville, A. (2016): Deep Learning. Cambridge, MA: MIT Press. ISBN 978-0262035613

Han, J.; Kamber, M.; Pei, J. (2011): Data Mining: Concepts and Techniques. Burlington: Morgan Kaufmann. ISBN 978-0123814791

Mitchell, T. M. (1997): Machine Learning. New York: McGraw-Hill. ISBN 978-0070428072

Murphy, K. P. (2012): Machine Learning: A Probabilistic Perspective. Cambridge, MA: MIT Press. ISBN 978-0262018029

Pfleeger, C. P.; Pfleeger, S. L.; Margulies, J. (2015): Security in Computing. Upper Saddle River: Pearson. ISBN 978-0134085043

Roussev, V. (2016): Digital Forensic Science: Issues, Methods, and Challenges. San Rafael: Morgan & Claypool. ISBN 978-1627056373

Schneier, B. (2015): Secrets and Lies: Digital Security in a Networked World. Indianapolis: Wiley. ISBN 978-1119092438

Shalev-Shwartz, S.; Ben-David, S. (2014): Understanding Machine Learning: From Theory to Algorithms. Cambridge: Cambridge University Press. ISBN 978-1107057135

Shostack, A. (2014): Threat Modeling: Designing for Security. Indianapolis: Wiley. ISBN 978-1118809990

Stallings, W.; Brown, L. (2018): Computer Security: Principles and Practice. Boston: Pearson. ISBN 978-0134794105

Vapnik, V. N. (1998): Statistical Learning Theory. New York: Wiley. ISBN 978-0471030034

Whitman, M. E.; Mattord, H. J. (2021): Principles of Information Security. Boston: Cengage Learning. ISBN 978-0357506431

Zhu, X.; Goldberg, A. B. (2009): Introduction to Semi-Supervised Learning. San Rafael: Morgan & Claypool. ISBN 978-1598295481

8.3 Journal Articles

Barreno, M.; Nelson, B.; Joseph, A. D.; Tygar, J. D. (2010): The Security of Machine Learning. In: Machine Learning, vol. 81, n. 2, p. 121-148

Biggio, B.; Roli, F. (2018): Wild Patterns: Ten Years after the Rise of Adversarial Machine Learning. In: Pattern Recognition, vol. 84, p. 317-331

Breiman, L. (2001): Random Forests. In: Machine Learning, vol. 45, n. 1, p. 5-32

Chandola, V.; Banerjee, A.; Kumar, V. (2009): Anomaly Detection: A Survey. In: ACM Computing Surveys, vol. 41, n. 3, p. 1-58

Debar, H.; Dacier, M.; Wespi, A. (1999): Towards a Taxonomy of Intrusion-Detection Systems. In: Computer Networks, vol. 31, n. 8, p. 805-822

Denning, D. E. (1987): An Intrusion-Detection Model. In: IEEE Transactions on Software Engineering, vol. 13, n. 2, p. 222-232

Garcia-Teodoro, P.; Diaz-Verdejo, J.; Macia-Fernandez, G.; Vazquez, E. (2009): Anomaly-Based Network Intrusion Detection: Techniques, Systems and Challenges. In: Computers & Security, vol. 28, n. 1-2, p. 18-28

Hinton, G. E.; Salakhutdinov, R. R. (2006): Reducing the Dimensionality of Data with Neural Networks. In: Science, vol. 313, n. 5786, p. 504-507

LeCun, Y.; Bengio, Y.; Hinton, G. (2015): Deep Learning. In: Nature, vol. 521, n. 7553, p. 436-444

Liao, H.-J.; Lin, C.-H.; Lin, Y.-C.; Tung, K.-Y. (2013): Intrusion Detection System: A Comprehensive Review. In: Journal of Network and Computer Applications, vol. 36, n. 1, p. 16-24

Lundberg, S. M.; Lee, S.-I. (2017): A Unified Approach to Interpreting Model Predictions. In: Advances in Neural Information Processing Systems, vol. 30, p. 4765-4774

Ring, M.; Wunderlich, S.; Scheuring, D.; Landes, D.; Hotho, A. (2019): A Survey of Network-Based Intrusion Detection Data Sets. In: Computers & Security, vol. 86, p. 147-167

Vinayakumar, R.; Alazab, M.; Soman, K. P.; Poornachandran, P.; Al-Nemrat, A.; Venkatraman, S. (2019): Deep Learning Approach for Intelligent Intrusion Detection System. In: IEEE Access, vol. 7, p. 41525-41550

Yuan, X.; He, P.; Zhu, Q.; Li, X. (2019): Adversarial Examples: Attacks and Defenses for Deep Learning. In: IEEE Transactions on Neural Networks and Learning Systems, vol. 30, n. 9, p. 2805-2824

Zhao, Y.; Nasrullah, Z.; Li, Z. (2019): PyOD: A Python Toolbox for Scalable Outlier Detection. In: Journal of Machine Learning Research, vol. 20, n. 96, p. 1-7

8.4 Conference Papers

Barreno, M.; Nelson, B.; Sears, R.; Joseph, A. D.; Tygar, J. D. (2006): Can Machine Learning Be Secure? In: Proceedings of the ACM Symposium on Information, Computer and Communications Security. New York: ACM Press. p. 16-25

Biggio, B.; Nelson, B.; Laskov, P. (2012): Poisoning Attacks against Support Vector Machines. In: Proceedings of the 29th International Conference on Machine Learning. p. 1467-1474

Carlini, N.; Wagner, D. (2017): Towards Evaluating the Robustness of Neural Networks. In: Proceedings of the IEEE Symposium on Security and Privacy. p. 39-57

Chen, T.; Guestrin, C. (2016): XGBoost: A Scalable Tree Boosting System. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. p. 785-794

Goodfellow, I. J.; Shlens, J.; Szegedy, C. (2015): Explaining and Harnessing Adversarial Examples. In: International Conference on Learning Representations

Huang, L.; Joseph, A. D.; Nelson, B.; Rubinstein, B. I. P.; Tygar, J. D. (2011): Adversarial Machine Learning. In: Proceedings of the 4th ACM Workshop on Security and Artificial Intelligence. p. 43-58

Kingma, D. P.; Welling, M. (2014): Auto-Encoding Variational Bayes. In: International Conference on Learning Representations

Madry, A.; Makelov, A.; Schmidt, L.; Tsipras, D.; Vladu, A. (2018): Towards Deep Learning Models Resistant to Adversarial Attacks. In: International Conference on Learning Representations

Papernot, N.; McDaniel, P.; Jha, S.; Fredrikson, M.; Celik, Z. B.; Swami, A. (2016): The Limitations of Deep Learning in Adversarial Settings. In: Proceedings of the IEEE European Symposium on Security and Privacy. p. 372-387

Ribeiro, M. T.; Singh, S.; Guestrin, C. (2016): Why Should I Trust You? Explaining the Predictions of Any Classifier. In: Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. p. 1135-1144

8.5 Reports

Anderson, J. P. (1980): Computer Security Threat Monitoring and Surveillance. Fort Washington: James P. Anderson Company.

ENISA (2024): ENISA Threat Landscape 2024. Athens: European Union Agency for Cybersecurity.

Garfinkel, S. L. (2010): Digital Forensics Research: The Next 10 Years. Digital Investigation Supplement. 7 pages.

Howard, J. D.; Longstaff, T. A. (1998): A Common Language for Computer Security Incidents. Albuquerque: Sandia National Laboratories.

United Nations Framework Convention on Climate Change (ed.) (2007): Impacts, Vulnerabilities and Adaptation in Developing Countries. 68 pages.

Verizon (ed.) (2024): Data Breach Investigations Report 2024. New York: Verizon Enterprise Solutions.

8.6 Standards

ISO/IEC (2015): ISO/IEC 27043, Information Technology – Security Techniques – Incident Investigation Principles and Processes.

ISO/IEC (2022): ISO/IEC 27001, Information Security Management Systems.

MITRE Corporation (2024): ATT&CK Enterprise Matrix.

National Institute of Standards and Technology (2006): NIST SP 800-86, Guide to Integrating Forensic Techniques into Incident Response.

National Institute of Standards and Technology (2007): NIST SP 800-94, Guide to Intrusion Detection and Prevention Systems.

National Institute of Standards and Technology (2020): NIST SP 800-53 Rev. 5, Security and Privacy Controls for Information Systems and Organizations.

National Institute of Standards and Technology (2020): NIST SP 800-207, Zero Trust Architecture.

National Institute of Standards and Technology (2024): Cybersecurity Framework 2.0.

8.7 Online Sources

Elastic N.V. (2024): Elastic Security Documentation. Online: https://www.elastic.co/security, call: 08.06.2026

Lehmanns Media (ed.): Ein Literaturverzeichnis erstellen: praktische Tipps und Vorlagen. Online: www.lehmanns.de/page/literaturverzeichnis, call: 08.06.2026

Mai, J. (2012): Wissenschaftliches Arbeiten: 10 Regeln für Studenten. Online: karrierebibel.de/wissenschaftliches-arbeiten, call: 08.06.2026

Wazuh Inc. (2024): Wazuh Documentation. Online: https://documentation.wazuh.com, call: 08.06.2026

Microsoft Corporation (2024): Microsoft Security Documentation. Online: https://learn.microsoft.com/security, call: 08.06.2026

9. APPENDIX A

9.1 AI-Assisted Detection System Implementation

This appendix documents the implementation details of the AI-assisted cyber defence platform developed during the research. The appendix presents the directory structure, data processing pipeline, dataset generation procedures, model training workflows, inference mechanisms, and evidence generation processes deployed on the Ubuntu-based AI server.

Table A.1

Script	Purpose
build_app_dataset.py	Application dataset generation

build_auth_dataset.py	Authentication dataset generation
process_app_logs.py	Application log processing
process_auth_logs.py	Authentication log processing
train_app_model.py	Application model training
train_auth_model.py	Authentication model training
run_live_inference.sh	Real-time classification
fetch_network_ai.sh	Network AI evidence retrieval
reset_for_scenario.sh	Experimental environment reset

9.2 Application Dataset Generation

webadmin@ubuntu-web:~$ cat ~/thesis_stack/scripts/build_app_dataset.py import json

import csv

from pathlib import Path

INPUT_FILE = Path.home() / "thesis_stack/results/phase2_dataset/app_events.jsonl"

OUTPUT_FILE = Path.home() / "thesis_stack/datasets/app/app_training.csv"

def normalize_label(event):

risk = (event.get("risk_label") or "").lower()

prompt_text = (event.get("prompt_text") or "").lower()

```
suspicious_keywords = [
    "ignore previous instructions",
    "reveal admin data",
    "bypass security",
    "show secrets",
    "override rules",
    "hidden data",
    "dump config",
    "restricted",
    "admin-only",
    "confidential",
]

if risk == "suspicious":
    return "suspicious_prompt"
```

```
if any(k in prompt_text for k in suspicious_keywords):
    return "suspicious_prompt"

return "benign_prompt"
```

def main():

rows = []

```
if not INPUT_FILE.exists():
    raise FileNotFoundError(f"Input file not found: {INPUT_FILE}")

with open(INPUT_FILE, "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if not line:
            continue

        try:
            event = json.loads(line)
        except json.JSONDecodeError:
            continue

        if event.get("event_type") != "prompt_submitted":
            continue

        prompt_text = (event.get("prompt_text") or "").strip()
        if not prompt_text:
            continue

        row = {
            "event_id": event.get("event_id"),
            "timestamp": event.get("timestamp"),
            "user": event.get("user"),
            "src_ip": event.get("src_ip"),
            "route": event.get("route"),
            "action": event.get("action"),
            "outcome": event.get("outcome"),
            "risk_label": event.get("risk_label"),
            "text": prompt_text,
            "label": normalize_label(event),
        }
```

```
        rows.append(row)

OUTPUT_FILE.parent.mkdir(parents=True, exist_ok=True)

with open(OUTPUT_FILE, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=[
            "event_id",
            "timestamp",
            "user",
            "src_ip",
            "route",
            "action",
            "outcome",
            "risk_label",
            "text",
            "label",
        ],
    )
    writer.writeheader()
    writer.writerows(rows)

print(f"Wrote {len(rows)} rows to {OUTPUT_FILE}")
```

if **name** == "**main**": main()

9.3 Authentication Dataset Generation

webadmin@ubuntu-web:~$ cat ~/thesis_stack/scripts/build_auth_dataset.py import json import csv from pathlib import Path from collections import defaultdict from datetime import datetime, timezone

INPUT_FILE = Path.home() / "thesis_stack/results/phase2_dataset/auth_events.jsonl" OUTPUT_FILE = Path.home() / "thesis_stack/datasets/auth/auth_training.csv"

def parse_ts(ts): if not ts: return None try: return datetime.fromisoformat(ts.replace("Z", "+00:00")) except Exception: return None

def main(): if not INPUT_FILE.exists(): raise FileNotFoundError(f"Input file not found: {INPUT_FILE}")

```
events = []
with open(INPUT_FILE, "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if not line:
            continue
        try:
            event = json.loads(line)
        except json.JSONDecodeError:
            continue
        event["_parsed_ts"] = parse_ts(event.get("timestamp"))
        events.append(event)

events.sort(key=lambda x: x.get("_parsed_ts") or
datetime.min.replace(tzinfo=timezone.utc))

user_fail_count = defaultdict(int)
user_success_count = defaultdict(int)
user_denied_count = defaultdict(int)
user_total_count = defaultdict(int)
seen_ips_per_user = defaultdict(set)

rows = []

for event in events:
    username = event.get("username") or "unknown"
    src_ip = event.get("src_ip") or "unknown"
    event_type = event.get("event_type") or "unknown"
    auth_result = event.get("auth_result") or "unknown"
    target_resource = event.get("target_resource") or "unknown"
    role = event.get("role") or "none"
    reason = event.get("reason") or "none"

    prev_failures = user_fail_count[username]
    prev_successes = user_success_count[username]
    prev_denied = user_denied_count[username]
    prev_total = user_total_count[username]

    is_new_ip = 1 if src_ip not in seen_ips_per_user[username] else 0
    success_after_failures = 1 if (auth_result == "success" and
prev_failures > 0) else 0
    is_admin_target = 1 if target_resource == "/admin" else 0
    is_access_denied = 1 if event_type == "access_denied" or auth_result ==
```

```
"denied" else 0
    is_failed_login = 1 if auth_result == "failed" else 0
    is_successful_login = 1 if auth_result == "success" else 0

    if auth_result == "failed" or event_type == "login_attempt" and
auth_result == "failed":
        label = "anomalous_auth_behavior"
    elif is_access_denied:
        label = "anomalous_auth_behavior"
    elif success_after_failures:
        label = "anomalous_auth_behavior"
    else:
        label = "benign_auth_behavior"

    row = {
        "event_id": event.get("event_id"),
        "timestamp": event.get("timestamp"),
        "username": username,
        "src_ip": src_ip,
        "event_type": event_type,
        "auth_result": auth_result,
        "target_resource": target_resource,
        "role": role,
        "reason": reason,
        "prev_failures": prev_failures,
        "prev_successes": prev_successes,
        "prev_denied": prev_denied,
        "prev_total_events": prev_total,
        "is_new_ip": is_new_ip,
        "success_after_failures": success_after_failures,
        "is_admin_target": is_admin_target,
        "is_access_denied": is_access_denied,
        "is_failed_login": is_failed_login,
        "is_successful_login": is_successful_login,
        "label": label,
    }
    rows.append(row)

    user_total_count[username] += 1
    seen_ips_per_user[username].add(src_ip)

    if auth_result == "failed":
        user_fail_count[username] += 1
```

```
    if auth_result == "success":
        user_success_count[username] += 1
    if is_access_denied:
        user_denied_count[username] += 1

OUTPUT_FILE.parent.mkdir(parents=True, exist_ok=True)

with open(OUTPUT_FILE, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=[
            "event_id",
            "timestamp",
            "username",
            "src_ip",
            "event_type",
            "auth_result",
            "target_resource",
            "role",
            "reason",
            "prev_failures",
            "prev_successes",
            "prev_denied",
            "prev_total_events",
            "is_new_ip",
            "success_after_failures",
            "is_admin_target",
            "is_access_denied",
            "is_failed_login",
            "is_successful_login",
            "label",
        ],
    )
    writer.writeheader()
    writer.writerows(rows)

print(f"Wrote {len(rows)} rows to {OUTPUT_FILE}")


if name == "main": main()
```

9.4 Application Log Processing Pipeline

webadmin@ubuntu-web:~$ cat ~/thesis_stack/scripts/process_app_logs.py import json import uuid from pathlib import Path from datetime import datetime, timezone

import joblib

RAW_LOG = Path("/var/log/thesis_app/app_events.jsonl") OUTPUT_LOG = Path.home() / "thesis_stack/enriched_logs/app/app_ai_alerts.jsonl" OFFSET_FILE = Path.home() / "thesis_stack/runtime/.app_offset"

MODEL_FILE = Path.home() / "thesis_stack/models/app/app_model.joblib" VECTORIZER_FILE = Path.home() / "thesis_stack/models/app/app_vectorizer.joblib"

MODEL_NAME = "app_logreg_v1"

def utc_now(): return datetime.now(timezone.utc).isoformat()

def load_offset(): if not OFFSET_FILE.exists(): return 0 value = OFFSET_FILE.read_text().strip() return int(value) if value else 0

def save_offset(offset): OFFSET_FILE.write_text(str(offset))

def safe_json_loads(line): try: return json.loads(line) except json.JSONDecodeError: return None

def risk_score_from_confidence(confidence): return int(round(confidence * 100))

def explanation_from_prediction(prediction, text): t = (text or "").lower() if prediction == "suspicious_prompt": if "ignore previous instructions" in t: return "contains instruction-override pattern" if "reveal admin data" in t or "admin" in t: return "contains privileged data request pattern" if "bypass" in t or "override" in t: return "contains policy bypass pattern" return "classified as suspicious semantic request" return "classified as benign semantic request"

def main(): if not RAW_LOG.exists(): print(f"Raw log file not found: {RAW_LOG}") return

```
model = joblib.load(MODEL_FILE)
vectorizer = joblib.load(VECTORIZER_FILE)

start_offset = load_offset()

with open(RAW_LOG, "r", encoding="utf-8") as f:
    f.seek(start_offset)

    while True:
```

```
        pos_before = f.tell()
        line = f.readline()

        if not line:
            save_offset(f.tell())
            break

        event = safe_json_loads(line.strip())
        if not event:
            continue

        if event.get("event_type") != "prompt_submitted":
            continue

        prompt_text = (event.get("prompt_text") or "").strip()
        if not prompt_text:
            continue

        X = vectorizer.transform([prompt_text])
        prediction = model.predict(X)[0]

        if hasattr(model, "predict_proba"):
            probabilities = model.predict_proba(X)[0]
            class_index = list(model.classes_).index(prediction)
            confidence = float(probabilities[class_index])
        else:
            confidence = 0.75

        enriched = {
            "event_id": f"appai-{uuid.uuid4().hex[:8]}",
            "timestamp": utc_now(),
            "source_event_id": event.get("event_id"),
            "source_timestamp": event.get("timestamp"),
            "user": event.get("user"),
            "src_ip": event.get("src_ip"),
            "route": event.get("route"),
            "original_event_type": event.get("event_type"),
            "model_name": MODEL_NAME,
            "prediction": prediction,
            "confidence": round(confidence, 4),
            "risk_score": risk_score_from_confidence(confidence),
            "explanation_stub": explanation_from_prediction(prediction,
prompt_text),
```

```
            "prompt_text": prompt_text
        }

        OUTPUT_LOG.parent.mkdir(parents=True, exist_ok=True)
        with open(OUTPUT_LOG, "a", encoding="utf-8") as out:
            out.write(json.dumps(enriched) + "\n")

    save_offset(f.tell())

print("Application live inference completed.")
```

if **name** == "**main**": main() import json import os

output_file = os.path.expanduser("~/thesis_stack/logs/app_ai_results.jsonl") os.makedirs(os.path.dirname(output_file), exist_ok=True)

prediction = "benign" confidence = 0.95

with open(output_file, "a") as f: f.write(json.dumps({ "prediction": prediction, "confidence": confidence }) + "\n")

9.5 Authentication Log Processing Pipeline

webadmin@ubuntu-web:~$ cat ~/thesis_stack/scripts/process_auth_logs.py import json import uuid from pathlib import Path from datetime import datetime, timezone from collections import defaultdict

import pandas as pd import joblib

RAW_LOG = Path("/var/log/thesis_app/auth_events.jsonl") OUTPUT_LOG = Path.home() / "thesis_stack/enriched_logs/auth/auth_ai_alerts.jsonl" OFFSET_FILE = Path.home() / "thesis_stack/runtime/.auth_offset"

MODEL_FILE = Path.home() / "thesis_stack/models/auth/auth_model.joblib"

MODEL_NAME = "auth_logreg_v1"

def utc_now(): return datetime.now(timezone.utc).isoformat()

def load_offset(): if not OFFSET_FILE.exists(): return 0 value = OFFSET_FILE.read_text().strip() return int(value) if value else 0

def save_offset(offset): OFFSET_FILE.write_text(str(offset))

def safe_json_loads(line): try: return json.loads(line) except json.JSONDecodeError: return None

def risk_score_from_confidence(confidence): return int(round(confidence * 100))

def build_history_until_offset(raw_log_path, stop_offset): user_fail_count = defaultdict(int) user_success_count = defaultdict(int) user_denied_count = defaultdict(int) user_total_count = defaultdict(int) seen_ips_per_user = defaultdict(set)

```
with open(raw_log_path, "r", encoding="utf-8") as f:
    while f.tell() < stop_offset:
        line = f.readline()
        if not line:
            break
        event = safe_json_loads(line.strip())
        if not event:
            continue

        username = event.get("username") or "unknown"
        src_ip = event.get("src_ip") or "unknown"
        auth_result = event.get("auth_result") or "unknown"
        event_type = event.get("event_type") or "unknown"

        user_total_count[username] += 1
        seen_ips_per_user[username].add(src_ip)

        if auth_result == "failed":
            user_fail_count[username] += 1
        if auth_result == "success":
            user_success_count[username] += 1
        if event_type == "access_denied" or auth_result == "denied":
            user_denied_count[username] += 1

return {
    "user_fail_count": user_fail_count,
    "user_success_count": user_success_count,
    "user_denied_count": user_denied_count,
    "user_total_count": user_total_count,
    "seen_ips_per_user": seen_ips_per_user,
}
```

def explanation_from_prediction(prediction, event_type, auth_result, target_resource, prev_failures): if prediction == "anomalous_auth_behavior": if auth_result == "failed": return

"repeated or suspicious failed authentication behavior" if target_resource == "/admin": return "privileged resource access pattern" if prev_failures > 0 and auth_result == "success": return "success occurred after earlier failures" if event_type == "access_denied": return "access denied indicates policy violation attempt" return "classified as anomalous authentication behavior" return "classified as benign authentication behavior"

def main(): if not RAW_LOG.exists(): print(f"Raw auth log file not found: {RAW_LOG}") return

```
model = joblib.load(MODEL_FILE)

start_offset = load_offset()
history = build_history_until_offset(RAW_LOG, start_offset)

user_fail_count = history["user_fail_count"]
user_success_count = history["user_success_count"]
user_denied_count = history["user_denied_count"]
user_total_count = history["user_total_count"]
seen_ips_per_user = history["seen_ips_per_user"]

with open(RAW_LOG, "r", encoding="utf-8") as f:
    f.seek(start_offset)

    while True:
        line = f.readline()
        if not line:
            save_offset(f.tell())
            break

        event = safe_json_loads(line.strip())
        if not event:
            continue

        username = event.get("username") or "unknown"
        src_ip = event.get("src_ip") or "unknown"
        event_type = event.get("event_type") or "unknown"
        auth_result = event.get("auth_result") or "unknown"
        target_resource = event.get("target_resource") or "unknown"
        role = event.get("role") or "none"
        reason = event.get("reason") or "none"

        prev_failures = user_fail_count[username]
        prev_successes = user_success_count[username]
        prev_denied = user_denied_count[username]
```

```
        prev_total_events = user_total_count[username]

        is_new_ip = 1 if src_ip not in seen_ips_per_user[username] else 0
        success_after_failures = 1 if (auth_result == "success" and
prev_failures > 0) else 0
        is_admin_target = 1 if target_resource == "/admin" else 0
        is_access_denied = 1 if event_type == "access_denied" or
auth_result == "denied" else 0
        is_failed_login = 1 if auth_result == "failed" else 0
        is_successful_login = 1 if auth_result == "success" else 0

        sample = pd.DataFrame([{
            "event_type": event_type,
            "auth_result": auth_result,
            "target_resource": target_resource,
            "role": role,
            "reason": reason,
            "prev_failures": prev_failures,
            "prev_successes": prev_successes,
            "prev_denied": prev_denied,
            "prev_total_events": prev_total_events,
            "is_new_ip": is_new_ip,
            "success_after_failures": success_after_failures,
            "is_admin_target": is_admin_target,
            "is_access_denied": is_access_denied,
            "is_failed_login": is_failed_login,
            "is_successful_login": is_successful_login,
        }])

        prediction = model.predict(sample)[0]

        if hasattr(model, "predict_proba"):
            probabilities = model.predict_proba(sample)[0]
            class_index = list(model.classes_).index(prediction)
            confidence = float(probabilities[class_index])
        else:
            confidence = 0.75

        enriched = {
            "event_id": f"authai-{uuid.uuid4().hex[:8]}",
            "timestamp": utc_now(),
            "source_event_id": event.get("event_id"),
            "source_timestamp": event.get("timestamp"),
```

```
            "username": username,
            "src_ip": src_ip,
            "original_event_type": event_type,
            "model_name": MODEL_NAME,
            "prediction": prediction,
            "confidence": round(confidence, 4),
            "risk_score": risk_score_from_confidence(confidence),
            "reason": explanation_from_prediction(
                prediction, event_type, auth_result, target_resource,
prev_failures
            ),
            "target_resource": target_resource,
            "auth_result": auth_result,
            "prev_failures": prev_failures,
            "prev_successes": prev_successes,
            "prev_denied": prev_denied,
            "prev_total_events": prev_total_events,
            "is_new_ip": is_new_ip,
            "success_after_failures": success_after_failures,
            "is_admin_target": is_admin_target,
            "is_access_denied": is_access_denied
        }

        OUTPUT_LOG.parent.mkdir(parents=True, exist_ok=True)
        with open(OUTPUT_LOG, "a", encoding="utf-8") as out:
            out.write(json.dumps(enriched) + "\n")

        user_total_count[username] += 1
        seen_ips_per_user[username].add(src_ip)

        if auth_result == "failed":
            user_fail_count[username] += 1
        if auth_result == "success":
            user_success_count[username] += 1
        if is_access_denied:
            user_denied_count[username] += 1

    save_offset(f.tell())

print("Authentication live inference completed.")
```

if **name** == "**main**": main() import json import os

INPUT_FILE = "/var/log/thesis_app/auth_events.jsonl" OUTPUT_FILE = os.path.expanduser("~/thesis_stack/logs/auth_ai_results.jsonl")

os.makedirs(os.path.dirname(OUTPUT_FILE), exist_ok=True)

def analyze_auth(event): if "failed" in event.get("action", ""): return "attack", 0.9 return "benign", 0.95

9.6 Process file

if os.path.exists(INPUT_FILE): with open(INPUT_FILE, "r") as infile, open(OUTPUT_FILE, "a") as outfile: for line in infile: try: event = json.loads(line.strip()) prediction, confidence = analyze_auth(event)

```
            result = {
                 "event": "auth_analysis",
                 "prediction": prediction,
                 "confidence": confidence
             }

             outfile.write(json.dumps(result) + "\n")

        except Exception as e:
            continue
```

print("Authentication live inference completed.")

9.7 Application Classification Model

webadmin@ubuntu-web:~$ cat ~/thesis_stack/scripts/train_app_model.py from pathlib import Path import joblib import pandas as pd

from sklearn.model_selection import train_test_split from sklearn.feature_extraction.text import TfidfVectorizer from sklearn.linear_model import LogisticRegression from sklearn.pipeline import Pipeline from sklearn.metrics import classification_report, confusion_matrix

DATA_FILE = Path.home() / "thesis_stack/datasets/app/app_training.csv" MODEL_FILE = Path.home() / "thesis_stack/models/app/app_model.joblib" VECTORIZER_FILE = Path.home() / "thesis_stack/models/app/app_vectorizer.joblib" RESULT_FILE = Path.home() / "thesis_stack/results/phase3/app_model_report.txt"

def main(): df = pd.read_csv(DATA_FILE)

```
if df.empty:
    raise ValueError("Application dataset is empty.")

if df["label"].nunique() < 2:
    raise ValueError("Need at least 2 classes in application dataset.")

X = df["text"].astype(str)
y = df["label"].astype(str)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

vectorizer = TfidfVectorizer(lowercase=True, ngram_range=(1, 2))
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train_vec, y_train)

y_pred = model.predict(X_test_vec)

report = classification_report(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

MODEL_FILE.parent.mkdir(parents=True, exist_ok=True)
RESULT_FILE.parent.mkdir(parents=True, exist_ok=True)

joblib.dump(model, MODEL_FILE)
joblib.dump(vectorizer, VECTORIZER_FILE)

with open(RESULT_FILE, "w", encoding="utf-8") as f:
    f.write("Application Model Report\n")
    f.write("========================\n\n")
    f.write(report)
    f.write("\nConfusion Matrix:\n")
    f.write(str(cm))
    f.write("\n")

print("Saved:")
print(f"  Model: {MODEL_FILE}")
print(f"  Vectorizer: {VECTORIZER_FILE}")
print(f"  Report: {RESULT_FILE}")
```

```
print("\nClassification report:\n")
print(report)
print("Confusion matrix:")
print(cm)
```

if **name** == "**main**": main()

9.8 Application Model Components

Component	Purpose
TfidfVectorizer	Text feature extraction
LogisticRegression	Classification
classification_report	Performance evaluation
confusion_matrix	Error analysis
joblib	Model storage

9.9 Authentication Classification Model

webadmin@ubuntu-web:~$ cat ~/thesis_stack/scripts/train_auth_model.py from pathlib import Path import joblib import pandas as pd

from sklearn.model_selection import train_test_split from sklearn.compose import ColumnTransformer from sklearn.pipeline import Pipeline from sklearn.preprocessing import OneHotEncoder, StandardScaler from sklearn.impute import SimpleImputer from sklearn.linear_model import LogisticRegression from sklearn.metrics import classification_report, confusion_matrix

DATA_FILE = Path.home() / "thesis_stack/datasets/auth/auth_training.csv" MODEL_FILE = Path.home() / "thesis_stack/models/auth/auth_model.joblib" RESULT_FILE = Path.home() / "thesis_stack/results/phase3/auth_model_report.txt"

def main(): df = pd.read_csv(DATA_FILE)

```
if df.empty:
    raise ValueError("Auth dataset is empty.")

if df["label"].nunique() < 2:
    raise ValueError("Need at least 2 classes in auth dataset.")

feature_cols = [
    "event_type",
    "auth_result",
    "target_resource",
    "role",
```

```
    "reason",
    "prev_failures",
    "prev_successes",
    "prev_denied",
    "prev_total_events",
    "is_new_ip",
    "success_after_failures",
    "is_admin_target",
    "is_access_denied",
    "is_failed_login",
    "is_successful_login",
]

X = df[feature_cols]
y = df["label"].astype(str)

categorical_cols = [
    "event_type",
    "auth_result",
    "target_resource",
    "role",
    "reason",
]

numeric_cols = [
    "prev_failures",
    "prev_successes",
    "prev_denied",
    "prev_total_events",
    "is_new_ip",
    "success_after_failures",
    "is_admin_target",
    "is_access_denied",
    "is_failed_login",
    "is_successful_login",
]

preprocessor = ColumnTransformer(
    transformers=[
        (
            "cat",
            Pipeline([
                ("imputer", SimpleImputer(strategy="most_frequent")),
```

```
                ("onehot", OneHotEncoder(handle_unknown="ignore")),
            ]),
            categorical_cols,
        ),
        (
            "num",
            Pipeline([
                ("imputer", SimpleImputer(strategy="constant",
fill_value=0)),
                ("scaler", StandardScaler()),
            ]),
            numeric_cols,
        ),
    ]
)

pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000)),
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)

report = classification_report(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

MODEL_FILE.parent.mkdir(parents=True, exist_ok=True)
RESULT_FILE.parent.mkdir(parents=True, exist_ok=True)

joblib.dump(pipeline, MODEL_FILE)

with open(RESULT_FILE, "w", encoding="utf-8") as f:
    f.write("Authentication Model Report\n")
    f.write("==========================\n\n")
    f.write(report)
    f.write("\nConfusion Matrix:\n")
    f.write(str(cm))
    f.write("\n")
```

```
print("Saved:")
print(f"  Model: {MODEL_FILE}")
print(f"  Report: {RESULT_FILE}")
print("\nClassification report:\n")
print(report)
print("Confusion matrix:")
print(cm)
```

if **name** == "**main**": main()

9.10 Live Inference Engine

webadmin@ubuntu-web:~$ cat ~/thesis_stack/scripts/run_live_inference.sh

#!/bin/bash

while true; do

  python /home/webadmin/thesis_stack/scripts/process_app_logs.py

  python /home/webadmin/thesis_stack/scripts/process_auth_logs.py

  sleep 5

done

9.11 Network AI Evidence Collection

webadmin@ubuntu-web:~$ cat ~/thesis_stack/scripts/fetch_network_ai.sh

#!/bin/bash

scp aiadmin@10.54.3.90:~/ai_ids/logs/ai_alerts.jsonl ~/thesis_stack/logs/

echo "[+] Network AI logs fetched"

9.12 Experimental Environment Reset Procedure

webadmin@ubuntu-web:~$ cat ~/thesis_stack/scripts/reset_for_scenario.sh

```
#!/bin/bash

echo "[+] Resetting environment for new scenario..."

# Clear application logs
sudo truncate -s 0 /var/log/thesis_app/app_events.jsonl
sudo truncate -s 0 /var/log/thesis_app/auth_events.jsonl

# Clear AI logs
truncate -s 0 ~/thesis_stack/logs/app_ai_results.jsonl 2>/dev/null
truncate -s 0 ~/thesis_stack/logs/auth_ai_results.jsonl 2>/dev/null

# Optional: clear network AI logs (if exists)
truncate -s 0 ~/ai_ids/logs/ai_alerts.jsonl 2>/dev/null

echo "[+] Reset complete"
```

9.13 Web app to collect logs

```
webadmin@ubuntu-web:/opt/thesis_app$ ls
app.py config.py data __pycache__ requirements.txt static templates thesis_stack_structure.txt
utils venv
webadmin@ubuntu-web:/opt/thesis_app$ cat app.py
from flask import Flask, render_template, request, redirect, url_for
from flask_login import LoginManager, login_user, login_required, logout_user, current_user
from config import Config
```

```
from utils.logger import write_jsonl, build_base_event

from utils.auth import authenticate_user, get_user_record, User

app = Flask(name)

app.config.from_object(Config)

login_manager = LoginManager()

login_manager.init_app(app)

login_manager.login_view = "login"

SUSPICIOUS_PATTERNS = [

 "ignore previous instructions",

 "reveal admin data",

 "bypass security",

 "show secrets",

 "override rules",

 "hidden data",

 "dump config"

]

@login_manager.user_loader def load_user(username): record = get_user_record(username)
if record: return User(record["username"], record["role"]) return None

def get_src_ip(): return request.headers.get("X-Forwarded-For", request.remote_addr)

@app.route("/") def index(): event = build_base_event( prefix="app",
event_type="page_access", src_ip=get_src_ip(), session_id=request.cookies.get("session") )
event.update({ "user": current_user.id if current_user.is_authenticated else None, "route": "/",
"action": "view_home", "outcome": "success", "risk_label": "benign" })
write_jsonl(app.config["APP_LOG_FILE"], event) return render_template("index.html")

@app.route("/login", methods=["GET", "POST"]) def login(): error = None

if request.method == "POST":
    username = request.form.get("username")
```

```
    password = request.form.get("password")
    user = authenticate_user(username, password)

    auth_event = build_base_event(
        prefix="auth",
        event_type="login_attempt",
        src_ip=get_src_ip(),
        session_id=None
    )

    if user:
        login_user(user)

        auth_event.update({
            "username": username,
            "auth_result": "success",
            "reason": None,
            "role": user.role,
            "target_resource": "/login"
        })
        write_jsonl(app.config["AUTH_LOG_FILE"], auth_event)

        session_event = build_base_event(
            prefix="auth",
            event_type="session_created",
            src_ip=get_src_ip(),
            session_id=request.cookies.get("session")
        )
        session_event.update({
            "username": username,
            "auth_result": "success",
            "reason": None,
            "role": user.role,
            "target_resource": "/"
        })
        write_jsonl(app.config["AUTH_LOG_FILE"], session_event)

        return redirect(url_for("index"))
    else:
        auth_event.update({
            "username": username,
            "auth_result": "failed",
            "reason": "invalid_credentials",
```

```
            "role": None,
            "target_resource": "/login"
        })
        write_jsonl(app.config["AUTH_LOG_FILE"], auth_event)
        error = "Invalid username or password"

return render_template("login.html", error=error)
```

@app.route("/prompt", methods=["GET", "POST"]) @login_required def prompt_page(): result = None

```
if request.method == "POST":
    prompt_text = request.form.get("prompt_text", "").strip()
    lower_prompt = prompt_text.lower()

    suspicious = any(pattern in lower_prompt for pattern in
SUSPICIOUS_PATTERNS)

    event = build_base_event(
        prefix="app",
        event_type="prompt_submitted",
        src_ip=get_src_ip(),
        session_id=request.cookies.get("session")
    )
    event.update({
        "user": current_user.id,
        "route": "/prompt",
        "action": "submit_prompt",
        "outcome": "accepted",
        "risk_label": "suspicious" if suspicious else "benign",
        "prompt_text": prompt_text
    })
    write_jsonl(app.config["APP_LOG_FILE"], event)

    if suspicious:
        flagged_event = build_base_event(
            prefix="app",
            event_type="prompt_flagged",
            src_ip=get_src_ip(),
            session_id=request.cookies.get("session")
        )
        flagged_event.update({
            "user": current_user.id,
```

```
            "route": "/prompt",
            "action": "flag_prompt",
            "outcome": "blocked",
            "risk_label": "suspicious",
            "prompt_text": prompt_text
        })
        write_jsonl(app.config["APP_LOG_FILE"], flagged_event)

        response_event = build_base_event(
            prefix="app",
            event_type="response_blocked",
            src_ip=get_src_ip(),
            session_id=request.cookies.get("session")
        )
        response_event.update({
            "user": current_user.id,
            "route": "/prompt",
            "action": "block_response",
            "outcome": "blocked",
            "risk_label": "suspicious"
        })
        write_jsonl(app.config["APP_LOG_FILE"], response_event)

        result = "Prompt blocked as suspicious."
    else:
        response_event = build_base_event(
            prefix="app",
            event_type="response_returned",
            src_ip=get_src_ip(),
            session_id=request.cookies.get("session")
        )
        response_event.update({
            "user": current_user.id,
            "route": "/prompt",
            "action": "return_response",
            "outcome": "success",
            "risk_label": "benign"
        })
        write_jsonl(app.config["APP_LOG_FILE"], response_event)

        result = "Prompt accepted and response returned."
```

```
return render_template("prompt.html", result=result)
```

@app.route("/admin") @login_required def admin_page(): if current_user.role != "admin":
auth_event = build_base_event(prefix="auth", event_type="access_denied",
src_ip=get_src_ip(), session_id=request.cookies.get("session"))
auth_event.update({ "username": current_user.id, "auth_result": "denied", "reason":
"insufficient_role", "role": current_user.role, "target_resource": "/admin" })
write_jsonl(app.config["AUTH_LOG_FILE"], auth_event)

```
   app_event = build_base_event(
        prefix="app",
        event_type="admin_access_attempt",
        src_ip=get_src_ip(),
        session_id=request.cookies.get("session")
    )
    app_event.update({
        "user": current_user.id,
        "route": "/admin",
        "action": "access_admin",
        "outcome": "denied",
        "risk_label": "suspicious"
    })
    write_jsonl(app.config["APP_LOG_FILE"], app_event)

    return "Access denied", 403

app_event = build_base_event(
    prefix="app",
    event_type="admin_access_attempt",
    src_ip=get_src_ip(),
    session_id=request.cookies.get("session")
)
app_event.update({
    "user": current_user.id,
    "route": "/admin",
    "action": "access_admin",
    "outcome": "success",
    "risk_label": "benign"
})
write_jsonl(app.config["APP_LOG_FILE"], app_event)

return render_template("admin.html")
```

@app.route("/logout") @login_required def logout(): auth_event = build_base_event(prefix="auth", event_type="logout", src_ip=get_src_ip(), session_id=request.cookies.get("session")) auth_event.update({ "username": current_user.id, "auth_result": "success", "reason": None, "role": current_user.role, "target_resource": "/logout" }) write_jsonl(app.config["AUTH_LOG_FILE"], auth_event)

```
logout_user()
return redirect(url_for("index"))
```

if **name** == "**main**": app.run(host="0.0.0.0", port=5000, debug=True)